

Data-Intensive Systems: A Practical Study Guide

Ajay Kumar

August 2026

Contents

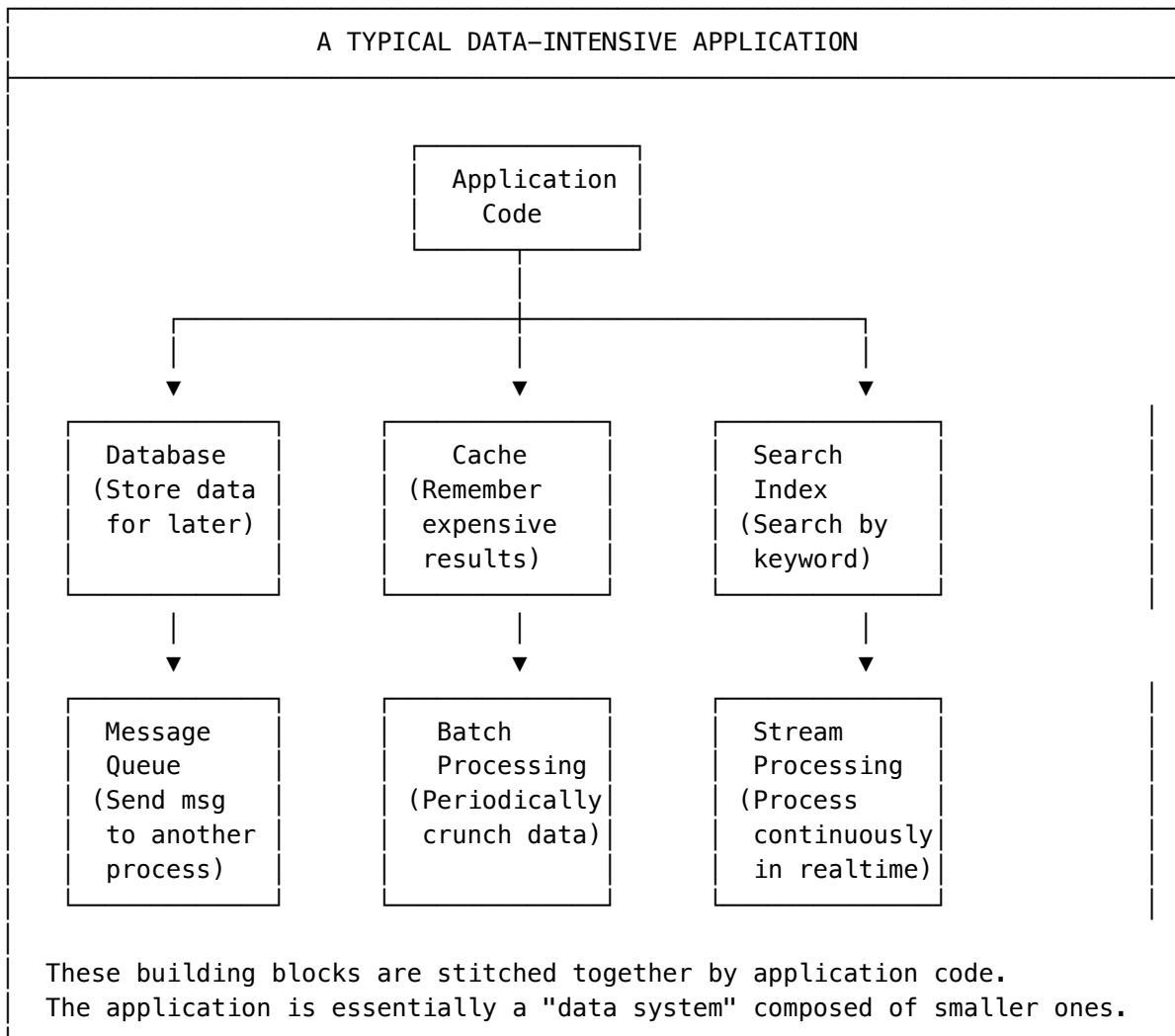
1	Chapter 1: Reliable, Scalable, and Maintainable Applications	3
1.1	Thinking About Data Systems	3
1.2	Reliability	4
1.3	Scalability	6
1.4	Maintainability	10
1.5	Real-World System Examples	11
1.6	Interview Questions	11
2	Chapter 2: Data Models and Query Languages	12
2.1	The Importance of Data Models	12
2.2	Relational Model	13
2.3	Document Model	15
2.4	Relational vs Document Today	16
2.5	Graph-Like Data Models	17
2.6	Query Languages	19
2.7	MapReduce Querying	20
2.8	Interview Questions	21
3	Chapter 3: Storage and Retrieval	22
3.1	The Simplest Database	22
3.2	Hash Indexes	23
3.3	SSTables and LSM-Trees	24
3.4	B-Trees	27
3.5	B-Trees vs LSM-Trees	29
3.6	Other Indexing Structures	31
3.7	Transaction Processing vs Analytics (OLTP vs OLAP)	31
3.8	Column-Oriented Storage	33
3.9	Interview Questions	35
4	Chapter 4: Encoding and Evolution	36
4.1	Formats for Encoding Data	36
4.2	Language-Specific Formats	36
4.3	JSON, XML, and Binary Variants	36
4.4	Thrift and Protocol Buffers	37
4.5	Avro	39
4.6	Schema Evolution Rules	41
4.7	Modes of Dataflow	42
4.8	Interview Questions	44
5	Chapter 5: Replication	45
5.1	Why Replicate?	45
5.2	Leaders and Followers	46
5.3	Synchronous vs Asynchronous Replication	47
5.4	Setting Up New Followers	47
5.5	Handling Node Outages	48
5.6	Problems with Replication Lag	49
5.7	Multi-Leader Replication	51

5.8	Leaderless Replication	52
5.9	Interview Questions	55
6	Chapter 6: Partitioning	56
6.1	Why Partition?	56
6.2	Partitioning of Key-Value Data	56
6.3	Partitioning and Secondary Indexes	58
6.4	Rebalancing Partitions	60
6.5	Request Routing	60
6.6	Interview Questions	61
7	Chapter 7: Transactions	62
7.1	The Slippery Concept of a Transaction	62
7.2	ACID Guarantees	63
7.3	Single-Object and Multi-Object Operations	63
7.4	Weak Isolation Levels	64
7.5	Serializability	68
7.6	Interview Questions	72
8	Chapter 8: The Trouble with Distributed Systems	73
8.1	Faults and Partial Failures	73
8.2	Unreliable Networks	73
8.3	Unreliable Clocks	75
8.4	Knowledge, Truth, and Lies	77
8.5	System Models and Reality	80
8.6	Interview Questions	80
9	Chapter 9: Consistency and Consensus	81
9.1	Consistency Guarantees	81
9.2	Linearizability	82
9.3	Ordering Guarantees	84
9.4	Distributed Transactions and Consensus	85
9.5	Consensus Algorithms	87
9.6	Interview Questions	90
10	Chapter 10: Batch Processing	91
10.1	Three Types of Systems	91
10.2	The Unix Philosophy	92
10.3	MapReduce and Distributed Filesystems	93
10.4	MapReduce Joins	96
10.5	Beyond MapReduce — Dataflow Engines	98
10.6	Interview Questions	99
11	Chapter 11: Stream Processing	100
11.1	Transmitting Event Streams	100
11.2	Message Brokers	101
11.3	Log-Based Message Brokers	102
11.4	Databases and Streams	103
11.5	Processing Streams	104
11.6	Stream Joins	106
11.7	Fault Tolerance in Streaming	107
11.8	Interview Questions	108
12	Chapter 12: The Future of Data Systems	109
12.1	Data Integration	109
12.2	Unbundling Databases	111
12.3	Designing for Correctness	112
12.4	Lambda and Kappa Architecture	114
12.5	End-to-End Argument for Databases	116
12.6	Doing the Right Thing — Ethics	116
12.7	Interview Questions	117

1 Chapter 1: Reliable, Scalable, and Maintainable Applications

1.1 Thinking About Data Systems

Modern applications are **data-intensive** rather than compute-intensive. The bottleneck is usually the amount of data, its complexity, and the speed at which it changes — not raw CPU cycles.



1.1.1 Why the Boundaries Are Blurring

Traditional categories (databases, message queues, caches) are becoming blurred:

Tool	Traditional Category	Also Used For
Redis	Cache	Message queue, data store
Apache Kafka	Message queue	Durable storage, event log
Elasticsearch	Search index	Primary data store
PostgreSQL	Relational database	JSON document store, message queue (LISTEN/NOTIFY)

When you combine several tools to provide a service, the **application code's job** is to keep them in sync: ensuring the cache is invalidated when the database is updated, the search index is rebuilt when data changes, etc.

1.1.2 Three Concerns for Every Data System

Every data system must address three fundamental concerns:

<u>RELIABILITY</u>	<u>SCALABILITY</u>	<u>MAINTAINABILITY</u>
The system continues to work correctly even when things go wrong.	The system can cope with growth (data volume, traffic, complexity).	The system can be worked on productively over time by many people.
"Tolerating faults"	"Measuring load & performance"	"Operability, simplicity, evolvability"

1.2 Reliability

Reliability means the system continues to work correctly even when things go wrong. It does not mean "nothing ever goes wrong" — it means the system is **fault-tolerant**.

1.2.1 Faults vs. Failures

FAULT ≠ FAILURE	
FAULT:	A component of the system deviates from its spec (one node crashes, one disk corrupts data)
FAILURE:	The system as a WHOLE stops providing the required service to the user
GOAL:	Design fault-tolerant systems that prevent faults from causing failures

Key insight: It is usually best to design fault-tolerance mechanisms that **prevent faults from causing failures**. Netflix's Chaos Monkey deliberately introduces faults in production to ensure the system can tolerate them.

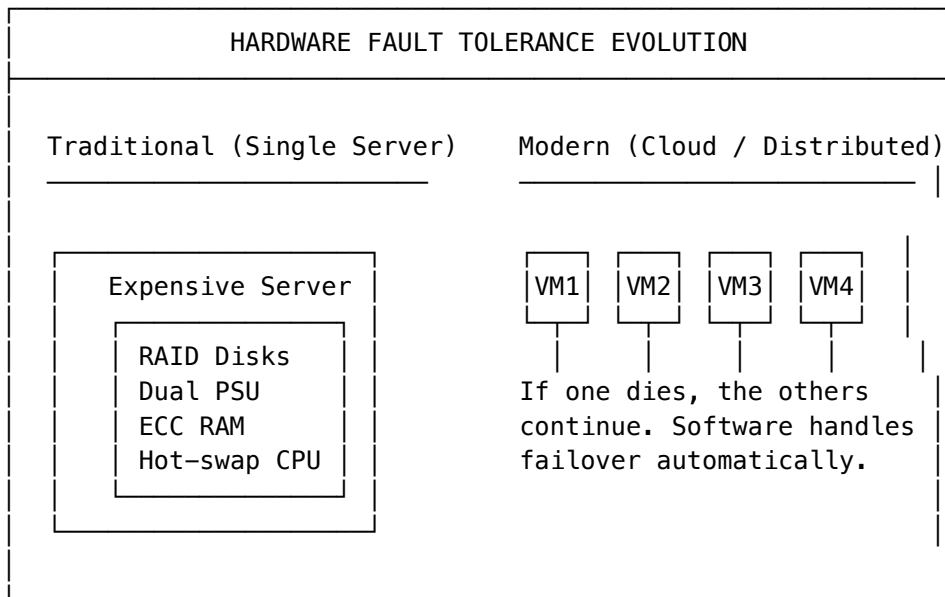
1.2.2 Types of Faults

1.2.2.1 1. Hardware Faults Hardware faults are the most straightforward type:

Component	Mean Time to Failure (MTTF)
Hard disk	~10-50 years
Server (complete failure)	~2-5 years
RAM module	~10 years
Network interface	~5-10 years

Traditional approach: Add redundancy — RAID for disks, dual power supplies, hot-swappable CPUs, diesel generators.

Modern approach: Use software fault-tolerance techniques on top of hardware redundancy because: - Cloud platforms (AWS, GCP) are designed so individual VMs are **disposable** - Systems are too large for individual hardware reliability to suffice - Rolling upgrades (patching one node at a time) avoid full downtime



1.2.2.2 2. Software Faults Software faults are **systematic** — they are correlated across nodes and tend to cause many more system failures than uncorrelated hardware faults.

Common examples: - **Kernel bug** that causes every server running a particular OS version to crash (e.g., Linux leap second bug on June 30, 2012) - **Runaway process** consuming shared resources (CPU, memory, disk, network bandwidth) - **Cascading failures** — a small fault triggers a chain reaction - **A service that the system depends on slows down**, becomes unresponsive, or starts returning corrupted responses

Mitigations: - Thorough testing (unit, integration, end-to-end) - Process isolation - Crash-and-restart approach - Careful monitoring and alerting - Measuring, monitoring, and analyzing system behavior in production

1.2.2.3 3. Human Errors Humans are unreliable. Studies show that **configuration errors by operators** are the leading cause of outages, not hardware or software faults.

Approaches to minimize human error:

MINIMIZING HUMAN ERROR

1. DESIGN FOR ERRORS
 - Well-designed abstractions, APIs, admin interfaces
 - Make it easy to do the right thing, hard to do wrong
2. DECOUPLE & SANDBOX
 - Non-production sandbox environments
 - Use real data without affecting real users
3. TEST THOROUGHLY
 - Unit tests → Integration tests → E2E → Manual QA
 - Automated testing at all levels
4. QUICK RECOVERY
 - Fast rollback of configuration changes
 - Gradual rollouts (canary deployments)
 - Tools to recompute data (in case old data corrupted)
5. DETAILED MONITORING
 - Performance metrics and error rates (telemetry)
 - Early warning signals for problem detection
6. GOOD MANAGEMENT & TRAINING
 - Not purely a technical issue

1.3 Scalability

Scalability is the system's ability to cope with increased **load**. It is not a one-dimensional label ("system X is scalable / not scalable"). Rather, scalability means asking: **"If the system grows in a particular way, what are our options for coping with the growth?"**

1.3.1 Describing Load

Load is described with **load parameters** — numbers that capture the key characteristics of your system's workload:

Load Parameter	Example
Requests per second	Web server: 10K req/s
Read/write ratio	Database: 100:1 reads to writes
Simultaneously active users	Chat app: 500K concurrent users
Hit rate on cache	CDN: 95% cache hit rate
Data volume	Data warehouse: 100TB, growing 10TB/year

1.3.2 The Twitter Fan-Out Problem (Case Study)

Twitter's two main operations (circa 2012): 1. **Post tweet**: User publishes a new message (avg 4.6K req/s, peak 12K req/s) 2. **Home timeline**: User views tweets posted by people they follow (300K req/s)

APPROACH 1: FAN-OUT ON READ

Post Tweet: Insert into global tweets table $O(1)$

Home Timeline Request:

SELECT tweets	JOIN tweets table with follows table
FROM tweets	to find all tweets from followed users
JOIN follows	Sort by time. Very expensive at scale!
WHERE ...	
ORDER BY time	Cost: $O(\text{follows})$ per timeline request

Problem: 300K timeline reads/sec $\times$ expensive JOIN = overload

APPROACH 2: FAN-OUT ON WRITE

Post Tweet:

For each follower of the tweeting user: Insert tweet into that follower's timeline cache (pre-computed home timeline)

User A tweets $\rightarrow$ Write to Follower 1's cache
 $\rightarrow$ Write to Follower 2's cache
 $\rightarrow$ Write to Follower 3's cache
 $\rightarrow$... (avg 75 followers)

Home Timeline: Simply read from pre-computed cache $O(1)$

Problem: Celebrities (30M+ followers) = 30M writes per tweet!

APPROACH 3: HYBRID (Twitter's Solution)

Normal users (< ~5000 followers):	Fan-out on write
Celebrities (many followers):	Fan-out on read

Timeline = pre-computed cache + live celebrity lookup

1.3.3 Describing Performance

Once you've described load, you can investigate what happens when load increases: 1. **Keep resources fixed** — how does performance degrade? 2. **Keep performance fixed** — how much do you need to increase resources?

1.3.3.1 Latency vs. Response Time

Response Time = Network delay + Queueing + Service time

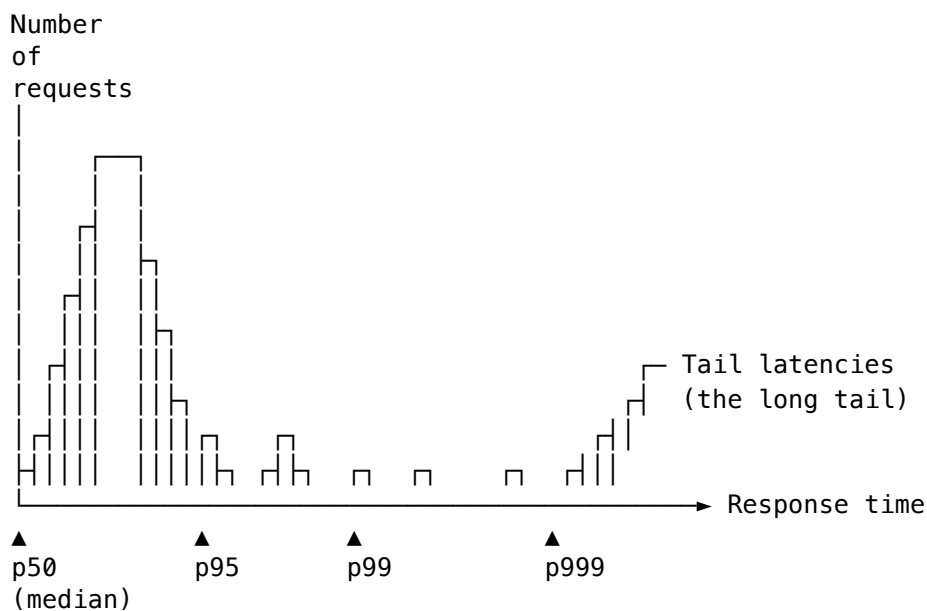
LATENCY: The duration a request is waiting, not being actively serviced (waiting in queue)

SERVICE TIME: The actual time to process the request

RESPONSE TIME: What the client sees (latency + service time)

1.3.3.2 Percentiles (p50, p95, p99, p999) The **median** (p50) is the most common measure, but **tail latencies** matter enormously:

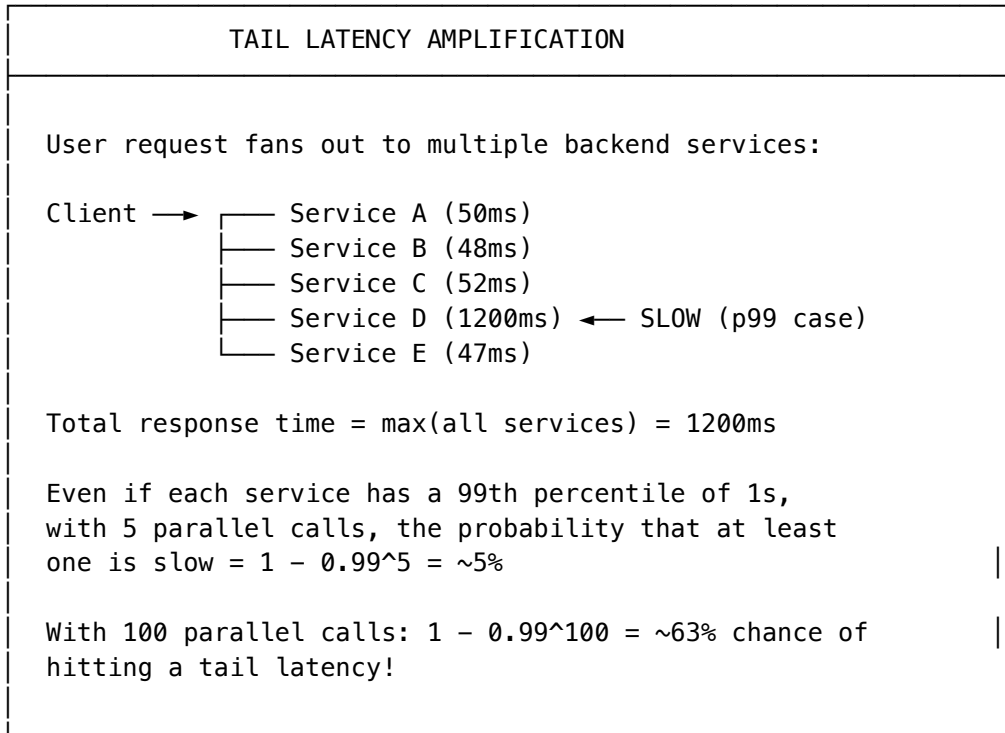
RESPONSE TIME DISTRIBUTION



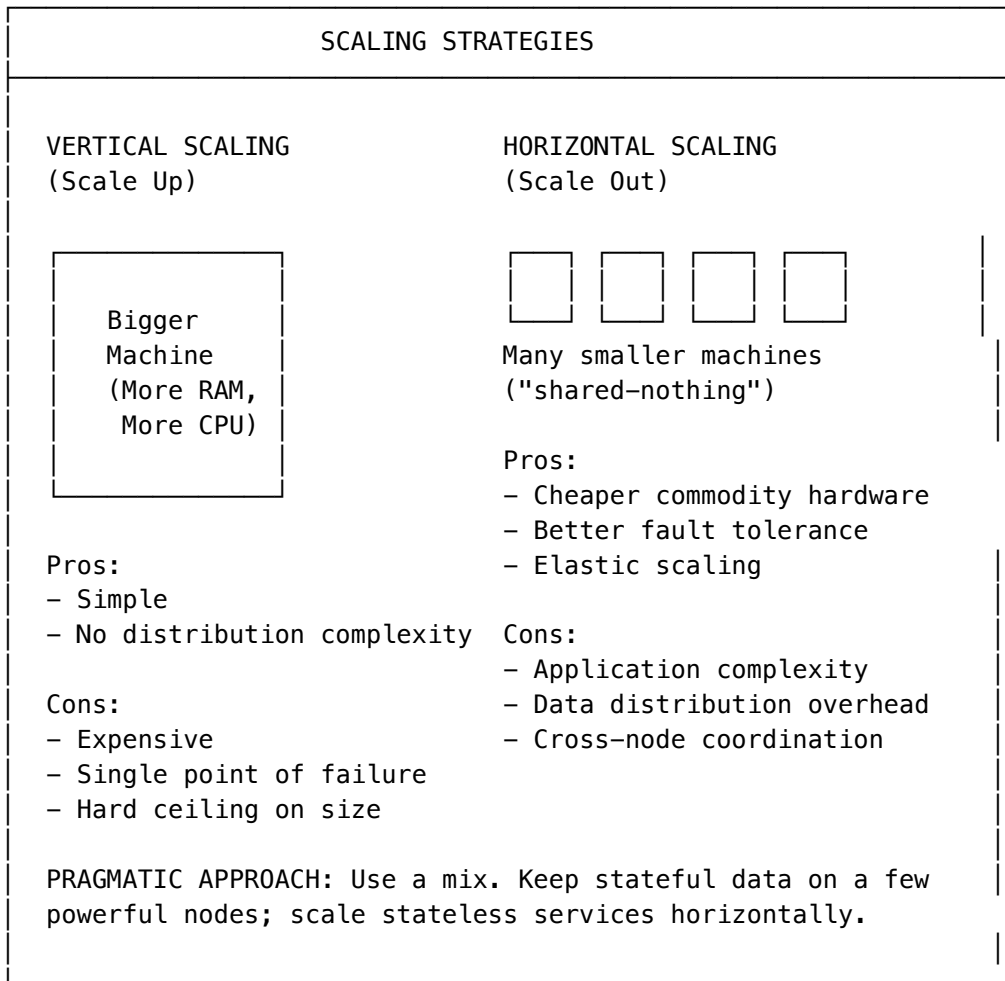
p50 = 200ms → Half of requests are faster than 200ms
p95 = 1s → 95% of requests complete within 1 second
p99 = 1.5s → 1 in 100 requests takes > 1.5 seconds
p999 = 2s → 1 in 1000 requests takes > 2 seconds

Why tail latencies matter: The customers with the slowest requests are often those with the most data — i.e., the most valuable customers (Amazon found that a 100ms increase in response time reduces sales by 1%).

1.3.3.3 Head-of-Line Blocking and Tail Latency Amplification



1.3.4 Approaches to Coping with Load



Key insight: There is no one-size-fits-all scalable architecture. An architecture that scales well for one application may not work for another. The architecture depends on the specific **load parameters** of your application.

1.4 Maintainability

The majority of the cost of software is in its **ongoing maintenance**, not the initial development. Three design principles help minimize pain:

1.4.1 1. Operability — Making Life Easy for Operations

Good operability means making routine tasks easy:

- **Monitoring** system health and restoring service when it goes bad
- **Tracking down** the cause of problems (system failures, degraded performance)
- **Keeping software and platforms up to date** (security patches)
- **Keeping tabs** on how different systems affect each other
- **Anticipating future problems** (capacity planning)
- **Establishing good practices** and tools for deployment, config management
- **Performing complex maintenance tasks** (platform migration)
- **Maintaining system security** as configuration changes
- **Defining processes** that make operations predictable
- **Preserving knowledge** about the system (documentation)

1.4.2 2. Simplicity — Managing Complexity

Complexity manifests as:

SYMPTOMS OF ACCIDENTAL COMPLEXITY

- Explosion of state space
- Tight coupling between modules
- Tangled dependencies
- Inconsistent naming and terminology
- Hacks aimed at solving performance problems
- Special-casing to work around issues elsewhere

ESSENTIAL complexity = inherent to the problem

ACCIDENTAL complexity = arises from the implementation

Best tool for removing accidental complexity:

ABSTRACTION – hide implementation details behind a clean façade (e.g., SQL hides B-trees, TCP hides retransmission)

1.4.3 3. Evolvability — Making Change Easy

System requirements change constantly: - New facts, business priorities, features requested - New platforms, regulatory requirements, growth

Agile techniques (TDD, refactoring) provide a framework for adapting to change at the code level. At the data system level, evolvability (also called **extensibility**, **modifiability**, or **plasticity**) depends on the simplicity and abstraction of the system.

1.5 Real-World System Examples

System	Reliability Approach	Scalability Strategy	Key Maintainability Feature
Netflix	Chaos Monkey, multi-region	Microservices, horizontal scaling	Simple API contracts
Google Search	Redundant everything, GFS/Colossus	MapReduce/Spanner, geo-distribution	
Borg/Kubernetes orchestration			
Amazon	Cell-based architecture	DynamoDB, auto-scaling	Service-oriented architecture
WhatsApp	Erlang fault tolerance	Mnesia DB, custom protocol	Small team, simple architecture

1.6 Interview Questions

1.6.1 Q1: What is the difference between a fault and a failure?

A **fault** is when a single component of the system deviates from its specification (e.g., one disk dies, one process crashes). A **failure** is when the system as a whole stops providing its required service. Fault-tolerant systems are designed so that faults do not escalate into failures. Example: A RAID array tolerates individual disk faults without data loss (failure).

1.6.2 Q2: Why are tail latencies (p99, p999) important?

Tail latencies affect the users with the most data and activity — often the most valuable customers. In microservices architectures, **tail latency amplification** occurs: if a single user request touches multiple services in parallel, even a small p99 on each service compounds. With 100 backend calls, the probability of at least one hitting a tail latency is $1 - 0.99^{100} \approx 63\%$.

1.6.3 Q3: Explain the Twitter fan-out problem and its solution.

Twitter's core operations are posting tweets (5K/s) and reading timelines (300K/s). **Fan-out on read** (query at read time) was too slow for 300K reads/sec. **Fan-out on write** (pre-compute timelines on tweet post) worked for most users but was prohibitively expensive for celebrities with millions of followers. The solution is a **hybrid**: normal users use fan-out on write; celebrity tweets are fetched at read time and merged into the pre-computed timeline.

1.6.4 Q4: What is the difference between vertical and horizontal scaling?

Vertical scaling (scaling up) means moving to a more powerful machine — more CPU, RAM, disk. It's simpler but has a hard ceiling and creates a single point of failure. **Horizontal scaling** (scaling out) means distributing load across multiple smaller machines (shared-nothing architecture). It offers better fault tolerance and elastic scaling but introduces complexity in data distribution and coordination. Most real systems use a pragmatic mix of both.

1.6.5 Q5: How do you minimize human errors in operations?

Design systems that make it easy to do the right thing (good APIs, admin UIs). Provide sandbox environments for safe experimentation. Use thorough automated testing at all levels. Allow quick and easy rollback (config changes, code deployments via canary releases). Implement detailed monitoring with early warning alerts. Practice blameless postmortems and invest in training.

1.6.6 Q6: What are the three pillars of maintainability?

1. **Operability**: Make it easy for operations teams to keep the system running (monitoring, automation, documentation).
2. **Simplicity**: Reduce accidental complexity through good abstractions so new engineers can understand the system.
3. **Evolvability**: Make it easy to adapt the system as requirements change (good modularity, loose coupling).

1.6.7 Q7: What is head-of-line blocking?

Head-of-line blocking occurs when a slow request on a server blocks subsequent requests from being processed, even if the subsequent requests could be handled quickly. In the context of latency, if a server has a limited number of threads and one request takes a long time, it occupies a thread and forces other requests to wait in the queue. This is why measuring response times on the client side (including queueing delay) is more meaningful than measuring only service time.

Based on Chapter 1 of "Designing Data-Intensive Applications" by Martin Kleppmann

2 Chapter 2: Data Models and Query Languages

2.1 The Importance of Data Models

Data models are the **most important** part of developing software. They affect not only how the software is written, but also how we **think about the problem** we are solving.

LAYERS OF DATA MODELS	
Layer 1:	Application developer models the real world → Objects, data structures, APIs
Layer 2:	Store those data structures → JSON, XML, tables, graphs
Layer 3:	Database engineers represent JSON/tables → Bytes in memory, on disk, on network
Layer 4:	Hardware engineers represent bytes → Electrical currents, magnetic fields, photons
Each layer HIDES the complexity of the layers below it through a clean data model (abstraction).	

2.2 Relational Model

Proposed by **Edgar Codd** in 1970. Data is organized into **relations** (tables), where each relation is an unordered collection of **tuples** (rows).

2.2.1 The Dominance of Relational Databases

Timeline of Database Models:

1960s–70s	Hierarchical Model (IMS) — Tree structure Network Model (CODASYL) — Graph with manual pointers
1970	Relational Model (Codd) — Tables with SQL ↓ Won because: • Query optimizer handles access paths • Declarative (WHAT not HOW) • Simpler programming model
1980s–2000s	Relational dominance (Oracle, MySQL, PostgreSQL) Object-relational attempted (failed to take over) XML databases attempted (niche use)
2010s+	NoSQL movement — Document + Graph models Polyglot persistence (use the right DB for the job)

2.2.2 The Object-Relational Mismatch (Impedance Mismatch)

Most application development is done in object-oriented languages, creating an awkward translation layer between objects and relational tables:

IMPEDANCE MISMATCH EXAMPLE: LinkedIn Profile

In-Memory Object:

```
{
  name: "Bill Gates",
  positions: [
    {title: "Co-chair",
     org: "Gates Foundation"},
    {title: "Co-founder",
     org: "Microsoft"}
  ],
  education: [
    {school: "Harvard",
     year: 1973}
  ],
  contact: {
    twitter: "@BillGates",
    blog: "gatesnotes.com"
  }
}
```

In Relational Tables:

users

id	name
1	Bill Gates

positions

user	title	org
1	Co-chair	Gates Fdn
1	Co-found	Microsoft

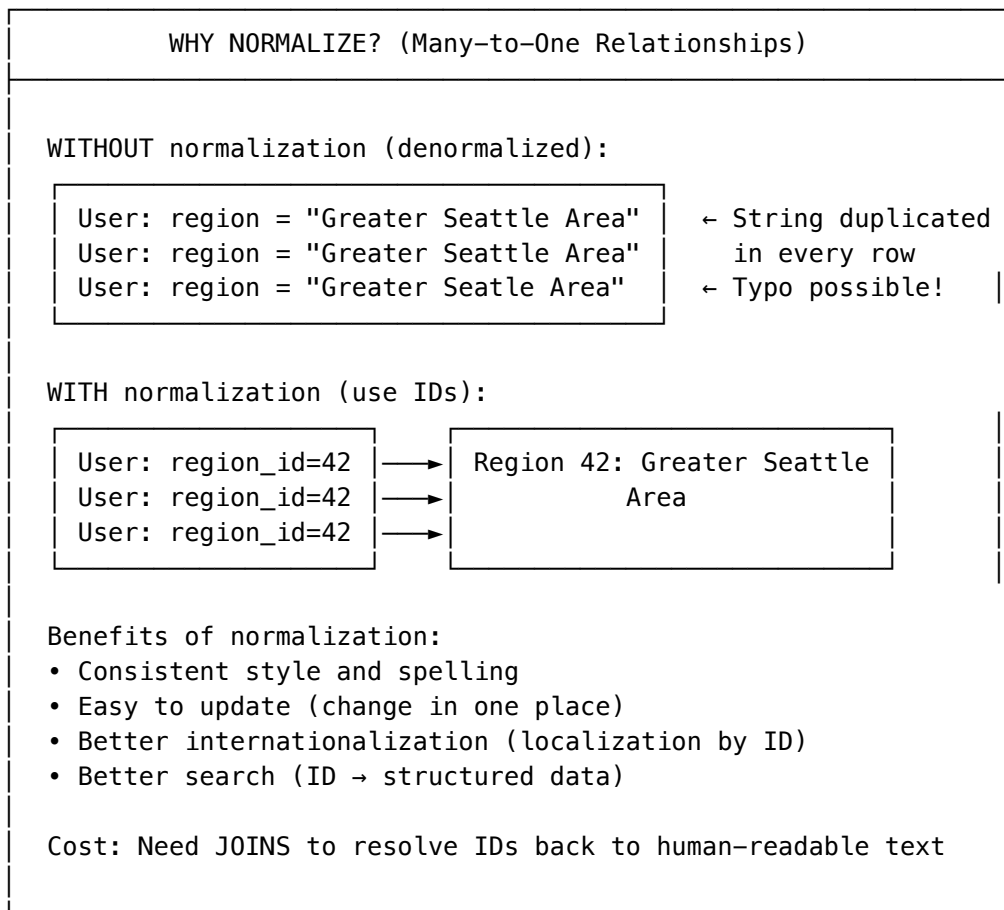
education

user	school	year
1	Harvard	1973

ONE object → MULTIPLE tables with foreign keys
 This mismatch requires an ORM layer (Hibernate, ActiveRecord)

Solutions to impedance mismatch: 1. **ORM frameworks** (Hibernate, SQLAlchemy) — reduce boilerplate but don't eliminate mismatch 2. **JSON/XML columns** in SQL (PostgreSQL JSONB, MySQL JSON) 3. **Document databases** — store the object as-is

2.2.3 Many-to-One and Many-to-Many Relationships



2.3 Document Model

Document databases (MongoDB, CouchDB, RethinkDB) store data as self-contained **documents** (usually JSON):

2.3.1 Advantages of the Document Model

Advantage	Explanation
Schema flexibility	No rigid schema; each document can have different fields
Data locality	Entire document stored together; one read fetches everything
Closer to application objects	Less impedance mismatch with OOP code
Better for 1:many (tree-structured)	Nested structures fit naturally

2.3.2 Disadvantages of the Document Model

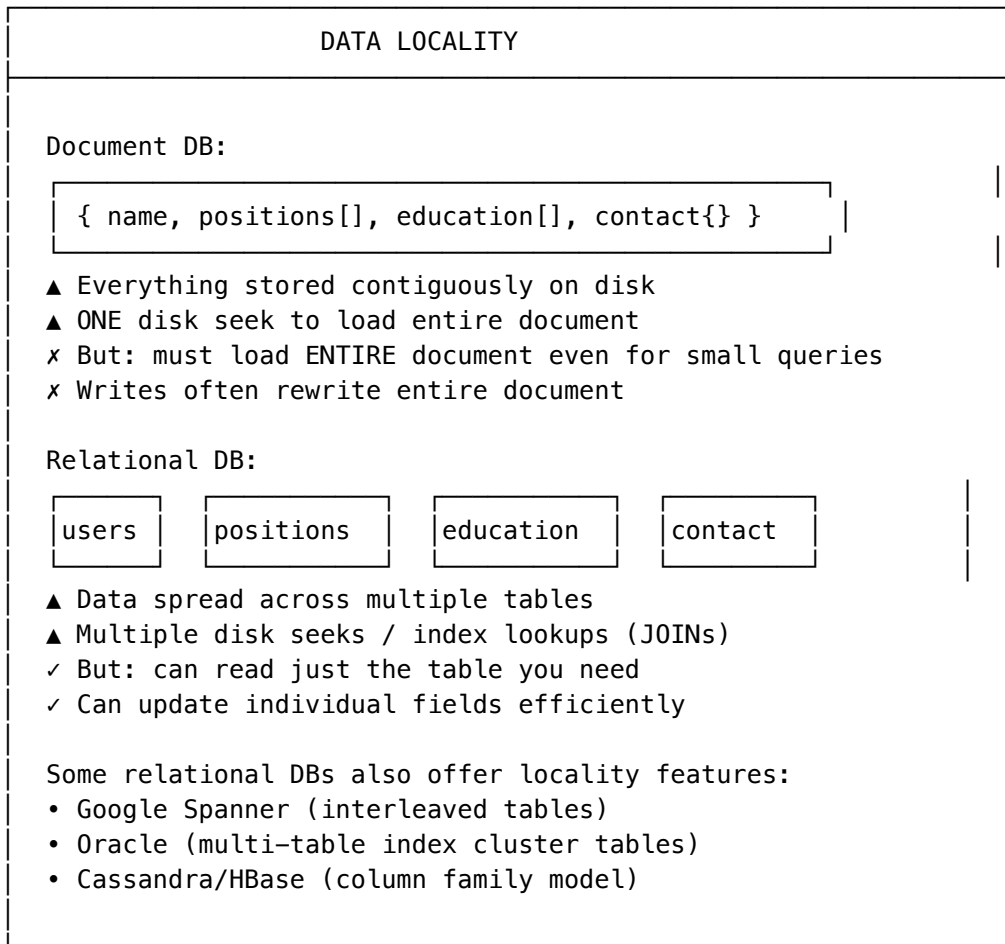
Disadvantage	Explanation
Poor for many-to-many	No joins; must denormalize or do application-level joins
Data duplication	Denormalization leads to inconsistency risks
Can't reference nested items	Hard to say "the second item in user X's positions list"
Large documents	Loading the whole document when you only need a small part is wasteful

2.4 Relational vs Document Today

2.4.1 Schema-on-Read vs Schema-on-Write

SCHEMA-ON-WRITE (Relational)	SCHEMA-ON-READ (Document)
Schema enforced at WRITE time (like static typing)	Schema interpreted at READ time (like dynamic typing)
<pre>ALTER TABLE users ADD COLUMN email VARCHAR(255); UPDATE users SET email = '...' WHERE ...;</pre>	<pre>// Just start writing // documents with email // field. Handle missing // fields in app code.</pre>
✓ Data integrity guaranteed ✓ Clear data documentation ✗ ALTER TABLE can be slow ✗ Schema migrations painful	✓ Flexible, no migrations ✓ Heterogeneous data OK ✗ No guarantee of structure ✗ Bugs hide in bad data
BEST WHEN: • Structure is well-known • All items have same structure • Data integrity is critical	BEST WHEN: • Structure varies per item • Schema changes frequently • External data (no control)

2.4.2 Data Locality

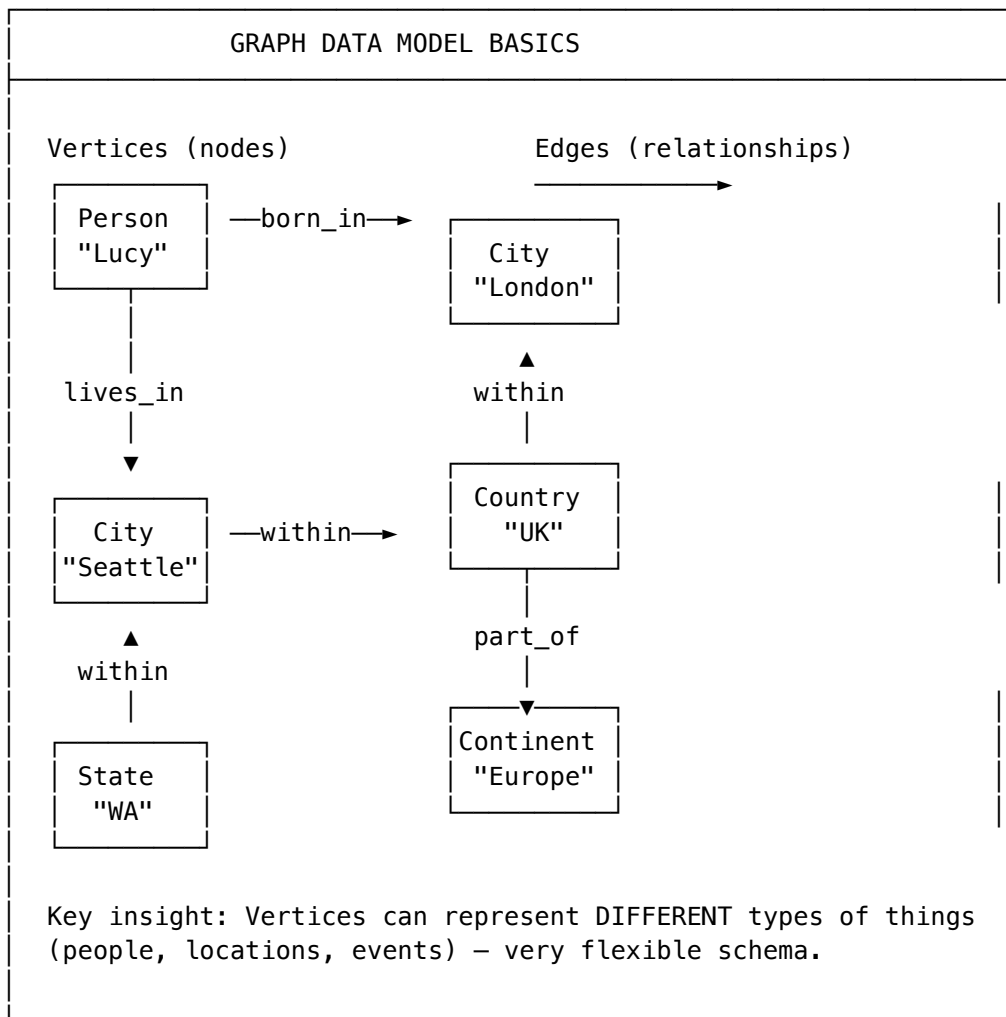


2.4.3 Convergence of Document and Relational

Modern databases increasingly support **both** models: - **PostgreSQL** (9.4+): JSONB columns with indexing and querying - **MySQL** (5.7+): JSON data type with path expressions - **MongoDB** (3.2+): \$lookup for joins (like LEFT OUTER JOIN) - **RethinkDB**: Supports joins in its query language

2.5 Graph-Like Data Models

When your data has many **many-to-many** relationships, a graph model is the most natural.



2.5.1 Property Graph Model (Neo4j, Titan)

Each **vertex** has: - A unique identifier - A set of outgoing edges - A set of incoming edges - A collection of properties (key-value pairs)

Each **edge** has: - A unique identifier - The tail vertex (start) - The head vertex (end) - A label describing the relationship - A collection of properties (key-value pairs)

-- Representing a property graph in relational tables:

```

CREATE TABLE vertices (
  vertex_id  INTEGER PRIMARY KEY,
  properties JSON
);

CREATE TABLE edges (
  edge_id    INTEGER PRIMARY KEY,
  tail_vertex INTEGER REFERENCES vertices(vertex_id),
  head_vertex INTEGER REFERENCES vertices(vertex_id),
  label      TEXT,
  properties JSON
);
  
```

-- Index for traversals

```

CREATE INDEX edges_tails ON edges(tail_vertex);
CREATE INDEX edges_heads ON edges(head_vertex);
  
```

2.5.2 Triple Stores (SPARQL, Datomic)

All information stored as **(subject, predicate, object)** triples:

```
(lucy,      age,      33)
(lucy,      born_in,  london)
(london,    name,     "London")
(london,    within,   england)
(england,   name,     "England")
(england,   within,   europe)
```

2.6 Query Languages

2.6.1 Declarative vs Imperative

IMPERATIVE (HOW)	DECLARATIVE (WHAT)
<pre>function getSharks(animals) { let sharks = []; for (let a of animals) { if (a.family === 'Sharks') { sharks.push(a); } } return sharks; }</pre> • YOU specify HOW • You choose the algorithm • Sequential execution • Order matters	<pre>SELECT * FROM animals WHERE family = 'Sharks';</pre> • Database chooses HOW • Optimizer picks best plan • Parallel execution free • Order doesn't matter

2.6.2 Cypher (Neo4j's Graph Query Language)

-- Find all people who emigrated from the US to Europe:

```
MATCH
  (person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (us:Location {name: 'United States'}),
  (person) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (eu:Location {name: 'Europe'})
RETURN person.name
```

The WITHIN*0.. means “follow a chain of WITHIN edges, zero or more times” — powerful graph traversal.

2.6.3 SPARQL (for Triple Stores)

PREFIX : <urn:example:>

```
SELECT ?personName WHERE {
  ?person :name ?personName .
  ?person :bornIn / :within* / :name "United States" .
  ?person :livesIn / :within* / :name "Europe" .
}
```

2.6.4 SQL Recursive Queries (Graph Traversal in SQL)

-- Same query in SQL using recursive CTEs (verbose!):

```
WITH RECURSIVE
  in_usa(vertex_id) AS (
    SELECT vertex_id FROM vertices WHERE properties->>'name' = 'United States'
    UNION
    SELECT edges.tail_vertex
    FROM edges JOIN in_usa ON edges.head_vertex = in_usa.vertex_id
    WHERE edges.label = 'within'
  ),
  in_europe(vertex_id) AS (
    SELECT vertex_id FROM vertices WHERE properties->>'name' = 'Europe'
    UNION
    SELECT edges.tail_vertex
    FROM edges JOIN in_europe ON edges.head_vertex = in_europe.vertex_id
    WHERE edges.label = 'within'
  )
SELECT v.properties->>'name' AS person_name
FROM vertices v
JOIN edges born ON born.tail_vertex = v.vertex_id AND born.label = 'born_in'
JOIN edges lives ON lives.tail_vertex = v.vertex_id AND lives.label = 'lives_in'
WHERE born.head_vertex IN (SELECT vertex_id FROM in_usa)
  AND lives.head_vertex IN (SELECT vertex_id FROM in_europe);
```

This shows why **graph databases exist** — the same query is dramatically simpler in Cypher.

2.7 MapReduce Querying

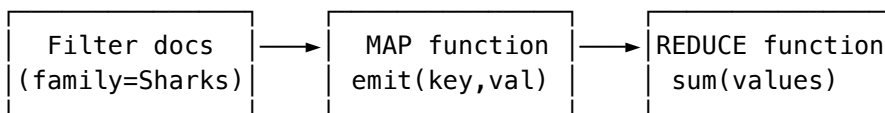
MapReduce is a programming model for processing large amounts of data in bulk across many machines (popularized by Google, implemented by MongoDB and Hadoop).

MapReduce: COUNT SHARKS PER MONTH

MongoDB MapReduce:

```
db.observations.mapReduce(  
  function map() { // Called once per doc  
    var year = this.observedAt.getFullYear();  
    var month = this.observedAt.getMonth() + 1;  
    emit(year + "-" + month, this.numAnimals);  
  },  
  function reduce(key, values) { // Called per group  
    return Array.sum(values);  
  },  
  {  
    query: { family: "Sharks" }, // Filter first  
    out: "monthlySharkReport" // Output collection  
  }  
);
```

Pipeline:



Equivalent MongoDB Aggregation Pipeline (preferred today):

```
db.observations.aggregate([  
  { $match: { family: "Sharks" } },  
  { $group: {  
    _id: { year: { $year: "$observedAt" },  
          month: { $month: "$observedAt" } },  
    totalAnimals: { $sum: "$numAnimals" }  
  } }  
]);
```

2.8 Interview Questions

2.8.1 Q1: What is impedance mismatch?

The disconnect between the object-oriented model used in application code and the relational model used in databases. Objects have nested structures, lists, and references, but relational tables are flat rows with foreign keys. ORMs (Hibernate, SQLAlchemy) reduce boilerplate but don't eliminate the fundamental mismatch. Document databases reduce this by storing objects as JSON directly.

2.8.2 Q2: When would you choose a document database over a relational database?

Choose document DB when: (1) your data has a natural document structure (1-to-many, tree-like), (2) the schema varies between records, (3) you need data locality (load everything about an entity in one read), (4) relationships between documents are rare. Choose relational when: data has many-to-many relationships, joins are common, referential integrity matters, or you need complex ad-hoc queries.

2.8.3 Q3: Explain schema-on-read vs schema-on-write.

Schema-on-write (relational DBs): The database enforces a schema at write time — writes that don't conform are rejected. Like static typing. **Schema-on-read** (document DBs): No schema is enforced; the structure is interpreted by the application when data is read. Like dynamic typing. Schema-on-read is better when items don't all have the same structure or when the schema changes frequently.

2.8.4 Q4: When should you use a graph database?

When your data has many **many-to-many** relationships and you need to traverse connections (shortest path, recommendation engines, social networks, fraud detection, knowledge graphs). Graph databases excel at queries like “find all people who live in Europe but were born in the US” which require recursive traversal — something that's verbose and slow in SQL but natural in Cypher/SPARQL.

2.8.5 Q5: Why are declarative query languages preferred?

Declarative languages (SQL, Cypher) specify **what** data you want, not **how** to get it. This allows the database query optimizer to choose the best execution strategy (which indexes to use, which join order, whether to parallelize). Imperative code locks in a specific algorithm. When the database adds a new index or optimization, declarative queries benefit automatically without code changes.

2.8.6 Q6: Compare the relational, document, and graph data models.

Aspect	Relational	Document	Graph
Structure	Tables with rows/columns	JSON/BSON documents	Vertices and edges
Schema	Schema-on-write (rigid)	Schema-on-read (flexible)	Schema-on-read (flexible)
Relationships	Foreign keys + JOINS	Embedded or references	Edges (first-class)
Best for	Structured, relational data	Tree-like, self-contained data	Highly connected data
Joins	Native, efficient	Weak/absent	Traversals (native)
Examples	PostgreSQL, MySQL	MongoDB, CouchDB	Neo4j, Amazon Neptune

Based on Chapter 2 of “Designing Data-Intensive Applications” by Martin Kleppmann

3 Chapter 3: Storage and Retrieval

3.1 The Simplest Database

The world's simplest database — two Bash functions:

```
#!/bin/bash
db_set() {
  echo "$1,$2" >> database           # Append to end of file
}

db_get() {
  grep "^$1," database | sed -e "s/^$1,//" | tail -n 1  # Scan for last occurrence
}
```

- **Writes are $O(1)$:** Appending to a file is very fast
- **Reads are $O(n)$:** Must scan entire file to find the latest value for a key
- **Key insight:** An **index** is an additional data structure that speeds up reads at the cost of slower writes

THE FUNDAMENTAL TRADE-OFF OF INDEXES	
No Index	With Index
Write: $O(1)$ append	$O(1)$ append + $O(\log n)$ index update
Read: $O(n)$ scan	$O(\log n)$ or $O(1)$ lookup
Well-chosen indexes speed up reads enormously. Every index slows down writes. Database doesn't index everything by default – the developer must choose indexes manually.	

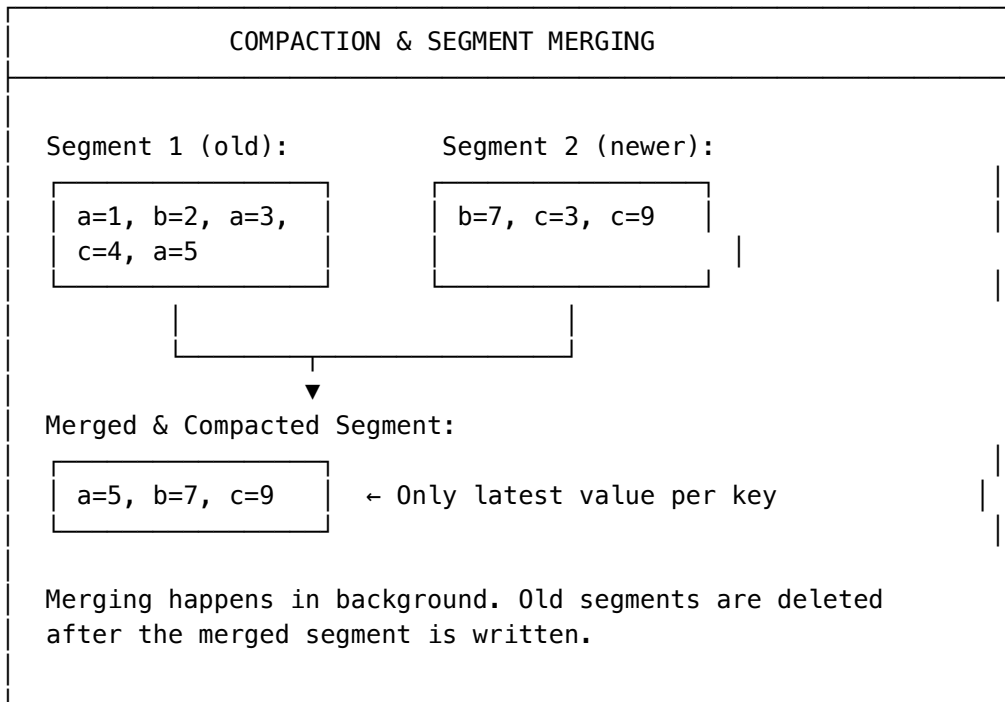
3.2 Hash Indexes

The simplest indexing strategy: keep an in-memory **hash map** of key → byte offset.

HASH INDEX (e.g., Bitcask in Riak)	
In-Memory Hash Map:	On-Disk Log File:
key123 → offset 0 key456 → offset 64 key789 → offset 128 key123 → offset 192	key123,{"name":"Alice"} key456,{"name":"Bob"} key789,{"name":"Carol"} key123,{"name":"Updated"}
• Hash map always points to LATEST value for each key • Reads: Look up key in hash map → seek to byte offset → read • Writes: Append to log → update hash map entry	

3.2.1 Compaction and Segment Merging

The log file grows forever — solved by **compaction** (removing duplicate keys, keeping only the latest):



3.2.2 Practical Considerations

Concern	Solution
File format	Binary format (length-prefixed strings) faster than CSV
Deleting records	Append a tombstone marker; merging discards tombstoned keys
Crash recovery	Rebuild hash map by scanning segments (slow); or keep hash map snapshots on disk
Partially written records	Use checksums to detect and ignore corrupted records
Concurrency	Single writer thread; reads can be concurrent (immutable segments)

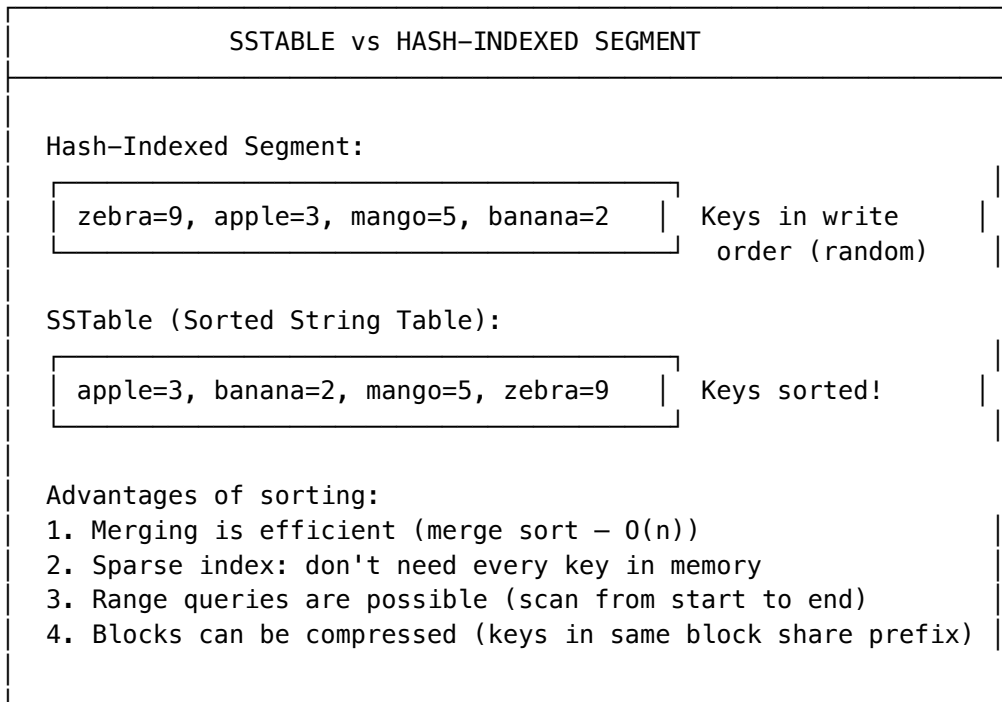
3.2.3 Limitations of Hash Indexes

- **Must fit in memory:** Entire hash map must be in RAM
- **Range queries inefficient:** Can't scan all keys between kitty00000 and kitty99999

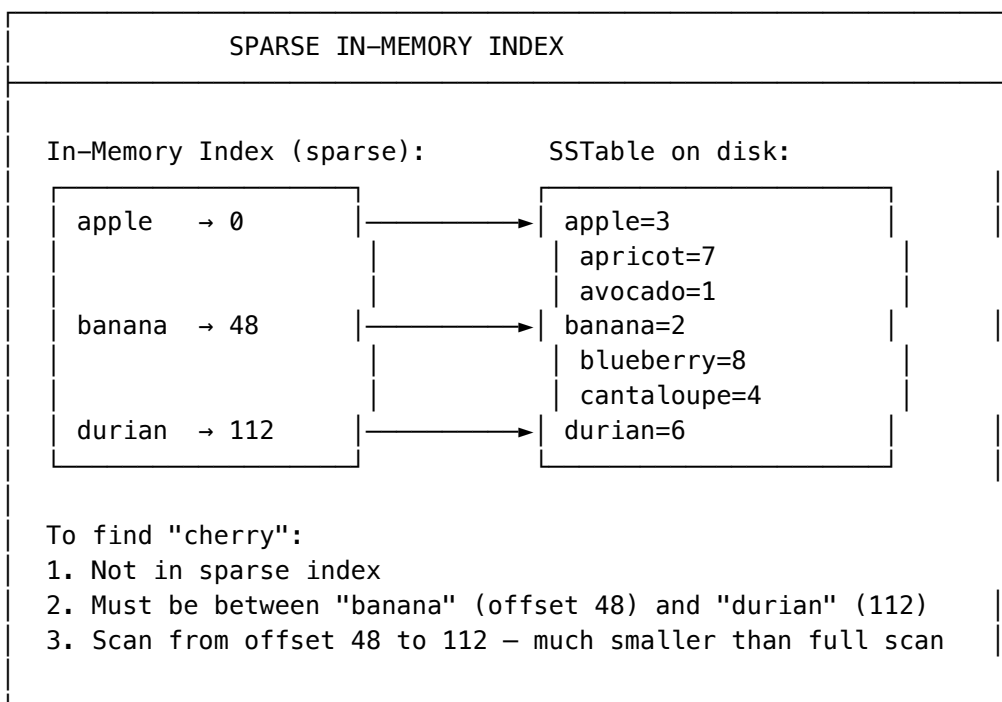
3.3 SSTables and LSM-Trees

3.3.1 SSTable (Sorted String Table)

An improvement over hash-indexed segments: **keys are sorted** within each segment.

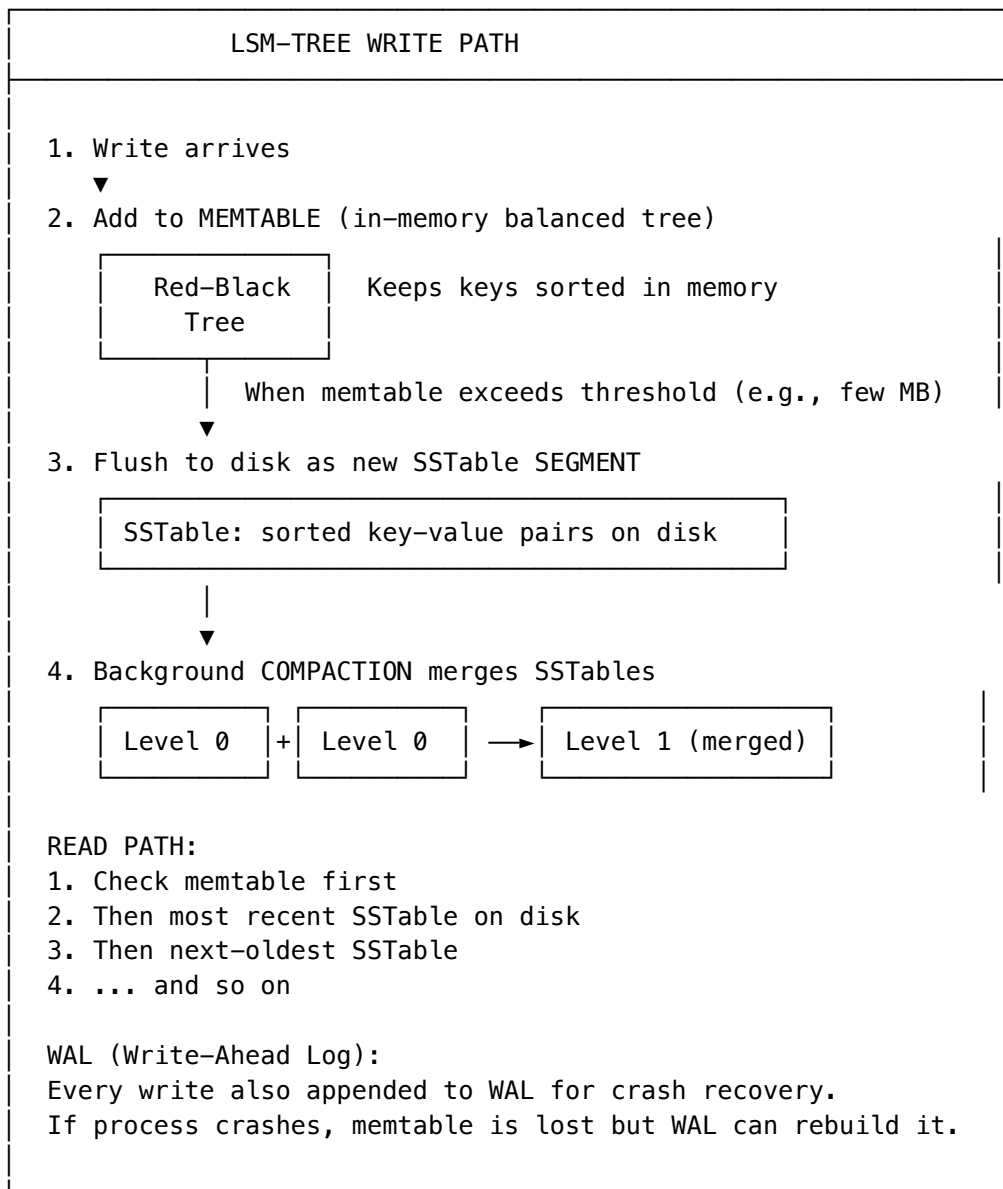


3.3.2 Sparse Index for SSTables



3.3.3 Constructing SSTables from a Memtable (LSM-Tree)

You can't sort writes as they arrive (random order). Solution: use an **in-memory balanced tree** (red-black tree or AVL tree) called a **memtable**:



3.3.4 LSM-Tree Compaction Strategies

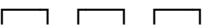
SIZE-TIERED vs LEVELED COMPACTION

SIZE-TIERED (HBase, Cassandra default):

- Newer, smaller SSTables merged into older, larger ones
- Each tier has SSTables of similar size
- Simpler, better write throughput
- More space amplification (temporarily stores duplicates)

LEVELED (LevelDB, RocksDB, Cassandra LCS):

- Key range split into smaller SSTables
- Each level has non-overlapping key ranges
- Level N is ~10x the size of Level N-1
- Better read performance, less space amplification
- More write amplification (rewrites data more often)

Level 0:  (may overlap)
Level 1:  (non-overlapping)
Level 2:  (non-overlapping, 10x)
Level 3: (even more SSTables, 10x larger total)

3.3.5 Bloom Filters — Optimizing Non-Existent Key Lookups

A key that doesn't exist forces checking every SSTable level. **Bloom filters** short-circuit this:

BLOOM FILTER

A memory-efficient probabilistic data structure:

- "Definitely NOT in the set" – 100% certain (no false neg)
- "Probably in the set" – might be wrong (false positive OK)

Used by LSM-tree to skip SSTables that definitely don't contain the requested key. Saves many unnecessary disk reads.

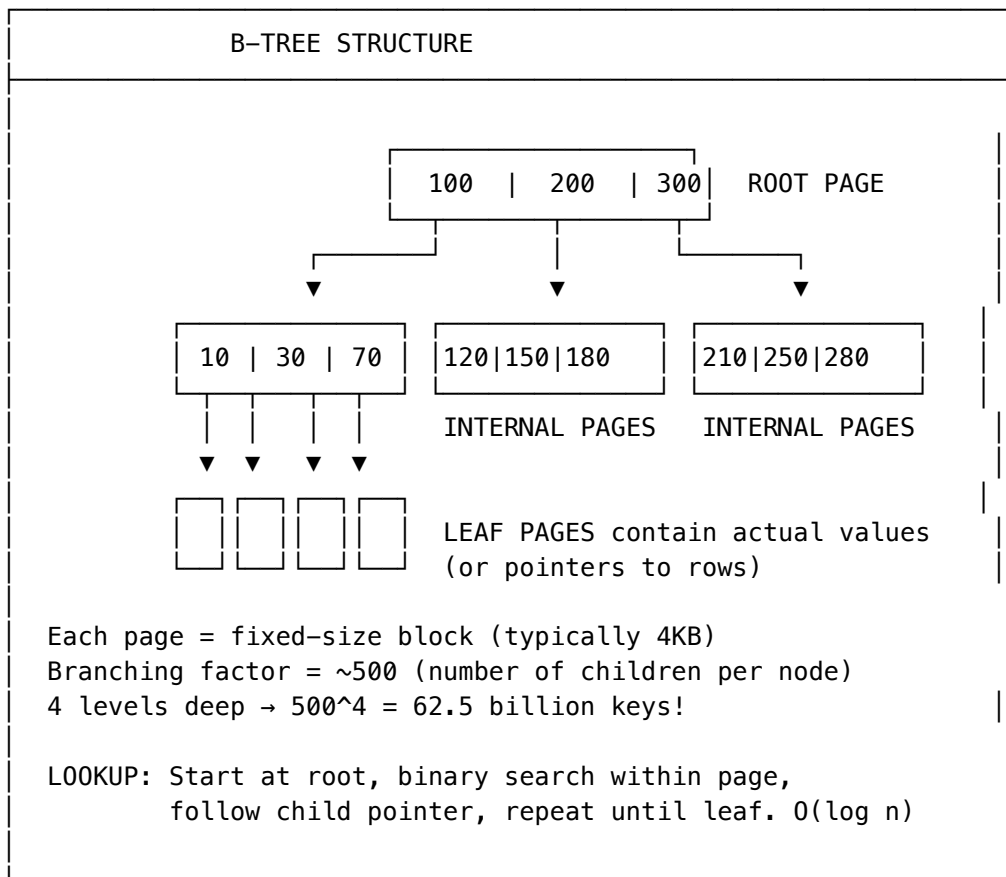
Bit array: [0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1]



Hash functions set these bits for each key

3.4 B-Trees

The most widely used index structure. Used by virtually every relational database.



3.4.1 B-Tree Updates

B-TREE WRITE PATH

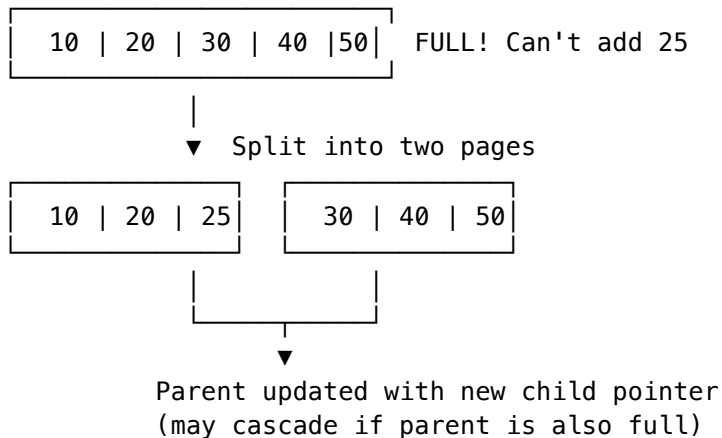
UPDATE existing key:

1. Search for leaf page containing the key
2. Overwrite the value IN PLACE on that page
3. Write the page back to disk

INSERT new key (page has space):

1. Find the leaf page where key belongs
2. Add key-value to that page
3. Write page back to disk

INSERT new key (page is FULL → SPLIT):



3.4.2 Write-Ahead Log (WAL) for B-Trees

Every B-tree modification is first written to a **write-ahead log** (WAL / redo log) before modifying the tree pages. On crash, the WAL is replayed to restore the B-tree to a consistent state.

Write flow: Write → WAL (append-only) → Modify B-tree page(s)

Crash: Replay WAL to fix any partially-written pages

3.5 B-Trees vs LSM-Trees

B-TREES vs LSM-TREES COMPARISON		
Aspect	B-Tree	LSM-Tree
Write pattern	Random I/O (update in place)	Sequential I/O (append-only)
Write throughput	Lower	Higher
Write amplific.	Lower (1 WAL + 1 page write)	Higher (compaction rewrites data)
Read performance	Faster (one lookup)	Slower (check multiple SSTables/levels)
Space usage	Fragmentation possible	Compaction reclaims space; but temp dup.
Predictability	More predictable latency	Compaction can cause latency spikes
Concurrency	Latch per page	Simpler (immutable SSTables)
Range queries	Excellent (sorted leaf pages linked)	Good (sorted within SSTables)
Key uniqueness	Easy (one place per key)	Harder (key may exist in multiple SSTables)
Transaction isolation	Easier (lock ranges of keys)	Harder
Used by:		
B-Tree: PostgreSQL, MySQL/InnoDB, Oracle, SQL Server		
LSM-Tree: RocksDB, LevelDB, Cassandra, HBase, ScyllaDB		

3.5.1 Write Amplification

WRITE AMPLIFICATION
One database write → multiple disk writes B-Tree: App write → WAL write + Page write = 2x amplification (Sometimes more if page splits cascade) LSM-Tree: App write → WAL + Memtable flush + L0→L1 compaction + L1→L2 compaction + ... = 10-30x amplification High write amplification concerns: • SSD lifetime (limited write cycles per cell) • Disk bandwidth consumed by compaction vs. actual writes But LSM writes are SEQUENTIAL → much faster on SSDs/HDDs than B-tree's RANDOM writes. Net throughput often better.

3.6 Other Indexing Structures

3.6.1 Clustered vs Non-Clustered Indexes

NON-CLUSTERED INDEX (most secondary indexes)	CLUSTERED INDEX (InnoDB primary key)
Index leaf page: key → pointer to heap → heap file	Index leaf page: key → actual row data (stored in index)
Extra hop to read data Write: update heap + index Multiple indexes → same heap	No extra hop (data locality) Write: update index directly Only ONE clustered index
COVERING INDEX (compromise): Store SOME columns in the index (not just the key). Queries reading only those columns are answered from the index alone – no heap lookup. Called an "index-only scan."	

3.6.2 Multi-Column Indexes

CONCATENATED INDEX: (last_name, first_name) → sorted by last_name, then first Good for: WHERE last_name = 'Smith' AND first_name = 'John' Good for: WHERE last_name = 'Smith' (prefix) Bad for: WHERE first_name = 'John' (not a prefix!)
MULTI-DIMENSIONAL INDEX (R-tree): For geospatial queries: SELECT * FROM restaurants WHERE latitude BETWEEN 51.4 AND 51.6 AND longitude BETWEEN -0.2 AND 0.1; B-tree can only search one dimension efficiently. R-tree indexes 2D+ space for bounding-box queries.

3.7 Transaction Processing vs Analytics (OLTP vs OLAP)

OLTP vs OLAP		
Property	OLTP	OLAP (Data Warehouse)
Read pattern	Few records per query fetched by key	Aggregate over many records
Write pattern	Random-access, low- latency user input	Bulk import (ETL) or event stream
Used by	End users via web app	Internal analysts
Data represents	Latest state (now)	History of events
Dataset size	GB to TB	TB to PB
Bottleneck	Disk seek time	Disk bandwidth


```

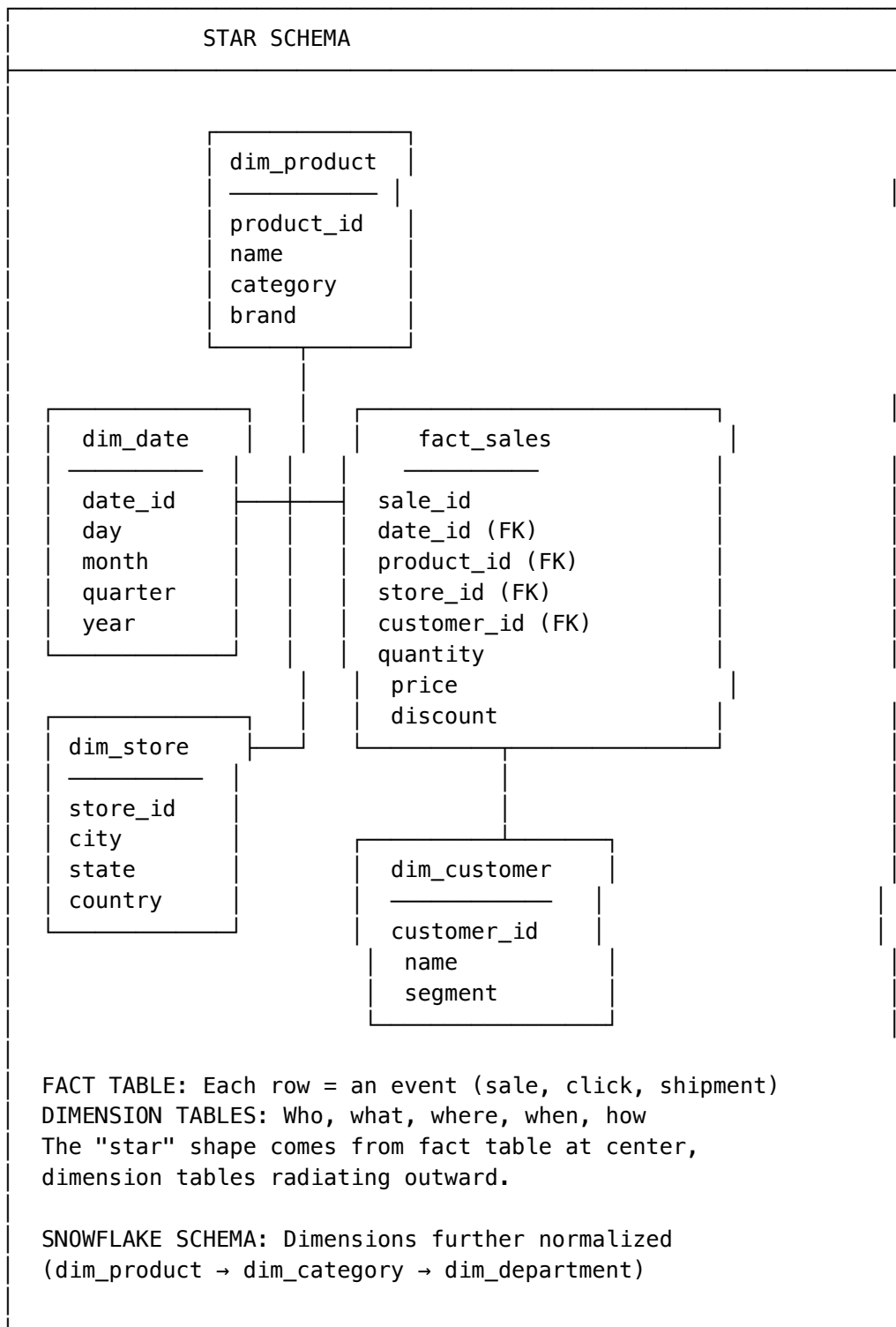
graph LR
    OLTP_DB_1[OLTP DB 1] -- ETL --> DW[Data Warehouse OLAP]
    OLTP_DB_2[OLTP DB 2] -- ETL --> DW
    OLTP_DB_3[OLTP DB 3] -- ETL --> DW
    subgraph ETL
        Extract
        Transform
        Load
    end
    DW --- Note[Optimized for analytics queries]
  
```

Examples:

OLTP: PostgreSQL, MySQL, Oracle (operational)

OLAP: Amazon Redshift, Snowflake, ClickHouse, BigQuery

3.7.1 Star Schema (Dimensional Modeling)



3.8 Column-Oriented Storage

In OLAP queries, you typically access only 3-5 columns out of 100+ in a table. Column-oriented storage stores all values of a column together:

ROW-ORIENTED vs COLUMN-ORIENTED STORAGE

ROW-ORIENTED (traditional):

Row 1: date=2024-01-01, product=68, store=4, qty=1 ...
Row 2: date=2024-01-01, product=69, store=5, qty=3 ...
Row 3: date=2024-01-02, product=68, store=4, qty=2 ...

Reading 3 columns out of 100 → must load ALL 100 columns

COLUMN-ORIENTED:

date column:	[2024-01-01, 2024-01-01, 2024-01-02, ...]
product column:	[68, 69, 68, ...]
store column:	[4, 5, 4, ...]
qty column:	[1, 3, 2, ...]

Reading 3 columns → load ONLY those 3 column files

```
Query: SELECT product_id, SUM(quantity)
      FROM fact_sales
      WHERE date BETWEEN '2024-01-01' AND '2024-01-31'
      GROUP BY product_id;
```

Row-oriented: Must read entire rows (100 columns × millions)

Column-oriented: Read only date + product + quantity columns

3.8.1 Column Compression

BITMAP ENCODING (Column Compression)

product_id column has values: [69, 69, 69, 74, 31, 31, ...]

Bitmap for product_id = 69: [1, 1, 1, 0, 0, 0, ...]

Bitmap for product_id = 74: [0, 0, 0, 1, 0, 0, ...]

Bitmap for product_id = 31: [0, 0, 0, 0, 1, 1, ...]

WHERE product_id IN (69, 74):

→ Bitwise OR of bitmaps for 69 and 74

→ [1, 1, 1, 1, 0, 0, ...] – very fast!

Run-Length Encoding (RLE):

[1,1,1,0,0,0,0,0,0,0,0,0,...] → "3 ones, then 9 zeros, ..."

Extremely compact for sparse bitmaps.

Vectorized processing:

Compressed columns loaded into CPU L1 cache → bitwise ops
process thousands of values in single CPU cycle (SIMD).

3.8.2 Materialized Views and Data Cubes

DATA CUBE (OLAP Cube)						
Product						
		P1	P2	P3	P4	TOTAL
Date	D1	52	30	18	44	144
	D2	63	41	22	35	161
	D3	48	28	25	51	152
	TOT	163	99	65	130	457

Pre-aggregated totals along each dimension.
Fast for known aggregate queries.
Inflexible – can't answer queries not in the cube's dimensions.

3.9 Interview Questions

3.9.1 Q1: Explain the difference between B-Trees and LSM-Trees.

B-Trees update data **in place** on fixed-size pages, using a balanced tree structure with $O(\log n)$ reads. LSM-Trees use an **append-only** approach: writes go to an in-memory memtable, which is periodically flushed to sorted SSTables on disk. Background compaction merges SSTables. B-Trees have faster reads (single lookup path) but slower writes (random I/O). LSM-Trees have faster writes (sequential I/O) but slower reads (must check multiple levels). B-Trees are used by most relational DBs; LSM-Trees by RocksDB, Cassandra, HBase.

3.9.2 Q2: What is write amplification and why does it matter?

Write amplification is when one application write causes multiple actual disk writes. In B-Trees: WAL write + page write = $\sim 2x$. In LSM-Trees: WAL + memtable flush + multiple compaction levels = 10-30x. It matters because: (1) it consumes disk bandwidth that could serve other operations, (2) on SSDs it reduces drive lifetime (limited write cycles per cell). However, LSM-Trees compensate with sequential writes which are much faster than B-Tree's random writes.

3.9.3 Q3: Why do analytics databases use column-oriented storage?

OLAP queries typically read only 3-5 columns out of 100+ in a wide fact table. Row-oriented storage must load all columns for each row. Column-oriented storage stores each column separately, so a query reads only the needed columns. Additional benefits: (1) better compression (similar values in a column compress well — bitmap encoding, RLE), (2) vectorized processing (compressed column chunks fit in CPU L1 cache for SIMD operations), (3) sort order can cluster related values for even better compression and locality.

3.9.4 Q4: What is a Bloom filter and why is it used in LSM-Trees?

A Bloom filter is a memory-efficient probabilistic data structure that tells you either “definitely NOT in the set” or “possibly in the set” (with a small false-positive rate). In LSM-Trees, looking up a non-existent key requires checking every SSTable level — very expensive. A Bloom filter per SSTable lets you skip SSTables that definitely don't contain the key, dramatically reducing unnecessary disk reads.

3.9.5 Q5: Explain the star schema in data warehousing.

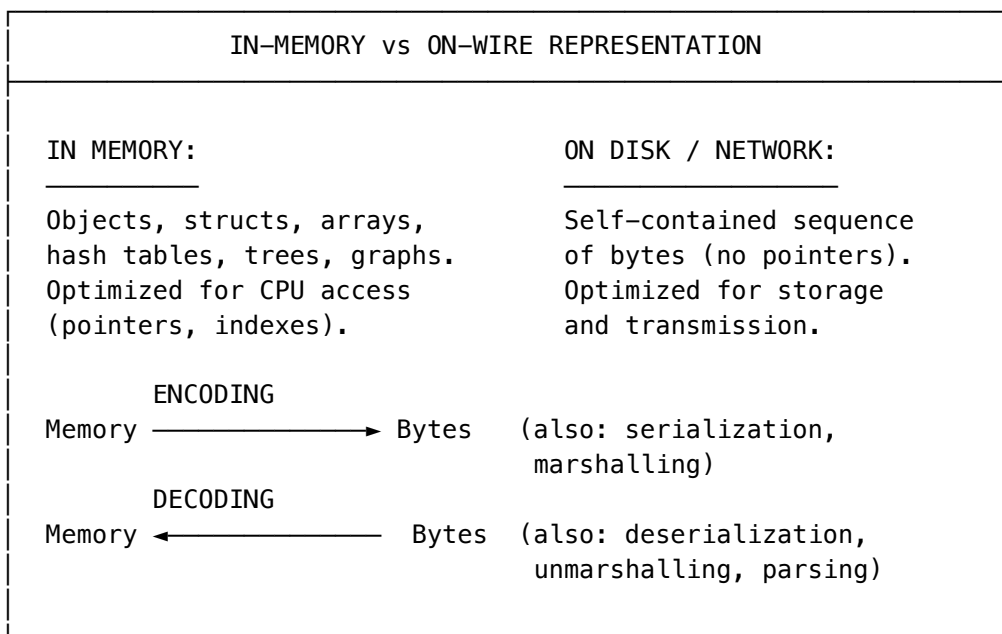
The star schema has a central **fact table** (each row = an event, like a sale) surrounded by **dimension tables** (who, what, where, when). The fact table contains foreign keys to dimensions plus metric columns (quantity, price). It's called "star" because the fact table sits at the center with dimension tables radiating outward. A variant called the **snowflake schema** further normalizes dimensions into sub-dimensions. Star schemas are simpler to query and are the standard for OLAP workloads.

Based on Chapter 3 of "Designing Data-Intensive Applications" by Martin Kleppmann

4 Chapter 4: Encoding and Evolution

4.1 Formats for Encoding Data

Programs work with data in two representations:



4.2 Language-Specific Formats

Many languages have built-in serialization: Java Serializable, Python pickle, Ruby Marshal.

Problems with language-specific encoding:

Problem	Details
Language lock-in	Data encoded with Java's Serializable can only be read by Java
Security	Decoding can instantiate arbitrary classes → remote code execution
Versioning	Poor support for forward/backward compatibility
Efficiency	Java's built-in serialization is notoriously slow and bloated

Rule: Never use language-specific serialization for anything beyond very transient purposes.

4.3 JSON, XML, and Binary Variants

4.3.1 Problems with JSON and XML

JSON/XML ENCODING ISSUES

1. NUMBER AMBIGUITY

JSON: no distinction between int and float

JavaScript: Numbers $> 2^{53}$ lose precision

Twitter API returns tweet IDs as both number AND string because JavaScript mangles large numbers.

2. NO BINARY STRING SUPPORT

JSON/XML: text only. Binary data must be Base64 encoded
→ 33% size increase.

3. SCHEMA SUPPORT

XML: XSD exists but complex and rarely used

JSON: JSON Schema exists but not widely adopted

Without schema → application must hardcode field names

4. VERBOSITY

XML: `<firstName>John</firstName>`

JSON: `{"firstName": "John"}`

Field names repeated in every record → wastes space

4.3.2 Binary JSON Encodings

BINARY JSON VARIANTS

Format	Used By	Key Feature
MessagePack	Redis, Fluentd	Simple binary JSON
BSON	MongoDB	Adds date, binary, int types
BJSON	Various	Binary JSON encoding
UBJSON	Various	Universal Binary JSON

Problem: These still include field names in every record.

Savings are modest (~15–30%) compared to text JSON.

For significant compression, need a SCHEMA-based approach.

4.4 Thrift and Protocol Buffers

Both use a **schema** to define the structure, then encode data using field tags (numbers) instead of field names.

4.4.1 Protocol Buffers Schema Example

```

message Person {
  required string user_name = 1;
  optional int64 favorite_number = 2;
  repeated string interests = 3;
}

```

4.4.2 Thrift Schema Example

```

struct Person {
  1: required string userName,
  2: optional i64 favoriteNumber,
  3: optional list<string> interests,
}

```

4.4.3 Binary Encoding Comparison

ENCODING SIZE COMPARISON		
Example record: {"userName":"Martin","favoriteNumber":1337,"interests":["daydreaming","hacking"]}		
Format	Size (bytes)	Notes
JSON (text)	81	Field names repeated
JSON (whitespace removed)	67	Still has field names
MessagePack (binary)	66	Barely smaller!
Thrift BinaryProtocol	59	Field TAGS not names
Thrift CompactProtocol	34	Var-int encoding
Protocol Buffers	33	Very compact
Key insight: Schema-based encodings replace field NAMES with numeric TAGS (1, 2, 3) → much smaller, and the schema serves as documentation.		

4.4.4 Schema Evolution with Field Tags

FORWARD & BACKWARD COMPATIBILITY

FORWARD COMPATIBILITY: Old code can read new data

BACKWARD COMPATIBILITY: New code can read old data

Adding a new field (tag = 4):

Old code reading new data:

Sees unknown tag 4 → IGNORES it (forward compatible)

New code reading old data:

Tag 4 not present → uses DEFAULT value (backward compat)

BUT: new field must be optional or have a default!

Removing a field:

- Can only remove OPTIONAL fields
- The tag number must NEVER be reused
(old data in the wild still uses that tag)

Changing data types:

- int32 → int64: OK (new code reads old 32-bit values)
- int64 → int32: RISKY (values may be truncated)
- single → repeated: Protobuf OK, Thrift not

GOLDEN RULE: Never change a field's tag number.

Tags are the identity of fields in the binary encoding.

4.5 Avro

Apache Avro uses a schema but takes a **different approach**: no field tags in the binary encoding at all.

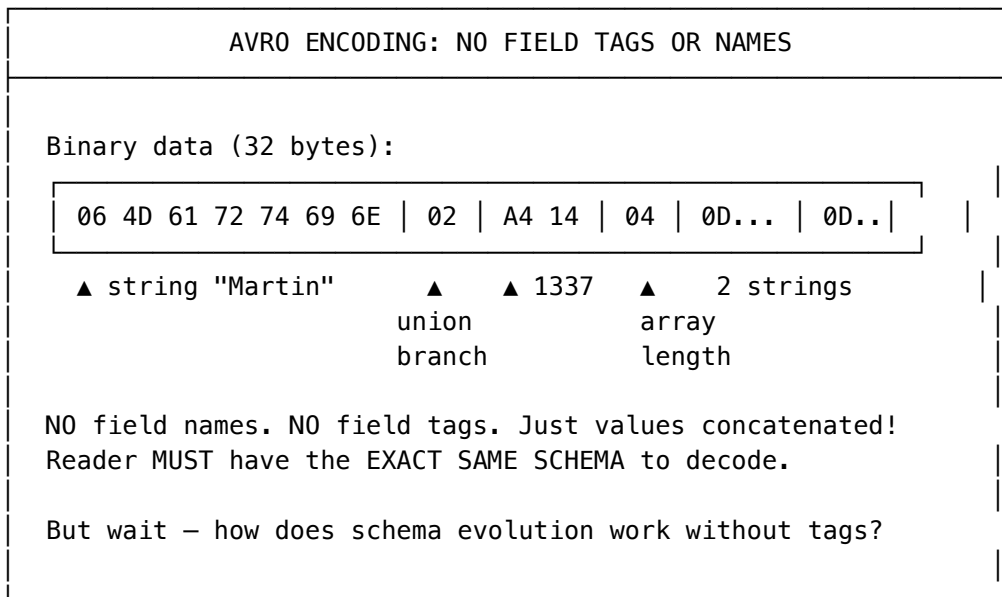
4.5.1 Avro Schema

```

{
  "type": "record",
  "name": "Person",
  "fields": [
    { "name": "userName", "type": "string" },
    { "name": "favoriteNumber", "type": ["null", "long"], "default": null },
    { "name": "interests", "type": { "type": "array", "items": "string" } }
  ]
}

```

4.5.2 Avro Binary Encoding



4.5.3 Writer's Schema vs Reader's Schema

AVRO SCHEMA RESOLUTION

Writer's Schema (v1):

```
userName: string
favoriteNumber: ?
```

Reader's Schema (v2):

```
userName: string
favoriteNumber: ?
favoriteColor: string
```

NEW!

Avro library resolves differences by FIELD NAME:

- userName: in both → transfer
- favoriteNumber: in both → transfer
- favoriteColor: in reader but NOT writer → use default
- (If writer has field reader doesn't → ignore)

Field matching by NAME (not position or tag).
This is why Avro uses names, not numeric tags.

How does the reader know the writer's schema?

- Large file (Hadoop): writer's schema in file header
- DB records: version number → schema registry lookup
- Network RPC: schema negotiated at connection setup

4.5.4 Why Avro for Hadoop?

Avro is ideal for Hadoop because:

1. Schema in file header → self-describing files
2. No code generation required (dynamically typed languages)
3. Schema evolution by field name (not tags)
4. Very compact encoding (no tag overhead per field)
5. Splittable container format (MapReduce can split files)

Protobuf/Thrift require code generation for each schema change → not practical for Hadoop pipelines with hundreds of schemas that evolve independently.

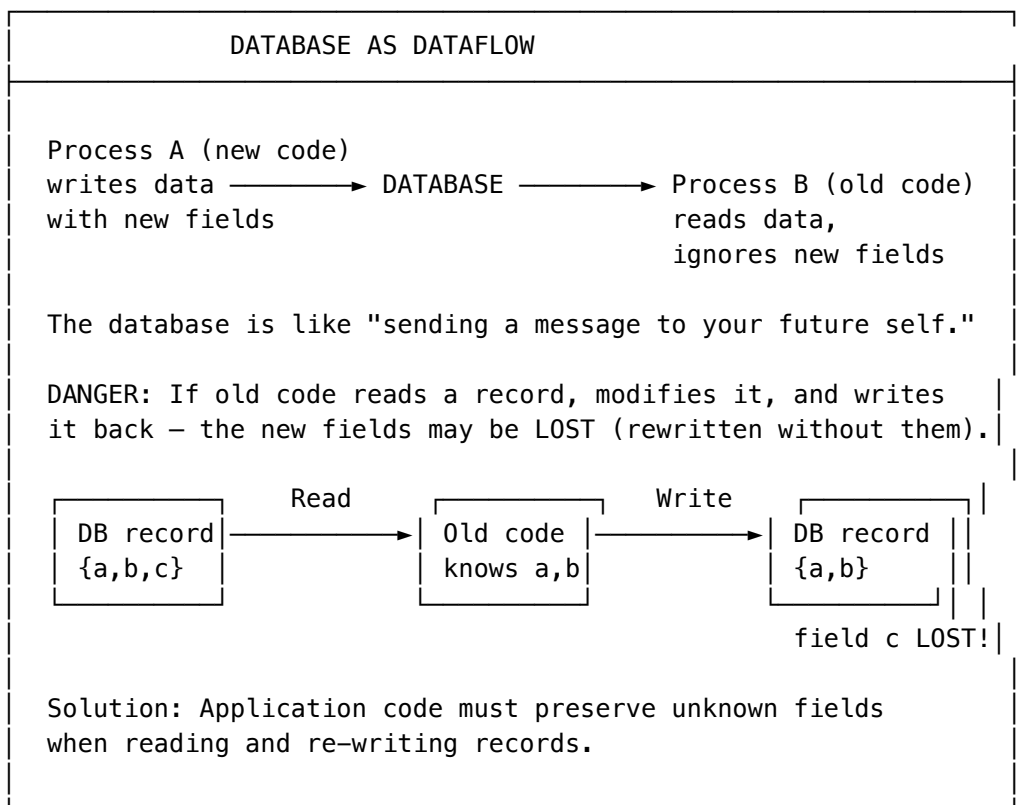
4.6 Schema Evolution Rules

SCHEMA EVOLUTION COMPARISON		
Operation	Protobuf/Thrift	Avro
Add optional field	✓ (new tag)	✓ (with default)
Remove optional field	✓ (never reuse tag)	✓ (keep in writer)
Add required field	x (breaks old)	x (breaks old)
Rename field	✓ (tag unchanged)	x (name = identity)
Change field type	Risky (truncation)	Limited
Field identification	Numeric tags	Field names
Forward compatibility = old readers ignore unknown fields		
Backward compatibility = new readers handle missing fields		
Both directions needed for rolling upgrades where old and new code coexist simultaneously.		

4.7 Modes of Dataflow

Data flows between processes in three main ways:

4.7.1 1. Dataflow Through Databases



4.7.2 2. Dataflow Through Services (REST and RPC)

REST vs RPC

REST (Representational State Transfer):

- Based on HTTP, uses URLs to identify resources
- Formats: JSON (most common), XML
- Good for public APIs, browser clients
- OpenAPI / Swagger for documentation

RPC (Remote Procedure Call):

- Tries to make network calls look like local function calls
- Formats: Protobuf (gRPC), Thrift, Avro
- Good for internal service-to-service communication

PROBLEMS WITH RPC:

Local function call:

- Predictable (returns or throws exception)
- Same parameters = same result
- Can pass references
- Same language
- Negligible latency

Network call:

- May be lost, delayed, or duplicated
- Timeout: did it succeed or not?
- Must serialize all data
- Cross-language needed
- Network latency varies

Modern RPC frameworks (gRPC, Finagle, Rest.li) embrace the difference rather than hiding it:

- Futures/promises for async calls
- Streaming (gRPC bidirectional streams)
- Service discovery

4.7.3 3. Dataflow Through Asynchronous Message Passing

MESSAGE BROKERS / MESSAGE QUEUES

Producer → Message Broker → Consumer
(RabbitMQ,
Kafka, etc.)

Advantages over direct RPC:

- BUFFER: Broker stores messages if consumer is slow
- RETRY: Automatically redeliver failed messages
- DECOUPLE: Producer doesn't need to know consumer
- FAN-OUT: One message → multiple consumers
- ASYNC: Producer doesn't wait for consumer response

Actor Model (Akka, Orleans, Erlang OTP):

- Each actor = single-threaded process with a mailbox
- Actors communicate by sending async messages
- Location transparency: actors can be on different nodes
- No shared state between actors → no locks needed

4.7.4 Rolling Upgrades and Compatibility

WHY BOTH FORWARD AND BACKWARD COMPATIBILITY?

During a rolling upgrade:

Node 1 v2 NEW	Node 2 v2 NEW	Node 3 v1 OLD	Node 4 v1 OLD	Node 5 v1 OLD
------------------	------------------	------------------	------------------	------------------

Node 1 (v2) sends data to Node 4 (v1):

→ v1 must handle v2's data → FORWARD COMPATIBILITY

Node 4 (v1) sends data to Node 1 (v2):

→ v2 must handle v1's data → BACKWARD COMPATIBILITY

Both are needed simultaneously during rollout period.

4.8 Interview Questions

4.8.1 Q1: Why is language-specific serialization (Java Serializable, Python pickle) a bad idea?

It locks you into a single programming language (can't read the data from any other language). It has security vulnerabilities (deserialization can instantiate arbitrary classes). It has poor support for schema evolution and versioning. And it's often slow and produces bloated output. Use a language-agnostic format (JSON, Protobuf,

Avro) instead.

4.8.2 Q2: Compare Protocol Buffers, Thrift, and Avro.

All three use schemas for compact binary encoding. **Protobuf** and **Thrift** use numeric **field tags** — each field has a number that never changes, and this number appears in the binary encoding instead of the field name. **Avro** uses **field names** for resolution and includes no tags in the binary encoding, making it the most compact. Avro requires the writer's schema to be available at read time (stored in file headers or a schema registry). Protobuf/Thrift are better for RPC (code generation); Avro is better for Hadoop/big data (dynamic schemas, self-describing files).

4.8.3 Q3: What is forward and backward compatibility?

Backward compatibility: Newer code can read data written by older code. **Forward compatibility:** Older code can read data written by newer code. Both are needed during rolling upgrades when old and new code versions coexist. Schema evolution rules (don't remove required fields, don't reuse field tags, provide defaults for new fields) enable both directions of compatibility.

4.8.4 Q4: What are the three modes of dataflow?

1. **Through databases:** One process writes, another reads later. Must handle schema evolution in stored data (old records coexist with new records).
2. **Through services (REST/RPC):** Client sends request, server responds. Must agree on API contract with versioning.
3. **Through async message passing:** Producer sends messages to a broker, consumer reads them. Decouples sender and receiver in time and availability.

4.8.5 Q5: Why does Avro not use field tags?

Avro was designed for Hadoop, where you have hundreds of schemas evolving independently, and generating code for each schema change is impractical. By using field **names** instead of numeric tags, Avro enables: (1) dynamic schema resolution at read time without code generation, (2) extremely compact encoding (no tag bytes), (3) easier schema evolution for dynamically-typed languages. The trade-off is that the reader must always have access to the writer's schema.

Based on Chapter 4 of "Designing Data-Intensive Applications" by Martin Kleppmann

5 Chapter 5: Replication

5.1 Why Replicate?

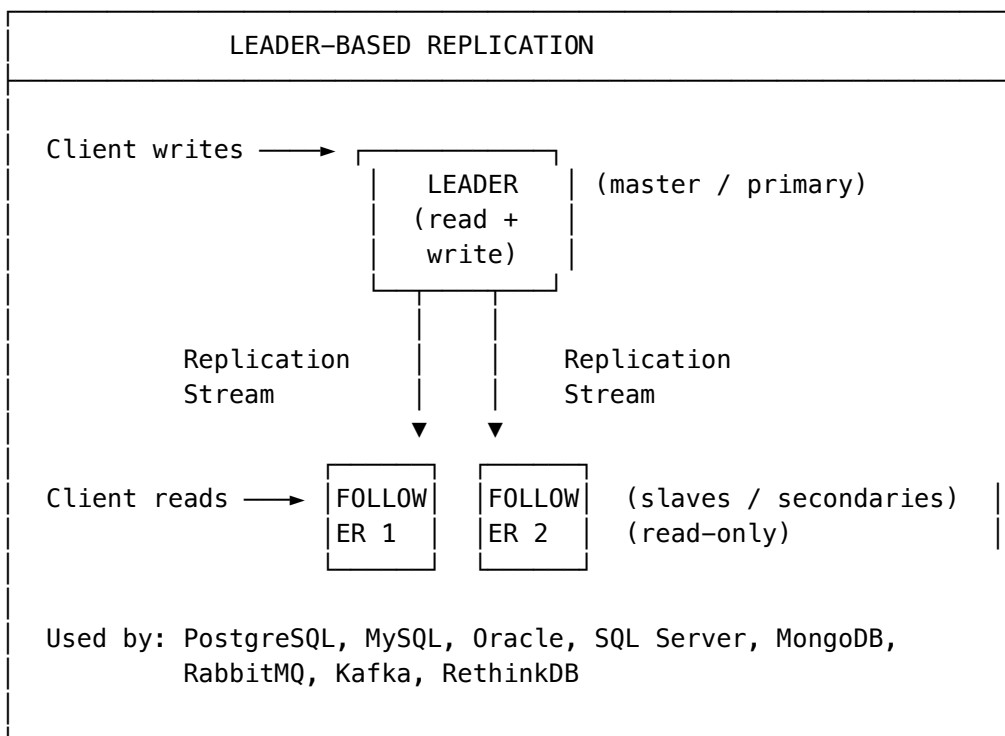
Replication means keeping a copy of the same data on multiple machines connected via a network.

REASONS TO REPLICATE
1. LATENCY: Keep data geographically close to users (read from nearest datacenter) 2. AVAILABILITY: System continues working even if some machines (or entire datacenters) go down 3. READ THROUGHPUT: Scale out read queries across multiple replicas The DIFFICULTY: Handling CHANGES to replicated data. If data never changed, replication would be trivial.

Three popular algorithms: **single-leader**, **multi-leader**, **leaderless**.

5.2 Leaders and Followers

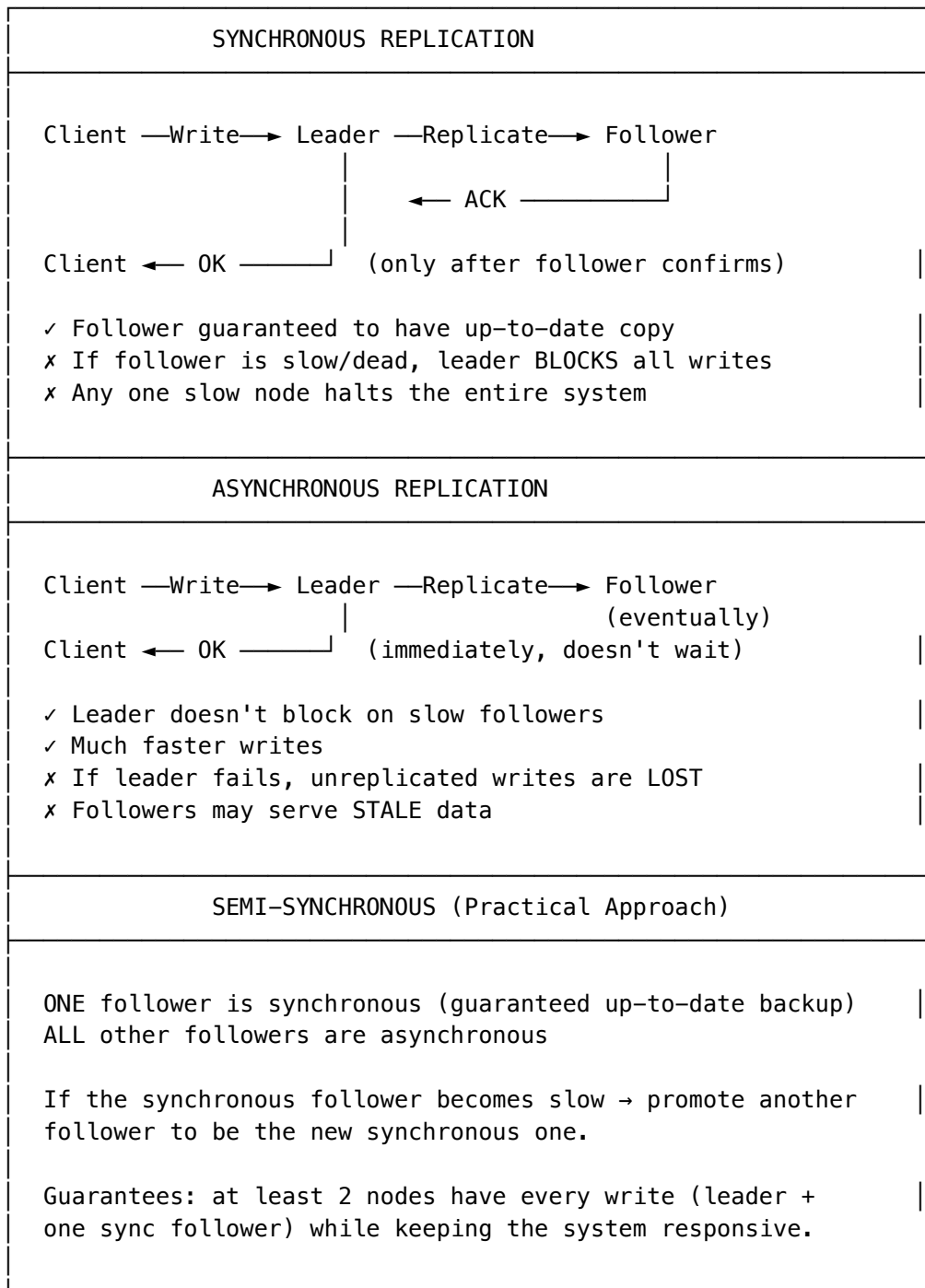
The most common approach: **leader-based replication** (also: master-slave, active-passive).



5.2.1 How It Works

1. One replica is designated the **leader**. All writes go to the leader.
2. The leader writes new data to its local storage.
3. The leader sends data changes to all **followers** via a **replication log** (or change stream).
4. Each follower applies the changes in the same order the leader processed them.
5. Clients can read from any replica (leader or follower), but writes go only to the leader.

5.3 Synchronous vs Asynchronous Replication



5.4 Setting Up New Followers

You can't just copy data files — the data is constantly changing. You can't lock the database (violates availability goals). The standard process:

SETTING UP A NEW FOLLOWER

Step 1: Take a consistent SNAPSHOT of the leader's database
(without locking if possible, e.g., innodb backup)

Step 2: Copy the snapshot to the new follower node

Step 3: Follower connects to leader and requests all changes
that happened SINCE the snapshot
(using the snapshot's position in the replication log)

Step 4: Follower processes the backlog of changes ("caught up")
and then continues processing new changes in real time

The snapshot's position is called:

- PostgreSQL: log sequence number (LSN)
- MySQL: binlog coordinates

5.5 Handling Node Outages

5.5.1 Follower Failure: Catch-Up Recovery

Each follower keeps a log of changes received from the leader. On restart, the follower knows the last transaction it processed, connects to the leader, and requests all changes since that point.

5.5.2 Leader Failure: Failover

Failover is the process of promoting a follower to be the new leader:

LEADER FAILOVER PROCESS

1. DETECT leader failure
→ Typically using a timeout (no heartbeat for 30 seconds)
2. CHOOSE a new leader
→ Election (consensus) or appointed by controller node
→ Best candidate: follower with most up-to-date data
3. RECONFIGURE the system
→ Clients redirected to new leader
→ Other followers start replicating from new leader
→ Old leader (when it comes back) must become a follower

THINGS THAT CAN GO WRONG:

- Async replication → new leader may be MISSING some writes from old leader (data loss!)
- Split brain: TWO nodes both believe they are the leader
→ Both accept writes → DATA DIVERGENCE / CORRUPTION
→ Must have a mechanism to shut down one leader
- Timeout too short → unnecessary failovers (flapping)
Timeout too long → longer recovery time

GitHub incident (2012): MySQL failover, new leader had out-of-date auto-increment counter, issued duplicate primary keys, which were also used as Redis keys → data leak from wrong user accounts. Fixed by shutting down MySQL.

5.6 Problems with Replication Lag

With asynchronous replication, followers can be behind. This **replication lag** causes various anomalies:

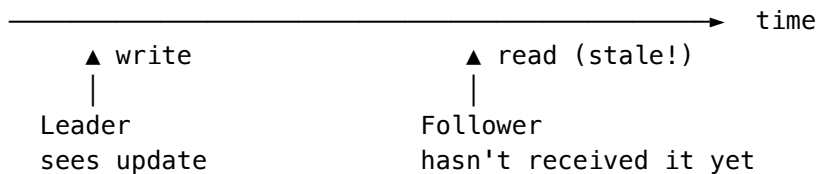
5.6.1 1. Reading Your Own Writes

READ-AFTER-WRITE CONSISTENCY

User writes profile → Leader (write succeeds)

User reads profile → Follower (stale! hasn't replicated)
"Where did my update go?!"

Timeline:



SOLUTIONS:

- Read your own data from LEADER (e.g., own profile)
- Track time of last write; for 1 minute after, read from leader only
- Client remembers timestamp of last write; replica must be at least that up-to-date before serving the read

5.6.2 2. Monotonic Reads

MONOTONIC READS

User reads from Follower 1 → sees comment by User X

User reads from Follower 2 → comment GONE (hasn't replicated)
"The comment appeared and then disappeared – time travel!"

Follower 1 (more up-to-date): Follower 2 (lagging):

Comment: "Hello!"

(no comment yet)

SOLUTION:

Each user always reads from the SAME replica
(e.g., pick replica based on hash of user ID).

Guarantees: if you saw data once, you'll never see
an older state on subsequent reads.

5.6.3 3. Consistent Prefix Reads

CONSISTENT PREFIX READS

A conversation between Mr. Poons and Mrs. Cake:

Actual order:

Mr. Poons: "How far into the future can you see?"

Mrs. Cake: "About ten seconds usually."

But a third-party observer may see:

Mrs. Cake: "About ten seconds usually."

Mr. Poons: "How far into the future can you see?"

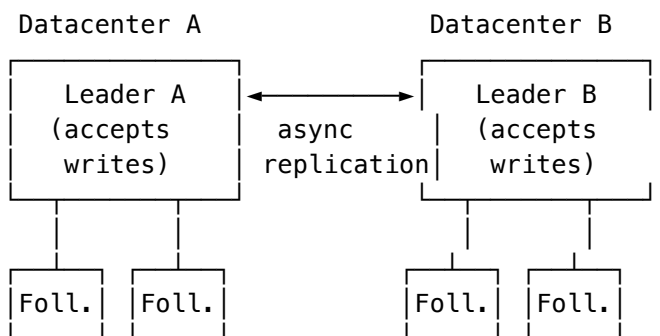
This happens when different partitions replicate at different speeds – causally related writes arrive out of order!

SOLUTION:

Ensure causally related writes are sent to the same partition, or use causal ordering mechanisms.

5.7 Multi-Leader Replication

MULTI-LEADER (ACTIVE/ACTIVE) REPLICATION



USE CASES:

- Multi-datacenter operation (each DC has a leader)
- Clients with offline operation (each device is a "leader")
- Collaborative editing (Google Docs – each user is a "leader")

BIGGEST PROBLEM: Write conflicts

5.7.1 Handling Write Conflicts

WRITE CONFLICT EXAMPLE

User 1 (via Leader A): UPDATE title = "B"

User 2 (via Leader B): UPDATE title = "C"

Both succeed locally. When replicated → CONFLICT!

Leader A: title was A → B Leader B: title was A → C

Leader A receives B → C Leader B receives A → B

Now what?! Both have conflicting changes.

CONFLICT RESOLUTION STRATEGIES:

1. Last Write Wins (LWW):

Attach a timestamp; highest timestamp wins.

Simple but LOSSY – silently drops conflicting writes.

Used by Cassandra, some DynamoDB configs.

2. Merge values:

Concatenate: "B/C" or store both versions (siblings).

Application resolves later (e.g., show both to user).

3. Custom conflict resolution logic:

Application provides a handler:

- On write: handler called when conflict detected
- On read: all versions stored; handler called at read time to merge (CouchDB approach)

4. CRDTs (Conflict-free Replicated Data Types):

Data structures that can be merged automatically

(e.g., G-Counter, OR-Set). Used by Riak.

5. Operational Transformation (Google Docs):

Transform concurrent edits to converge.

5.8 Leaderless Replication

No leader at all. Clients send writes to **several replicas directly** (or via a coordinator node that doesn't enforce ordering).

LEADERLESS REPLICATION (Dynamo-style)
Client write → Replica 1 ✓ (success) → Replica 2 ✓ (success) → Replica 3 ✗ (down – missed write) Write succeeds if 2 out of 3 replicas confirm ($w=2$) Client read → Replica 1 → value v2 (up-to-date) → Replica 2 → value v2 (up-to-date) → Replica 3 → value v1 (STALE!) Read succeeds if 2 out of 3 replicas respond ($r=2$) Client sees both v1 and v2, picks the newer one (v2) Used by: Amazon DynamoDB, Riak, Cassandra, Voldemort

5.8.1 Quorum Condition

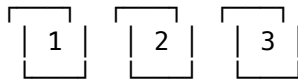
QUORUM: $w + r > n$

n = total replicas
 w = write quorum (number of nodes that must confirm write)
 r = read quorum (number of nodes that must respond to read)

If $w + r > n$, at least ONE node in the read set will have the latest write.

Typical: $n=3, w=2, r=2 \rightarrow 2+2=4 > 3 \checkmark$

$n=3$ replicas:



Write $w=2$:

$\checkmark$ $\checkmark$ $\times$

Read $r=2$:

$\checkmark$ $\times$ $\checkmark$



└ Node 1 was in both sets $\rightarrow$ has latest value

TRADE-OFFS:

- Larger $w \rightarrow$ slower writes, but more durable
- Larger $r \rightarrow$ slower reads, but more likely to get latest
- $w=1, r=n \rightarrow$ fast writes, slow reads
- $w=n, r=1 \rightarrow$ slow writes, fast reads

WARNING: Even with $w+r>n$, edge cases can return stale reads:

- Sloppy quorums
- Concurrent writes (which "wins"?)
- Concurrent read and write
- Failed write on some replicas (not rolled back)

5.8.2 Anti-Entropy and Read Repair

How stale replicas catch up:

READ REPAIR:

Client reads from multiple replicas, detects stale value, writes the newer value back to the stale replica. Works well for frequently-read data.

ANTI-ENTROPY PROCESS:

Background process compares data between replicas and copies missing data. Doesn't preserve ordering.

5.8.3 Version Vectors

To detect concurrent writes vs. causally-ordered writes, leaderless systems use **version vectors** (generalization of version clocks):

VERSION VECTORS

Each replica maintains a version number per key.
A version vector = collection of version numbers from all replicas.

Example with 3 replicas (A, B, C):

Write 1 (to A): version vector = {A:1, B:0, C:0}

Write 2 (to B): version vector = {A:1, B:1, C:0}

Are these concurrent? Compare element-wise:

{A:1, B:0, C:0} vs {A:1, B:1, C:0}

A: $1 \leq 1$ ✓

B: $0 < 1$ → First happened before second

Write 1 HAPPENED BEFORE Write 2 (not concurrent)

Write 3 (to A): {A:2, B:0, C:0}

Write 4 (to B): {A:1, B:1, C:0}

Neither dominates → CONCURRENT → need conflict resolution

5.9 Interview Questions

5.9.1 Q1: Compare single-leader, multi-leader, and leaderless replication.

Single-leader: All writes go to one leader, replicated to followers. Simple, no write conflicts, but leader is a bottleneck and single point of failure. **Multi-leader:** Multiple nodes accept writes; changes replicated between leaders. Better for multi-datacenter setups but introduces write conflicts that must be resolved. **Leaderless:** No leader; clients write to multiple replicas using quorums ($w + r > n$). High availability, no failover needed, but complex conflict resolution and weaker consistency guarantees.

5.9.2 Q2: What is the quorum condition and why is it important?

The quorum condition is $w + r > n$, where n = total replicas, w = write quorum, r = read quorum. It ensures that the set of nodes read from and the set of nodes written to **overlap** — at least one node in every read has the latest write. This provides a probabilistic guarantee of reading up-to-date data without requiring a leader. However, it's not foolproof: sloppy quorums, concurrent writes, and network partitions can still cause stale reads.

5.9.3 Q3: Explain three problems caused by replication lag.

1. **Read-after-write:** User writes data, then reads from a stale follower and doesn't see their own write. Fix: read own data from leader.
2. **Non-monotonic reads:** User reads from a fresh follower, then a stale one, and sees data "go back in time." Fix: always read from the same replica.
3. **Consistent prefix reads:** Causally related writes appear in wrong order because different partitions replicate at different speeds. Fix: ensure causal writes go to the same partition.

5.9.4 Q4: How does leader failover work, and what can go wrong?

Failover detects leader failure (via timeout), elects a follower as the new leader, and reconfigures clients and other followers. Problems: (1) **Data loss:** async replication means the new leader may be missing some writes. (2) **Split brain:** two nodes both believe they're leader, both accept writes → data corruption. (3) **Wrong**

timeout: too short causes unnecessary failovers; too long means longer downtime. GitHub's 2012 incident showed how stale auto-increment counters after failover caused duplicate primary keys and data leaks.

5.9.5 Q5: What are CRDTs and when are they useful?

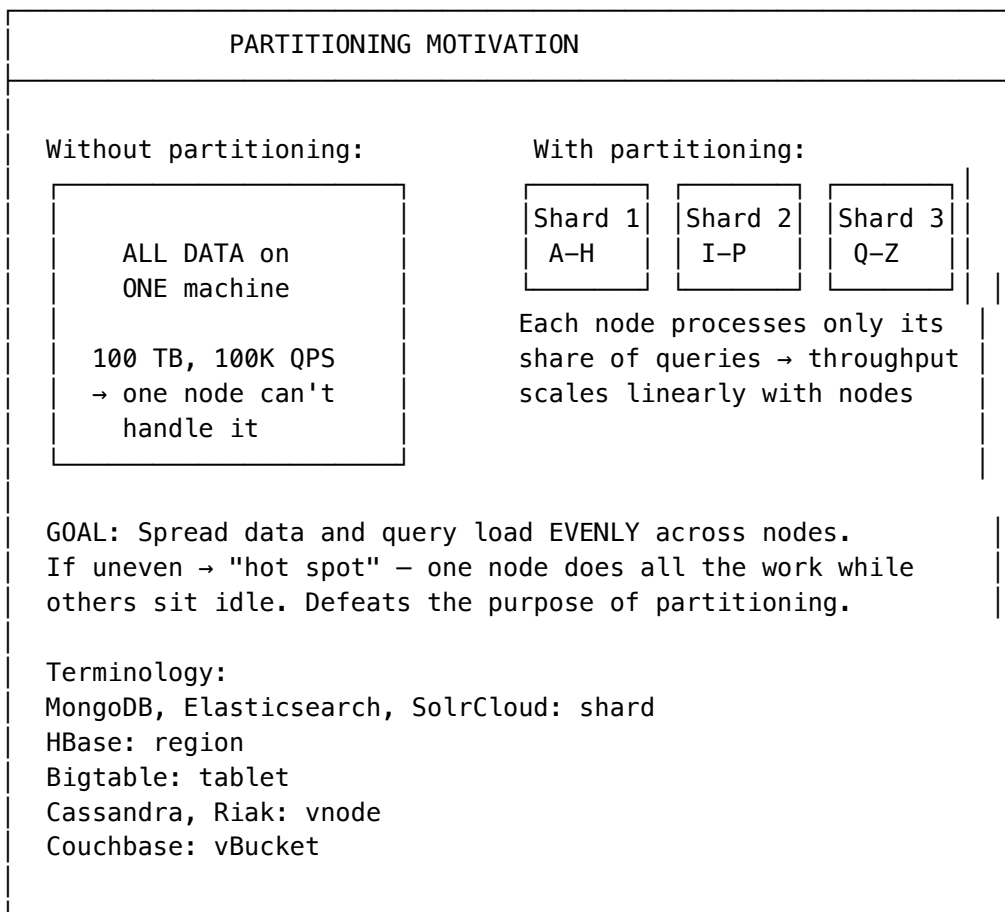
Conflict-free Replicated Data Types are data structures designed to be merged automatically without conflicts. Examples: G-Counter (grow-only counter), PN-Counter (increment/decrement), G-Set (grow-only set), OR-Set (observed-remove set). They're useful in multi-leader and leaderless systems where concurrent writes to the same data are common. CRDTs guarantee eventual convergence regardless of the order in which updates are applied. Used by Riak for distributed counters and sets.

Based on Chapter 5 of "Designing Data-Intensive Applications" by Martin Kleppmann

6 Chapter 6: Partitioning

6.1 Why Partition?

Partitioning (also called **sharding**) splits a large dataset across multiple nodes so that each node stores a subset of the data.



Partitioning is usually combined with replication — each partition is replicated on multiple nodes for fault tolerance.

6.2 Partitioning of Key-Value Data

6.2.1 Key Range Partitioning

KEY RANGE PARTITIONING

Assign a continuous range of keys to each partition:

Partition 1 A – G	Partition 2 H – N	Partition 3 O – Z
Keys sorted within part.	Keys sorted within part.	Keys sorted within part.

- ✓ Efficient range queries (scan from start key to end key)
- ✓ Keys sorted within partition → efficient range scans
- ✗ Risk of HOT SPOTS if access pattern is skewed

Example hot spot:

Sensor data partitioned by timestamp (YYYY-MM-DD)

All writes for TODAY go to the same partition!

Fix: Prefix key with sensor_id → sensor42/2024-03-15

Used by: HBase, Bigtable, RethinkDB, MongoDB (before 2.4)

6.2.2 Hash Partitioning

HASH PARTITIONING

$\text{partition} = \text{hash}(\text{key}) \bmod \text{num_partitions}$

Key "user123" → $\text{hash}(\text{"user123"}) = 0x7A3F\dots$
→ $0x7A3F\dots \bmod 4 = \text{partition } 2$

Hash function distributes keys uniformly:

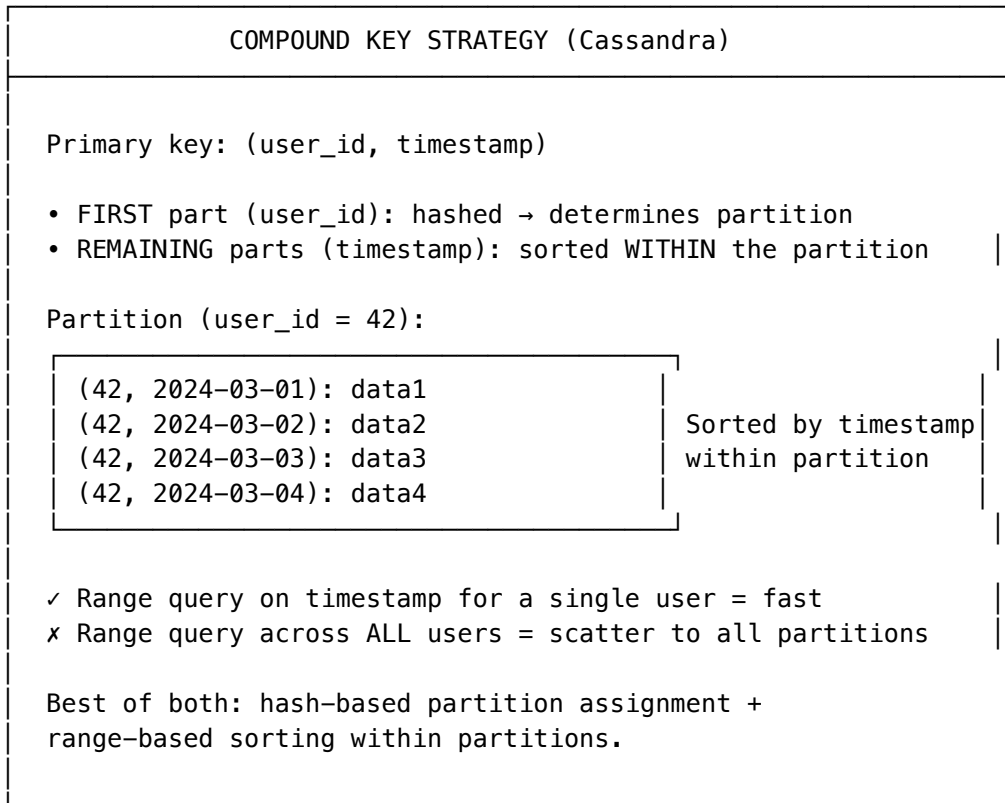
Part. 0 hash 0x00–3F	Part. 1 hash 0x40–7F	Part. 2 hash 0x80–BF	Part. 3 hash 0xC0–FF
----------------------------	----------------------------	----------------------------	----------------------------

- ✓ Evenly distributes keys → no hot spots (usually)
- ✗ Loses ability to do efficient range queries
(adjacent keys scattered across different partitions)

IMPORTANT: Use a good hash function (MD5, Murmur3),
NOT language built-in hash (Java's hashCode() is not
consistent across processes!)

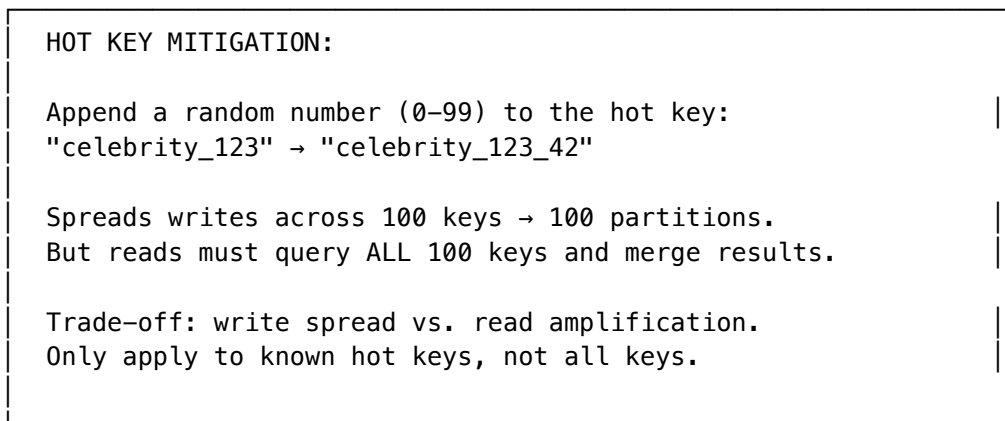
Used by: Cassandra, DynamoDB, MongoDB, Riak, Voldemort

6.2.3 Compound Key (Cassandra's Approach)



6.2.4 Dealing with Hot Spots

Even with hashing, a single hot key (e.g., a celebrity's user ID) causes all requests to hit one partition.



6.3 Partitioning and Secondary Indexes

Secondary indexes don't map neatly to partitions. Two approaches:

6.3.1 Document-Partitioned Index (Local Index)

LOCAL INDEX (Document-Partitioned)

Each partition maintains its OWN secondary index covering only the documents in that partition.

Partition 0:

Docs: car191 (red),
car392 (blue)

Index:
color:red → [191]
color:blue → [392]

Partition 1:

Docs: car389 (yellow)
car556 (red)

Index:
color:red → [556]
color:yellow → [389]

Query "color=red" must hit ALL partitions (scatter/gather):

- Partition 0: finds car191
- Partition 1: finds car556
- Merge results

- ✓ Writes are local (update one partition's index)
- ✗ Reads are expensive (scatter/gather across all partitions)

Used by: MongoDB, Riak, Cassandra, Elasticsearch, VoltDB

6.3.2 Term-Partitioned Index (Global Index)

GLOBAL INDEX (Term-Partitioned)

Index itself is partitioned separately from the data.

Index Partition 0 (a-m):

color:blue → [392]

Index Partition 1 (n-z):

color:red → [191,556]
color:yellow → [389]

Query "color=red" → go to index partition 1 only.

- ✓ Reads are efficient (query one index partition)
- ✗ Writes are slow and complex (may need to update index partitions on DIFFERENT nodes → distributed transaction)
- ✗ Index updates are often ASYNCHRONOUS

Used by: DynamoDB (global secondary indexes, async update)

6.4 Rebalancing Partitions

When adding/removing nodes, partitions need to be redistributed.

6.4.1 Strategies

REBALANCING STRATEGIES

- ✗ DON'T: hash mod N
If N (number of nodes) changes, MOST keys move.
Adding one node = rehash everything = terrible.
- ✓ FIXED NUMBER OF PARTITIONS:
Create many more partitions than nodes (e.g., 1000 partitions on 10 nodes = 100 per node).
Adding a node → steal a few partitions from each node.
Partition count never changes; only assignment changes.

Before: 3 nodes, 12 partitions (4 each)

Node A: [P1, P2, P3, P4]

Node B: [P5, P6, P7, P8]

Node C: [P9, P10, P11, P12]

After adding Node D:

Node A: [P1, P2, P3]

Node B: [P5, P6, P7]

Node C: [P9, P10, P11]

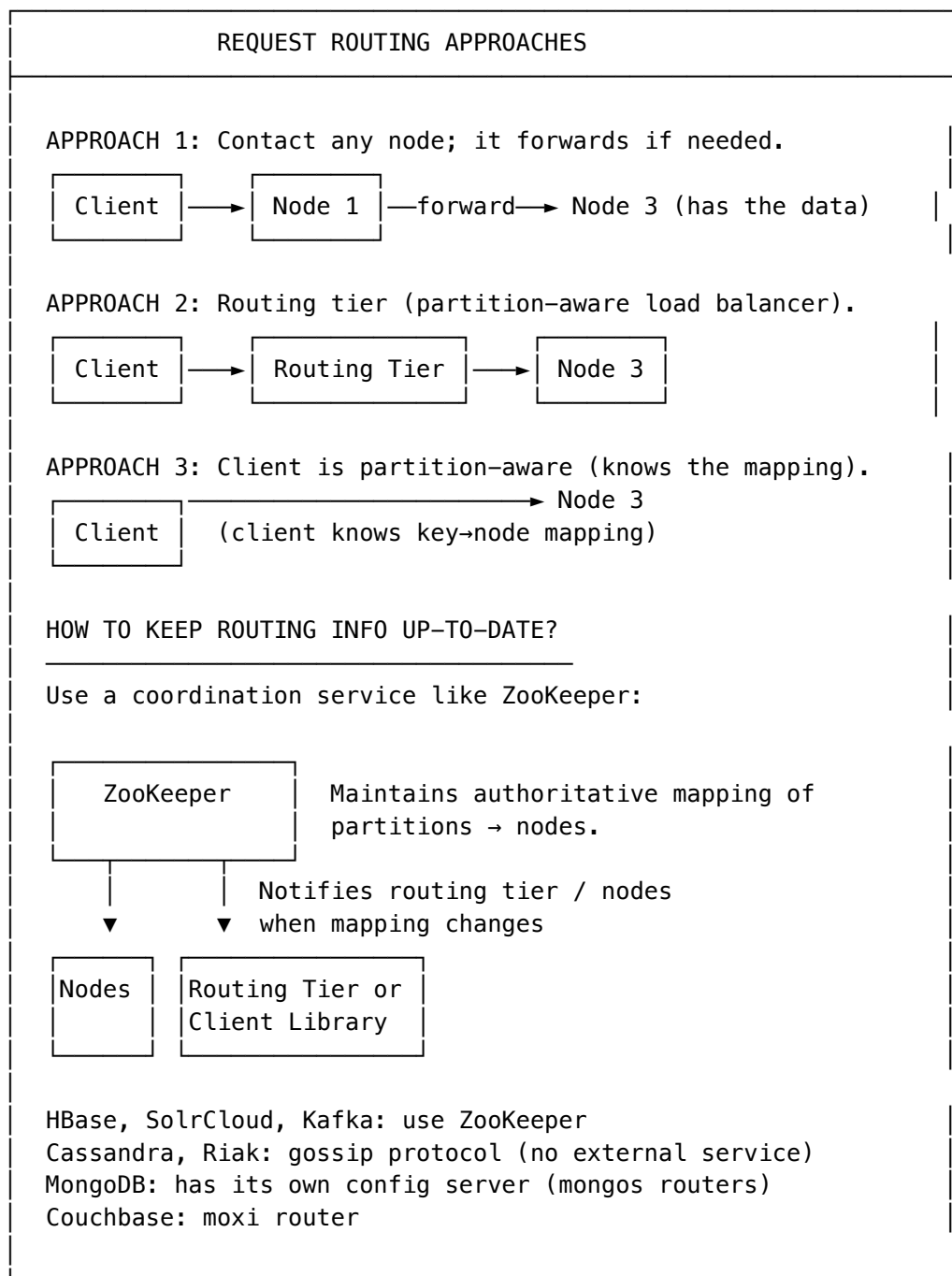
Node D: [P4, P8, P12] ← stole one from each

Used by: Riak, Elasticsearch, Couchbase, Voldemort

- ✓ DYNAMIC PARTITIONING:
Partition splits when it exceeds a size threshold.
Partition merges when it shrinks below a threshold.
Number of partitions adapts to data volume.
Used by: HBase, RethinkDB, MongoDB
- ✓ CONSISTENT HASHING (Virtual Nodes):
Each node gets multiple positions on a hash ring.
Adding a node → it takes over part of the ring.
Only K/N keys move on average (K =keys, N =nodes).
Used by: Cassandra, DynamoDB

6.5 Request Routing

How does a client know which node to connect to for a given key?



6.6 Interview Questions

6.6.1 Q1: What is the difference between key range and hash partitioning?

Key range: Each partition owns a contiguous range of keys (A-G, H-N, O-Z). Supports efficient range queries but risks hot spots if access is skewed (e.g., all writes for today's date go to one partition). **Hash partitioning:** Keys are hashed and partitions own hash ranges. Distributes keys evenly (no hot spots) but loses the ability to do range queries since adjacent keys are scattered. Cassandra's compound keys combine both: hash the first part for partition assignment, sort the remaining parts within each partition.

6.6.2 Q2: Explain scatter/gather and why it's a problem.

In a **document-partitioned (local) secondary index**, each partition maintains its own index covering only its local documents. A query on the secondary index (e.g., "find all red cars") must be sent to **every partition**

(scatter), and results must be merged (gather). This is expensive and adds tail latency — the response is only as fast as the slowest partition. The alternative is a **global (term-partitioned) index** which is efficient for reads but makes writes slower and more complex.

6.6.3 Q3: Why shouldn't you use hash mod N for partitioning?

If N (number of nodes) changes by even one, $\text{hash}(\text{key}) \bmod N$ changes for almost every key, requiring nearly all data to be moved between nodes. This is catastrophically expensive. Instead, use **fixed number of partitions** (more partitions than nodes, reassign whole partitions), **dynamic partitioning** (split/merge partitions based on size), or **consistent hashing** (only K/N keys move when a node is added/removed).

6.6.4 Q4: How does request routing work in partitioned databases?

Three approaches: (1) **Any node**: client contacts any node, which forwards the request to the correct partition owner. (2) **Routing tier**: a partition-aware proxy/load balancer routes requests to the correct node. (3) **Client awareness**: the client itself maintains the partition-to-node mapping. The mapping is kept up-to-date via a coordination service like ZooKeeper (used by HBase, Kafka) or a gossip protocol (used by Cassandra, Riak).

6.6.5 Q5: What is consistent hashing and how does it help with rebalancing?

Consistent hashing maps both keys and nodes onto a circular hash ring. Each key is assigned to the nearest node clockwise on the ring. When a node is added, it takes over keys from the next node on the ring; when removed, its keys move to the next node. Only K/N keys move on average (where K = total keys, N = nodes), compared to nearly all keys with hash mod N. **Virtual nodes** (each physical node gets multiple positions on the ring) improve balance and ensure even distribution.

Based on Chapter 6 of "Designing Data-Intensive Applications" by Martin Kleppmann

7 Chapter 7: Transactions

7.1 The Slippery Concept of a Transaction

A **transaction** groups several reads and writes into a logical unit. The entire transaction either **commits** (succeeds) or **aborts** (rolls back) — no partial results.

WHY TRANSACTIONS EXIST
Without transactions, many things can go wrong: • Database software/hardware fails mid-write • Application crashes mid-operation • Network interruption cuts off application from database • Multiple clients write simultaneously → overwrite each other • Client reads partially-updated data • Race conditions between clients Transactions SIMPLIFY the programming model: Instead of handling every possible failure mode, the database guarantees "all or nothing." Not every application needs transactions. But dismissing them outright leads to subtle bugs that are hard to reproduce.

7.2 ACID Guarantees

A C I D
A = ATOMICITY NOT about concurrency (that's Isolation). Means: If a transaction fails partway through, ALL changes are rolled back. No partial writes. "Abortability" would be a better name. C = CONSISTENCY Application-level concept, not a database guarantee. Means: Data satisfies certain invariants (e.g., credits = debits in accounting). The APPLICATION is responsible for defining correct transactions. The database just provides A, I, D. I = ISOLATION Concurrently executing transactions don't interfere. Ideally: each transaction runs as if it's the ONLY one running (serializable isolation). In practice: databases offer WEAKER isolation levels for performance reasons. D = DURABILITY Once a transaction commits, its data is safe even if the system crashes. Implemented via WAL, replication, or writing to non-volatile storage. Note: "ACID" was coined by Theo Härder and Andreas Reuter in 1983. It's a marketing term – the definitions are imprecise and different databases implement them differently.

7.2.1 BASE — The NoSQL Alternative

BASE = Basically Available, Soft state, Eventually consistent Even MORE vague than ACID. Essentially means: "We don't guarantee ACID, but we'll try our best." Used as a marketing counter to ACID by NoSQL databases. Not particularly useful as a precise term.
--

7.3 Single-Object and Multi-Object Operations

SINGLE-OBJECT writes:

Almost every database provides atomicity for writes to a single object (e.g., one row, one document, one key).

- Atomicity: WAL for crash recovery
- Isolation: Lock on the object
- Compare-and-set: Write only if value hasn't changed

These are NOT transactions in the usual sense.

MULTI-OBJECT transactions:

Coordinate writes to MULTIPLE objects (rows, documents, keys).

Needed when:

- Foreign keys must be consistent across tables
- Document DB: denormalized data must be updated together
- Secondary indexes must match the primary data

Example (email app):

1. Insert new email into mailbox
2. Update unread counter

Both must succeed or both must fail → need multi-object txn.

7.4 Weak Isolation Levels

True serializability has a performance cost. Most databases use **weaker** isolation levels:

7.4.1 Level 1: Read Committed

READ COMMITTED

Two guarantees:

1. No dirty reads: You only read data that has been COMMITTED
2. No dirty writes: You only overwrite data that has been COMMITTED

DIRTY READ (prevented):

Tx1: SET x = 3	(not yet committed)
Tx2: READ x	→ sees old value (2), not 3
Tx1: COMMIT	
Tx2: READ x	→ NOW sees 3

DIRTY WRITE (prevented):

Alice: UPDATE listing SET buyer = 'Alice'	
Bob: UPDATE listing SET buyer = 'Bob'	(must wait
UPDATE invoice SET payer = 'Bob'	for Alice's
	commit/abort)

Implementation:

- Dirty writes: Row-level locks (held until commit)
- Dirty reads: Database remembers both old and new values; serves OLD value to other transactions until commit. (Not locks – locks would kill read performance.)

Default level in: PostgreSQL, SQL Server, Oracle

7.4.2 Level 2: Snapshot Isolation (Repeatable Read)

SNAPSHOT ISOLATION (MVCC)

Problem with Read Committed – NON-REPEATABLE READ:

Alice has two accounts: Account 1 = \$500, Account 2 = \$500
She transfers \$100 from Acct 1 to Acct 2:

Transfer Tx:	Alice reads her accounts:
	READ Acct 1 → \$500
UPDATE Acct 1 SET bal=400	
UPDATE Acct 2 SET bal=600	
COMMIT	
	READ Acct 2 → \$600
Alice sees: \$500 + \$600 = \$1100 ???	
Or if reads reversed: \$400 + \$500 = \$900 ???	
Either way, she sees an INCONSISTENT snapshot!	

SNAPSHOT ISOLATION fixes this:

Each transaction reads from a CONSISTENT SNAPSHOT of the database – the state at the time the transaction started.

Implementation: MVCC (Multi-Version Concurrency Control)

Each row has multiple versions:

Row: Account 1

created_by: tx5	created_by: tx13
deleted_by: tx13	deleted_by: -
balance: 500	balance: 400

Tx12 (started before tx13) reads the old version (500)

Tx15 (started after tx13) reads the new version (400)

VISIBILITY RULES:

A transaction can see a row version if:

- created_by a transaction that committed BEFORE this one
- NOT deleted_by a transaction that committed BEFORE this one

Used by: PostgreSQL, MySQL/InnoDB, Oracle, SQL Server

7.4.3 Problem: Lost Updates

LOST UPDATE PROBLEM

Two transactions read-modify-write the same value:

Counter = 42

Tx1: READ counter → 42

Tx2: READ counter → 42

Tx1: WRITE counter = 42 + 1 = 43

Tx2: WRITE counter = 42 + 1 = 43 ← Should be 44!

Tx2's increment is LOST.

SOLUTIONS:

1. Atomic operations:

UPDATE counters SET value = value + 1 WHERE key = 'foo';
Database handles concurrency internally.

2. Explicit locking (SELECT ... FOR UPDATE):

SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
-- Other transactions wait here
UPDATE accounts SET balance = balance + 100;

3. Compare-and-set:

UPDATE wiki SET content = 'new' WHERE id = 1
AND content = 'old';
Fails if content was modified concurrently.

4. Automatic detection (PostgreSQL, Oracle, SQL Server):

Database detects lost updates and aborts one transaction.
MySQL/InnoDB does NOT detect this! Beware.

7.4.4 Problem: Write Skew and Phantoms

WRITE SKEW

Example: Hospital requires at least 1 doctor on call.

Currently on call: Alice AND Bob (2 doctors)

Alice's Tx:	Bob's Tx:
SELECT COUNT(*) FROM doctors	SELECT COUNT(*) FROM doctors
WHERE on_call = true;	WHERE on_call = true;
→ 2 (OK, can leave)	→ 2 (OK, can leave)
UPDATE doctors SET on_call	UPDATE doctors SET on_call
= false WHERE name='Alice';	= false WHERE name='Bob';
COMMIT	COMMIT

Result: ZERO doctors on call! Both checked, both left.
Neither violated the constraint individually.

This is WRITE SKEW – a generalization of lost updates involving DIFFERENT objects.

PHANTOM: The initial SELECT reads rows that match a condition. Another transaction INSERT/UPDATE/DELETE changes which rows match → the original transaction's decision was based on stale data. The new rows are "phantoms."

More examples of write skew:

- Meeting room double-booking
- Multiplayer game: two players move to same position
- Claiming a unique username
- Double-spending in financial systems

SOLUTIONS:

- Serializable isolation (eliminates all anomalies)
- Materializing conflicts: create a lock table
(e.g., rows for each time slot → SELECT FOR UPDATE)

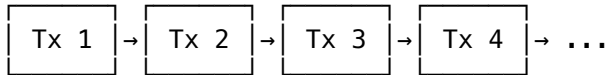
7.5 Serializability

The strongest isolation level: guarantees transactions execute as if they ran one at a time, serially.

7.5.1 Approach 1: Actual Serial Execution

ACTUAL SERIAL EXECUTION

Run transactions one at a time on a SINGLE THREAD.



Became feasible because:

- RAM is cheap → entire dataset fits in memory
- OLTP transactions are short and fast

Requirements:

- Transactions must be SHORT (no waiting for user input)
- Use STORED PROCEDURES (submit entire tx as one request)
- Dataset must fit in memory
- Write throughput must be low enough for single core

Used by: VoltDB, Redis, Datomic

Partitioned serial execution:

Each partition has its own serial thread.

Cross-partition transactions require coordination (locking across partitions → much slower).

7.5.2 Approach 2: Two-Phase Locking (2PL)

TWO-PHASE LOCKING (2PL)

Rules:

- To READ an object: acquire a SHARED lock
- To WRITE an object: acquire an EXCLUSIVE lock
- If Tx A holds shared lock, Tx B wanting exclusive → WAITS
- If Tx A holds exclusive lock, ALL others → WAIT
- Locks held until END of transaction (commit or abort)

Phase 1 (Growing): Acquire locks

Phase 2 (Shrinking): Release locks (at commit/abort)

```
Tx1: LOCK(A, shared)    → reads A
Tx2: LOCK(A, shared)    → reads A (shared OK)
Tx1: LOCK(A, exclusive) → BLOCKED (Tx2 has shared lock)
Tx2: COMMIT → releases lock
Tx1: acquires exclusive → writes A → COMMIT
```

Prevents ALL anomalies including write skew and phantoms.

Problems:

- Performance: transactions spend a lot of time WAITING
- Deadlocks: Tx1 waits for Tx2, Tx2 waits for Tx1
→ Database detects deadlocks and aborts one transaction
- At high percentiles, response time can be very long
- One slow transaction blocks many others

PREDICATE LOCKS (for phantoms):

Lock not just existing rows but the CONDITION itself:

"Lock all rows WHERE room=123 AND time BETWEEN 12:00-13:00"

Any INSERT matching this condition must also acquire the lock.

INDEX-RANGE LOCKS (practical approximation):

Lock a larger range via the index (e.g., all bookings for room 123, or all bookings between 12:00-13:00).

Coarser-grained but much faster than per-row predicate locks.

Used by: MySQL/InnoDB (SERIALIZABLE mode), SQL Server

7.5.3 Approach 3: Serializable Snapshot Isolation (SSI)

SERIALIZABLE SNAPSHOT ISOLATION (SSI)

An OPTIMISTIC concurrency control approach:

2PL (pessimistic):
"If anything might go wrong, WAIT."
Block first, proceed only when safe.

SSI (optimistic):
"Let it proceed, CHECK at commit time."
If a conflict occurred → ABORT and retry.
If no conflict → COMMIT.

Based on snapshot isolation + conflict detection:

The database tracks two things:

1. STALE READS: Did another committed transaction modify data that this transaction read? (MVCC check)
→ If yes, abort this transaction.
2. STALE CONDITIONS: Did another committed transaction write data that affects the result of a query this transaction executed? (Write after read detection)
→ If yes, abort this transaction.

ADVANTAGES over 2PL:

- No blocking – transactions never wait for locks
- No deadlocks
- Better performance (especially for read-heavy workloads)
- Reads from a consistent snapshot (no lock overhead)

DISADVANTAGES:

- Aborted transactions must be retried
- Higher abort rate under high contention
- Wasted work if transactions are long-running

Used by: PostgreSQL (9.1+ SERIALIZABLE), FoundationDB, CockroachDB

7.5.4 Comparison of Serialization Approaches

Aspect	Actual Serial	2PL	SSI
Strategy	Pessimistic (one at a time)	Pessimistic (locks)	Optimistic (detect conflicts)
Concurrency	None (single thread)	Limited (blocked by locks)	High (snapshot reads)
Deadlocks	Impossible	Possible (detected)	Impossible
Performance	Good if in-memory, low contention	Poor under contention	Good, but retries on abort
Phantoms	Prevented (serial)	Predicate/range locks	Detected at commit
Cross-partition	Very expensive	Possible but slow	Works across partitions

7.6 Interview Questions

7.6.1 Q1: What does each letter in ACID mean precisely?

Atomicity: All writes in a transaction succeed or all are rolled back — no partial results. (Not about concurrency — “abortability” is a better name.) **Consistency:** Application-level invariants are maintained — this is the application’s responsibility, not the database’s. **Isolation:** Concurrent transactions don’t interfere with each other; ideally each behaves as if running alone. **Durability:** Once committed, data survives crashes — implemented via WAL, replication, non-volatile storage.

7.6.2 Q2: Explain the difference between Read Committed and Snapshot Isolation.

Read Committed prevents dirty reads (reading uncommitted data) and dirty writes (overwriting uncommitted data). But it allows non-repeatable reads: within one transaction, reading the same data twice may give different results if another transaction commits in between. **Snapshot Isolation** fixes this by giving each transaction a consistent snapshot of the database as of the transaction’s start time, using MVCC (multi-version concurrency control). All reads within the transaction see the same snapshot, regardless of concurrent commits.

7.6.3 Q3: What is write skew and how do you prevent it?

Write skew occurs when two transactions read the same data, make decisions based on what they read, then write to **different** objects — causing a violation of an invariant that depends on both objects. Example: two doctors both check that 2 doctors are on call, then both go off call → zero on call. Prevention: (1) **Serializable isolation** (2PL or SSI) prevents it automatically. (2) **Materializing conflicts:** create explicit lock rows for the constraint, then use SELECT FOR UPDATE. (3) **Explicit application-level locking.**

7.6.4 Q4: Compare 2PL and SSI.

2PL (pessimistic): Transactions acquire shared/exclusive locks and wait for conflicting locks to be released. Prevents all anomalies but causes blocking, deadlocks, and poor performance under contention. **SSI** (optimistic): Transactions proceed without locks using snapshot reads. At commit time, the database checks whether the transaction’s reads are still valid. If a conflict is detected, the transaction is aborted and must retry. SSI has higher concurrency and no deadlocks but wastes work when transactions are aborted.

7.6.5 Q5: When is actual serial execution appropriate?

Actual serial execution works when: (1) the dataset fits in RAM (no disk I/O delays), (2) transactions are very short (OLTP, not OLAP), (3) write throughput is low enough for a single CPU core, (4) transactions are encapsulated as stored procedures (no multi-round-trip interactive transactions). Used by VoltDB, Redis, and Datomic. Cross-partition transactions degrade performance significantly because they require coordination across serial threads.

Based on Chapter 7 of “Designing Data-Intensive Applications” by Martin Kleppmann

8 Chapter 8: The Trouble with Distributed Systems

8.1 Faults and Partial Failures

SINGLE MACHINE vs DISTRIBUTED SYSTEM
SINGLE MACHINE: • Deterministic: same operation → same result • Either works correctly or fails completely • Hardware fault → total failure (crash, bluescreen) • Software bugs are usually reproducible DISTRIBUTED SYSTEM: • NON-DETERMINISTIC: same operation may succeed or fail • PARTIAL FAILURE: some parts work, some don't • You may not even KNOW if something succeeded • Failures are often not reproducible The defining characteristic: PARTIAL FAILURES that are non-deterministic. This makes distributed systems fundamentally harder to reason about.

8.2 Unreliable Networks

Distributed systems communicate via **asynchronous packet networks** (e.g., Ethernet, TCP/IP). The network is fundamentally unreliable.

WHAT CAN GO WRONG WITH A NETWORK REQUEST
Node A —request→ Node B 1. Request LOST (dropped by network) 2. Request QUEUED (sitting in a switch buffer, delivered late) 3. Node B CRASHED (received request but can't process) 4. Node B PAUSED (GC pause, busy processing other requests) 5. Response LOST (B processed it but reply was dropped) 6. Response QUEUED (reply delayed in the network) A → B ????? A sent request. No response. What happened? A CANNOT DISTINGUISH between these cases. The only information A has is: "I haven't received a response yet."

8.2.1 Timeouts and Unbounded Delays

THE TIMEOUT DILEMMA

How long should A wait before declaring B dead?

TIMEOUT TOO SHORT:

- Declares nodes dead that are merely slow
- Unnecessary failovers
- Extra load (transferred actions + retries) can cause CASCADING FAILURES (node was slow due to overload, now we've moved its work to OTHER nodes which also overload!)

TIMEOUT TOO LONG:

- Long wait before a dead node is declared dead
- Users see errors or stale data while waiting

NO "CORRECT" TIMEOUT VALUE:

Network delays are UNBOUNDED – a packet might take 1ms or 1 minute. You can only choose a trade-off.

Practical approach:

- Measure the distribution of network round-trip times
- Consider the application's requirements
- Use an adaptive timeout (like TCP's retransmission timeout which adjusts based on observed RTT variance)
- Phi Accrual failure detector (Akka, Cassandra)

8.2.2 Network Congestion and Queueing

WHERE DO NETWORK DELAYS COME FROM?

1. NETWORK SWITCH QUEUE

Switch buffer full → packets DROPPED (need retransmission)

2. OS RECEIVE BUFFER

Destination CPU is busy → packets queue in OS buffer

3. VM MONITOR QUEUE

In virtualized environments, the hypervisor pauses VMs to let other VMs run → incoming packets buffer

4. TCP FLOW CONTROL

Sender throttled if receiver can't keep up
(TCP window, congestion avoidance)

5. TCP RETRANSMISSION

Lost packet → retransmit after timeout (seconds!)

In public clouds (AWS, GCP), you share network with other tenants → noisy neighbors cause variable delays.

On a single datacenter Ethernet: <1ms typical
Congested network / cloud: can exceed 100ms or seconds

8.3 Unreliable Clocks

Each machine has its own clock (quartz oscillator). Clocks drift: faster or slower than real time.

8.3.1 Time-of-Day Clocks vs. Monotonic Clocks

TWO KINDS OF CLOCKS

TIME-OF-DAY CLOCK:

- Returns wall-clock time (e.g., 2024-03-15 14:23:05.123)
- Synchronized with NTP (Network Time Protocol)
- Can JUMP forwards or backwards (NTP correction!)
- Coarse resolution (often ~millisecond)
- `System.currentTimeMillis()` (Java), `time.time()` (Python)
- BAD for measuring elapsed time (may jump during measurement)

MONOTONIC CLOCK:

- Guaranteed to always move forward
- Only the DIFFERENCE between two readings is meaningful
- Not synchronized across machines
- Fine resolution (nanosecond on modern systems)
- `System.nanoTime()` (Java), `clock_gettime(CLOCK_MONOTONIC)`
- GOOD for measuring elapsed time (timeouts, benchmarks)
- The absolute value is meaningless (arbitrary epoch)

RULE: Use monotonic clocks for measuring durations.
Use time-of-day clocks only when you need wall time.

8.3.2 Clock Synchronization Problems (NTP)

NTP CLOCK SYNCHRONIZATION ISSUES

- Quartz clock DRIFT: up to 200 ppm (parts per million)
= 6 seconds per month = 17 seconds per day in worst case
- NTP accuracy: ~35ms over the internet, <1ms on LAN
But can be much worse with network congestion
- NTP can JUMP the clock forward or backward
(step correction, not just rate adjustment)
- Leap seconds: occasionally a minute has 61 seconds
Has caused outages (e.g., Reddit, 2012)
- Virtualization: VM pause freezes the clock; when resumed,
the clock jumps forward suddenly
- Firewall may block NTP traffic → no synchronization

CONSEQUENCE: You cannot assume two machines' clocks are
in sync. Even a "small" difference of 100ms can cause bugs.

8.3.3 Timestamps for Ordering Events — A Dangerous Assumption

LAST WRITE WINS (LWW) – DANGER!

Node A writes $x=1$ at timestamp $t=42.004s$ (A's clock)

Node B writes $x=2$ at timestamp $t=42.001s$ (B's clock)

LWW picks the "latest" timestamp $\rightarrow x=1$ wins

But B's write ACTUALLY happened AFTER A's write!

B's clock was just 3ms behind A's clock.

Node A: ———[write $x=1$]—————→

Node B: —————[write $x=2$]—————→

▲ Actually later, but clock is behind

LWW silently drops B's write. Data loss with no error!

This is a REAL problem in Cassandra, DynamoDB, Riak.

The "winning" write is determined by clock skew, not causality.

8.3.4 Logical Clocks (Lamport Timestamps)

LAMPORT TIMESTAMPS

Instead of physical time, use a LOGICAL COUNTER:

Each node maintains a counter. On each event:

1. Increment own counter
2. Attach counter to messages
3. On receiving a message: $\text{counter} = \max(\text{own}, \text{received}) + 1$

Node A: 1 → 2 —————→ 5 → 6

Node B: \ /
 1 → 3 → 4 → 5

Lamport timestamps give TOTAL ORDER of events.

If event A happened before event B, then

$\text{timestamp}(A) < \text{timestamp}(B)$.

But: If $\text{timestamp}(A) < \text{timestamp}(B)$, A did NOT necessarily happen before B! (They might be concurrent.)

For true causal ordering $\rightarrow$ need VECTOR CLOCKS.

8.4 Knowledge, Truth, and Lies

8.4.1 The Truth Is Defined by the Majority

TRUTH BY MAJORITY (QUORUM)

A node cannot trust its OWN assessment of its status.

Example: Node A has a GC pause for 15 seconds.

- Node A thinks: "I'm fine, just processing a request."
- All other nodes think: "Node A is dead (no heartbeat)."
- The other nodes are RIGHT. Node A's lease has expired.
If A continues acting as leader → SPLIT BRAIN.

SOLUTION: Decisions require votes from a QUORUM (majority).
A node that claims "I'm the leader" must have agreement from a majority of nodes. A single node can't unilaterally decide anything in a distributed system.

8.4.2 Process Pauses

PROCESS PAUSES

A process can be paused for an extended time:

- GC pause (Java, Go, .NET): can be 100ms to several SECONDS
 - "stop-the-world" pauses freeze the entire process
- VM migration (live migration): hypervisor pauses VM
- OS swapping (thrashing): process waiting for disk I/O
- Ctrl+Z (SIGSTOP): process suspended by operator
- Context switches: CPU taken away by scheduler

During a pause, the process has NO IDEA it was paused.
It resumes and thinks no time has passed.

This is why LEASE-BASED approaches are dangerous:

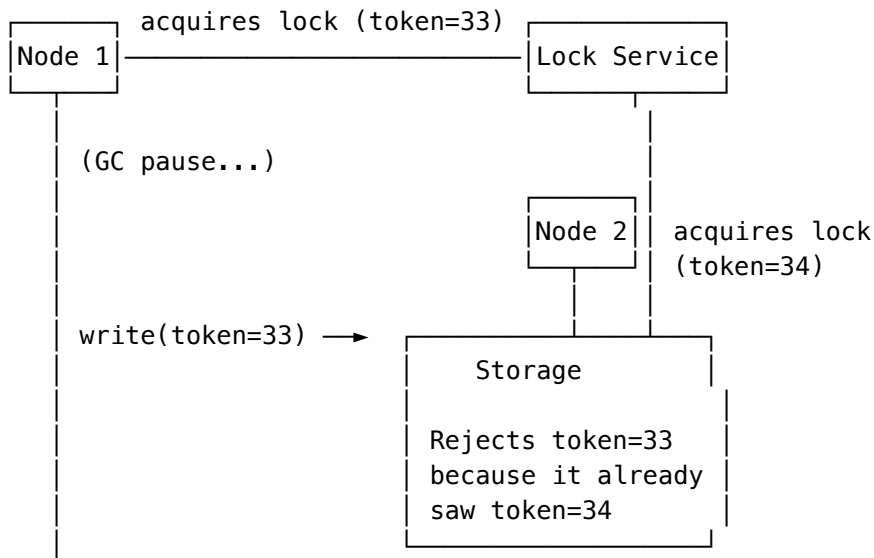
1. Node obtains lease (valid for 10 seconds)
2. Node starts processing (has 10s)
3. GC pause for 15 seconds
4. Node resumes – LEASE HAS EXPIRED but node doesn't know it. Another node became leader!
5. Node continues writing → DATA CORRUPTION

8.4.3 Fencing Tokens

FENCING TOKENS

Solution to the expired-lease problem:

Lock service issues a **MONOTONICALLY INCREASING** token with each lease grant.



Storage server rejects writes with OLD fencing tokens.
Requires storage to **CHECK** the token on every write.

8.4.4 Byzantine Faults

BYZANTINE FAULTS

All the problems above assume nodes are **HONEST** but may fail.

BYZANTINE FAULT: A node may **LIE** – send contradictory or fabricated messages to other nodes.

Relevant in:

- Aerospace (radiation may flip bits in CPU)
- Blockchain / cryptocurrency (nodes don't trust each other)
- Systems with multiple organizations (no mutual trust)

Most database systems **ASSUME** no Byzantine faults.

It's too expensive to tolerate. Instead:

- Use checksums to detect corrupted data
- Use TLS/authentication to verify message authenticity
- Sanitize inputs from users

8.5 System Models and Reality

SYSTEM MODELS	
TIMING ASSUMPTIONS:	
Synchronous:	Known upper bound on network delay and process pauses. UNREALISTIC.
Partially sync:	Usually behaves like synchronous, but occasionally exceeds bounds. REALISTIC model for most systems.
Asynchronous:	No timing assumptions at all. No timeouts possible (can't tell slow from crashed). Very restrictive.
NODE FAILURE MODELS:	
Crash-stop:	Node crashes and never comes back.
Crash-recovery:	Node crashes but may restart later with stable storage intact. MOST REALISTIC for real systems.
Byzantine:	Node may behave arbitrarily (including maliciously). Hardest to handle.
MOST PRACTICAL MODEL:	
Partially synchronous + crash-recovery	
This is what most distributed algorithms assume.	

8.6 Interview Questions

8.6.1 Q1: Why can't you simply use a timeout to detect node failure?

In an asynchronous network, there is no upper bound on message delivery time. A node might be alive but experiencing a long GC pause, network congestion, or VM migration. A short timeout causes false positives (declaring live nodes dead), which can trigger unnecessary failovers and even cascading failures (overloading other nodes with the "dead" node's work). A long timeout means genuine failures take too long to detect. There is no perfect timeout — only trade-offs between speed of detection and false positive rate.

8.6.2 Q2: Why are physical clocks unreliable for ordering events in distributed systems?

Each machine's clock drifts independently (up to 200 ppm). NTP synchronization provides at best ~35ms accuracy over the internet. NTP can jump clocks forward or backward. GC pauses and VM suspensions can freeze a process while its clock advances. Using physical timestamps for Last-Write-Wins (LWW) ordering means clock skew between nodes silently determines which writes survive — leading to data loss. Instead, use logical clocks (Lamport timestamps, vector clocks) for causal ordering.

8.6.3 Q3: What are fencing tokens and why are they needed?

Fencing tokens solve the problem of expired leases after process pauses. When a lock/lease is granted, the lock service issues a monotonically increasing token number. Every write to the storage system must include the token. The storage server rejects writes with a token lower than the highest token it has already seen. This prevents a node that held an old lease (and experienced a long pause) from making writes that conflict with the new leaseholder.

8.6.4 Q4: What is a Byzantine fault?

A Byzantine fault is when a node not only crashes but acts **maliciously** — sending contradictory, fabricated, or incorrect messages. Named after the Byzantine Generals Problem. Most database systems don't try to tolerate Byzantine faults (it requires $3f+1$ nodes to tolerate f Byzantine nodes). Byzantine fault tolerance is mainly relevant for blockchain/cryptocurrency systems, aerospace (bit-flip protection), and multi-party computation where nodes don't trust each other.

8.6.5 Q5: What is the most realistic system model for distributed databases?

Partially synchronous timing + crash-recovery failure model. Partially synchronous means the system usually meets timing bounds but occasionally exceeds them (realistic for networks and OS scheduling). Crash-recovery means nodes can crash and restart with their persistent storage intact (realistic for servers with SSDs/HDDs). This model is assumed by most practical distributed algorithms (Raft, Paxos, etc.).

Based on Chapter 8 of "Designing Data-Intensive Applications" by Martin Kleppmann

9 Chapter 9: Consistency and Consensus

9.1 Consistency Guarantees

Different databases offer different consistency guarantees, from weak (eventual consistency) to strong (linearizability).

CONSISTENCY SPECTRUM			
WEAK ← —————→ STRONG			
Eventual Consistency	Causal Consistency	Sequential Consistency	Linearizable (Strongest)
"If you stop writing, all replicas will eventually converge"	"Causally related events seen in correct order"	"All nodes see ops in the SAME order"	"Behaves as if there's only ONE copy of the data, and all ops are atomic"
✓ Best avail. ✗ Confusing for users	✓ Good perf. ✓ No coord. across DCs	✓ Predictable	✗ Worst perf. ✗ Worst avail. ✓ Easiest to reason about

9.2 Linearizability

The strongest single-object consistency model. Also called **atomic consistency**, **strong consistency**, or **external consistency**.

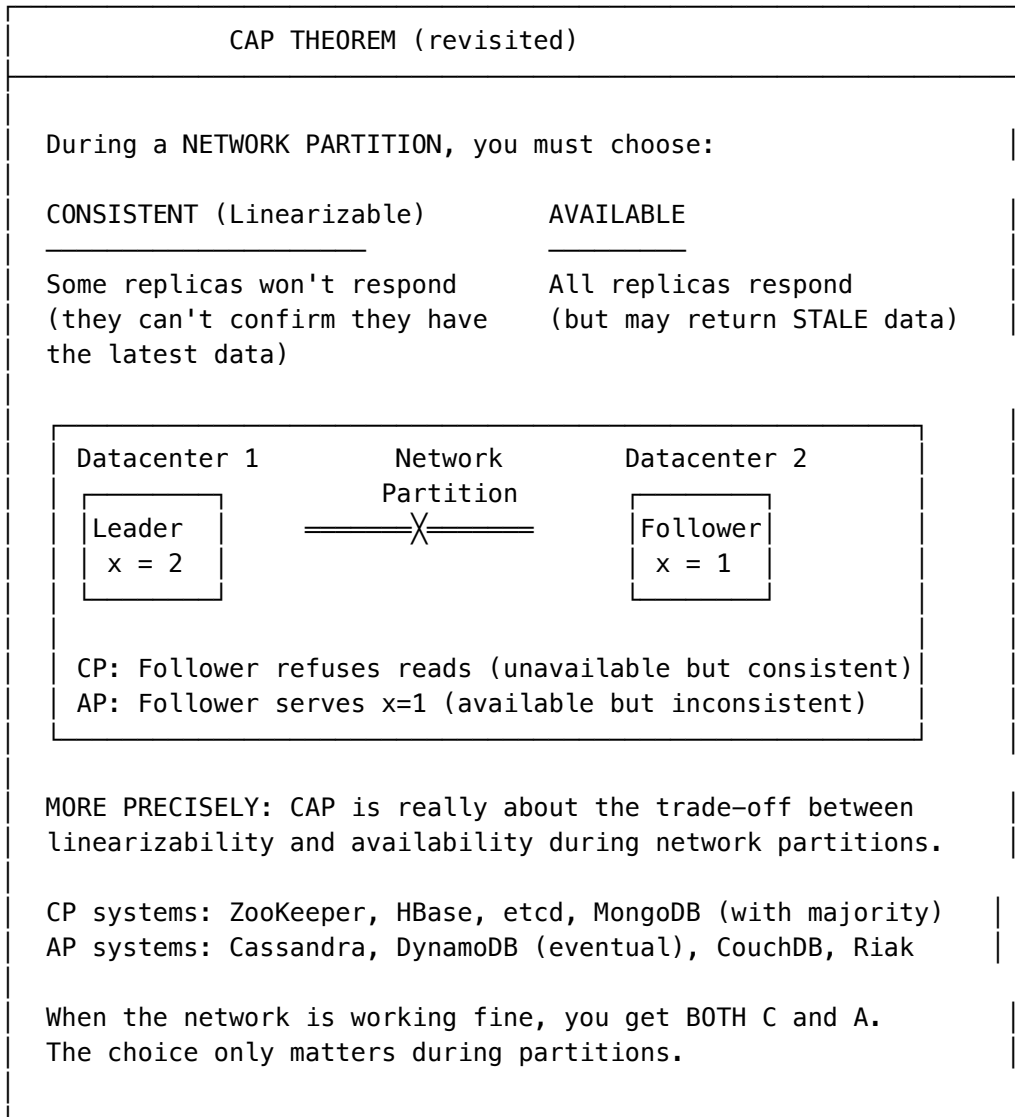
9.2.1 What Linearizability Means

LINEARIZABILITY DEFINITION
The system behaves AS IF there is only ONE copy of the data, and EVERY operation takes effect atomically at some point between its start and end.
Client A: —[write(x=1)]————→ Client B: —————[read(x)]————→ Client C: —————[read(x)]————→
Linearizable: Once ANY read returns x=1, ALL subsequent reads must also return x=1 (or a later value).
NOT linearizable: Client B reads x=1, then Client C reads x=0 (old value) → violates linearizability (went "back in time")
KEY PROPERTY: RECENCY GUARANTEE A read always returns the value of the most recently completed write (or a concurrent write).

9.2.2 When is Linearizability Required?

USE CASES REQUIRING LINEARIZABILITY:
1. LEADER ELECTION (e.g., ZooKeeper lock) All nodes must agree on who the leader is. If the lock is not linearizable, two nodes may both believe they are the leader → split brain!
2. UNIQUE CONSTRAINTS (username, file names) Two users registering the same username concurrently. Must ensure exactly one succeeds. Requires an atomic compare-and-set.
3. CROSS-CHANNEL COORDINATION File storage + message queue: "I uploaded image, now resize it." If message arrives before image is visible → error. Linearizability prevents this.
4. BANK ACCOUNT BALANCE After a debit, the balance must immediately reflect the change to prevent overdrafts.

9.2.3 The CAP Theorem



9.2.4 Linearizability and Performance

Even WITHOUT network partitions, many systems choose NOT to be linearizable – for PERFORMANCE reasons.
Multi-leader replication: Not linearizable (by design). Leaderless replication: Usually not linearizable (even with quorums – due to sloppy quorums, read repair).
The performance penalty of linearizability: Every read must contact the leader or do a quorum read. Cross-datacenter linearizable reads have high latency.
Many applications don't need linearizability and work correctly with weaker guarantees (causal consistency).

9.3 Ordering Guarantees

9.3.1 Causal Consistency

CAUSAL CONSISTENCY
CAUSALLY RELATED events must be seen in the correct order. CONCURRENT events (no causal relationship) can be in any order. Examples of causal dependencies: • Question before answer • Create before update • Insert before foreign key reference Causal consistency is WEAKER than linearizability: • Linearizable: total order (all events ordered) • Causal: partial order (only causally related events ordered) Causal consistency is the STRONGEST consistency model that doesn't sacrifice availability (unlike linearizability). Implementation: Track causal dependencies using VERSION VECTORS or LAMPORT TIMESTAMPS.

9.3.2 Total Order Broadcast

TOTAL ORDER BROADCAST

A protocol for exchanging messages between nodes with TWO guarantees:

1. RELIABLE DELIVERY: If a message is delivered to one node, it is delivered to ALL nodes (no message loss)
2. TOTALLY ORDERED: All nodes see messages in the SAME order

Node A sees: [msg1, msg2, msg3, msg4]

Node B sees: [msg1, msg2, msg3, msg4] ← Same order!

Node C sees: [msg1, msg2, msg3, msg4] ← Same order!

If you have total order broadcast, you can build:

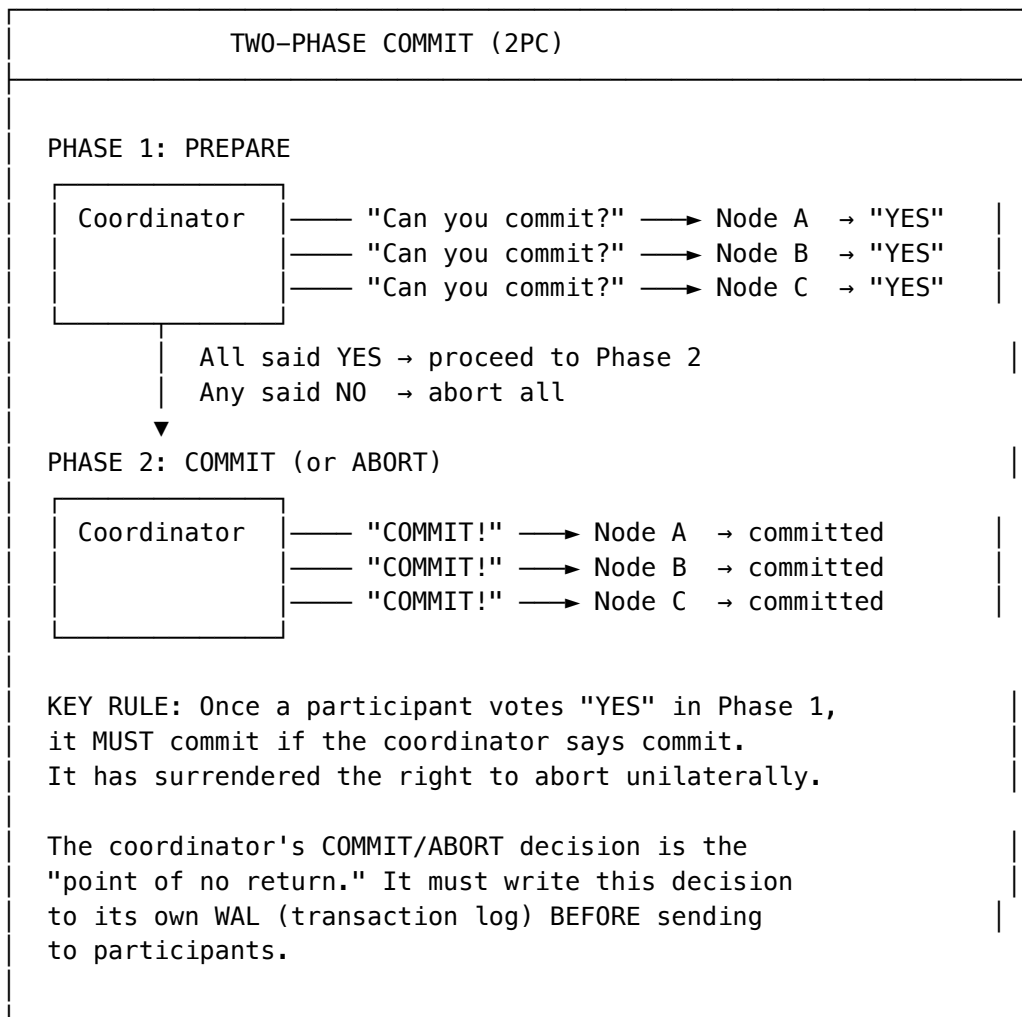
- Linearizable storage (use it as a write-ahead log)
- Database replication (each message = a write)
- Serializable transactions
- Lock services
- Unique ID generation

EQUIVALENT to consensus: if you can solve total order broadcast, you can solve consensus, and vice versa.

Implemented by: ZooKeeper (Zab), etcd (Raft), Kafka

9.4 Distributed Transactions and Consensus

9.4.1 Two-Phase Commit (2PC)



9.4.2 The Coordinator Failure Problem

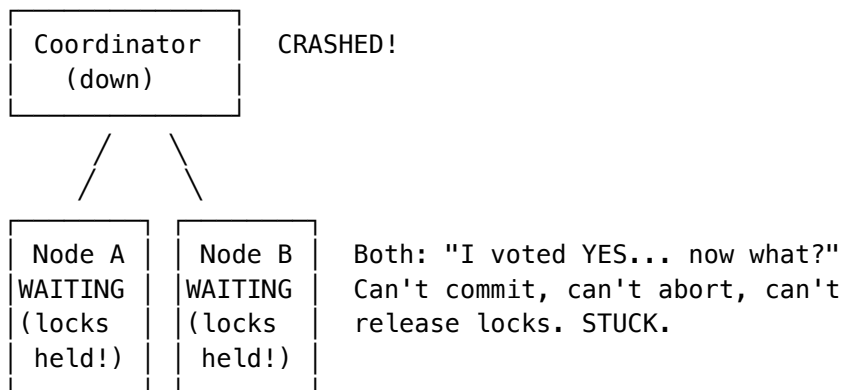
COORDINATOR FAILURE – IN DOUBT

What if the coordinator crashes AFTER Phase 1 but BEFORE sending Phase 2 messages?

Participants voted "YES" and are now WAITING.
They cannot commit (don't know if all voted yes).
They cannot abort (coordinator might have decided commit).
They are IN DOUBT / UNCERTAIN.

They MUST WAIT for the coordinator to recover.
This can take minutes, hours, or even days!

During this time, participants HOLD LOCKS on the affected rows → other transactions are BLOCKED.



This is a FUNDAMENTAL problem with 2PC – the coordinator is a single point of failure that can block the whole system.

9.5 Consensus Algorithms

Consensus means getting several nodes to **agree on something** (leader election, transaction commit, total order of events). The formal properties:

CONSENSUS PROPERTIES	
1. UNIFORM AGREEMENT:	All nodes decide the same value
2. INTEGRITY:	No node decides twice
3. VALIDITY:	The decided value was proposed by some node (not made up)
4. TERMINATION:	Every non-crashed node eventually decides (liveness/progress)
FLP IMPOSSIBILITY THEOREM (1985):	
In an asynchronous system where nodes can crash, it is IMPOSSIBLE to guarantee consensus will be reached.	
Practical algorithms (Paxos, Raft, Zab) work around FLP by using timeouts (assuming partial synchrony) – they may not terminate during network partitions, but they NEVER violate safety (agreement, integrity, validity).	

9.5.1 Raft Consensus Algorithm

RAFT CONSENSUS (simplified)	
Three roles: • LEADER: Handles all client requests, replicates log • FOLLOWER: Passive, replicates leader's log • CANDIDATE: Trying to become leader	
LEADER ELECTION: 1. Follower hasn't heard from leader (timeout) 2. Becomes CANDIDATE, increments TERM, votes for itself 3. Sends RequestVote to all other nodes 4. If majority votes YES → becomes LEADER 5. If another leader discovered → steps down 6. If election timeout → starts new election	
LOG REPLICATION: 1. Client sends write to LEADER 2. Leader appends to its log 3. Leader sends AppendEntries RPC to all followers 4. Once MAJORITY have replicated → entry is COMMITTED 5. Leader responds to client 6. Followers apply committed entries to state machine	
TERM: A logical clock period. Each term has at most one leader. If a node sees a higher term → steps down.	
Key invariant: If two logs contain an entry with the same index and term, all preceding entries are identical.	

9.5.2 Comparison of Consensus Approaches

Aspect	2PC	Paxos	Raft	Zab (ZooKeeper)
Type	Distributed transaction	Consensus	Consensus	Total order broadcast
Coordinator	Single (SPOF)	No fixed leader	Elected leader	Elected leader
Fault tolerance	Blocks if coordinator fails	Tolerates f of $2f+1$ failures	Tolerates f of $2f+1$	Tolerates f of $2f+1$
Safety	Always safe	Always safe	Always safe	Always safe
Liveness	Can block indefinitely	Eventually (partial sync)	Eventually (partial sync)	Eventually (partial sync)
Complexity	Simple	Very complex	Simpler than Paxos	Moderate
Used by	XA transactions	Google Chubby	etcd, CockroachDB	Apache ZooKeeper

9.5.3 What Consensus Gives You (via ZooKeeper)

ZOOKEEPER – CONSENSUS AS A SERVICE
Instead of implementing Paxos/Raft yourself, use ZooKeeper:
1. LEADER ELECTION Nodes compete for a ZooKeeper lock. Winner = leader. Others watch for leader failure.
2. DISTRIBUTED LOCKS Acquire lock via ZooKeeper node (ephemeral znode). Lock auto-released if holder crashes (session timeout).
3. SERVICE DISCOVERY Services register in ZooKeeper. Clients watch for changes → notified when services come/go.
4. CONFIGURATION MANAGEMENT Store config in ZooKeeper. All nodes watch → notified on change → reload.
5. PARTITION ASSIGNMENT Which node owns which partition (e.g., Kafka uses ZK to track partition→broker mapping).
Alternatives to ZooKeeper: etcd (Raft), Consul (Raft)

9.6 Interview Questions

9.6.1 Q1: What is linearizability and how is it different from serializability?

Linearizability is a recency guarantee on a single object: all operations appear to take effect atomically at some point between their invocation and response, and once a read returns a value, all subsequent reads return that value or a later one. **Serializability** is an isolation property of transactions: transactions appear to execute in some serial order. They are different dimensions — linearizability is about single-object recency; serializability is about multi-object transaction ordering. A system can be serializable but not linearizable (e.g., snapshot isolation), or linearizable but not serializable. “Strict serializability” provides both.

9.6.2 Q2: Explain the CAP theorem.

The CAP theorem states that during a **network partition**, a distributed system must choose between **Consistency** (linearizability) and **Availability**. A CP system returns errors or timeouts from partitioned replicas rather than serve stale data. An AP system responds from all replicas even if they have stale data. When the network is healthy, you can have both. CAP is often misunderstood — it’s specifically about linearizability during partitions, not a general “pick two of three” choice.

9.6.3 Q3: What is two-phase commit and what is its main weakness?

2PC coordinates a distributed transaction across multiple nodes. In Phase 1 (Prepare), the coordinator asks each participant “can you commit?” In Phase 2 (Commit), if all said yes, the coordinator tells everyone to commit; if any said no, abort. The **main weakness**: if the coordinator crashes between Phase 1 and Phase 2,

participants that voted YES are stuck in an “in doubt” state — they can’t commit or abort unilaterally and must hold locks until the coordinator recovers. This can block the system for an unbounded time.

9.6.4 Q4: Compare Paxos and Raft.

Both are consensus algorithms tolerating f failures among $2f+1$ nodes. **Paxos** (Lamport, 1998) is the foundational algorithm — mathematically elegant but notoriously difficult to understand and implement correctly. It separates leader election and log replication, making it flexible but complex. **Raft** (Ongaro & Ousterhout, 2014) was designed specifically for understandability — it bundles leader election, log replication, and safety into a single coherent protocol with strong leader semantics. Raft is more widely implemented in practice (etcd, CockroachDB).

9.6.5 Q5: What is total order broadcast and why is it equivalent to consensus?

Total order broadcast guarantees that all nodes deliver the same messages in the same order, reliably. It’s equivalent to consensus because: (1) if you have consensus, you can implement total order broadcast (use consensus to agree on the next message to deliver), and (2) if you have total order broadcast, you can implement consensus (broadcast a proposal, first one delivered wins). This equivalence means solutions to one problem directly solve the other.

9.6.6 Q6: Why can’t consensus be guaranteed in an asynchronous system?

The **FLP impossibility theorem** (Fischer, Lynch, Paterson, 1985) proves that in a purely asynchronous system where even one node can crash, no deterministic consensus algorithm can guarantee termination. This is because you can’t distinguish a crashed node from a slow one without timeouts. Practical algorithms like Raft and Paxos work around this by assuming partial synchrony — they use timeouts and may fail to make progress during network issues, but they **never** violate safety (they won’t make contradictory decisions).

Based on Chapter 9 of “Designing Data-Intensive Applications” by Martin Kleppmann

10 Chapter 10: Batch Processing

10.1 Three Types of Systems

THREE TYPES OF DATA SYSTEMS

1. ONLINE SERVICES (OLTP):

Request → Response. Low latency critical.

Measure: response time, availability.

Examples: web servers, APIs, databases.

2. BATCH PROCESSING:

Process a LARGE AMOUNT of data at once.

Input: bounded dataset. Output: derived dataset.

Measure: throughput (records/sec or time to complete).

Examples: MapReduce, Spark, data warehouse ETL.

Runs periodically (hourly, daily).

3. STREAM PROCESSING:

Process events as they arrive (near real-time).

Input: unbounded stream. Output: derived data.

Measure: latency (event-to-output delay).

Examples: Kafka Streams, Flink, Storm.

Runs continuously.

This chapter: BATCH PROCESSING.

10.2 The Unix Philosophy

Batch processing has deep roots in Unix pipes:

```
cat /var/log/nginx/access.log |  
  awk '{print $7}' |  
  sort |  
  uniq -c |  
  sort -rn |  
  head -5
```

This pipeline finds the 5 most-requested URLs from a web server log.

10.2.1 Unix Design Principles

UNIX PHILOSOPHY (Doug McIlroy, 1978)

1. Make each program do ONE THING well.
To do a new job → build a new program.
2. Expect OUTPUT of every program to become INPUT
to another, as yet unknown, program.
3. Design and build software to be TRIED early.
Don't hesitate to throw away clumsy parts and rebuild.
4. Use TOOLS to lighten a programming task,
even if you have to detour to build the tools.

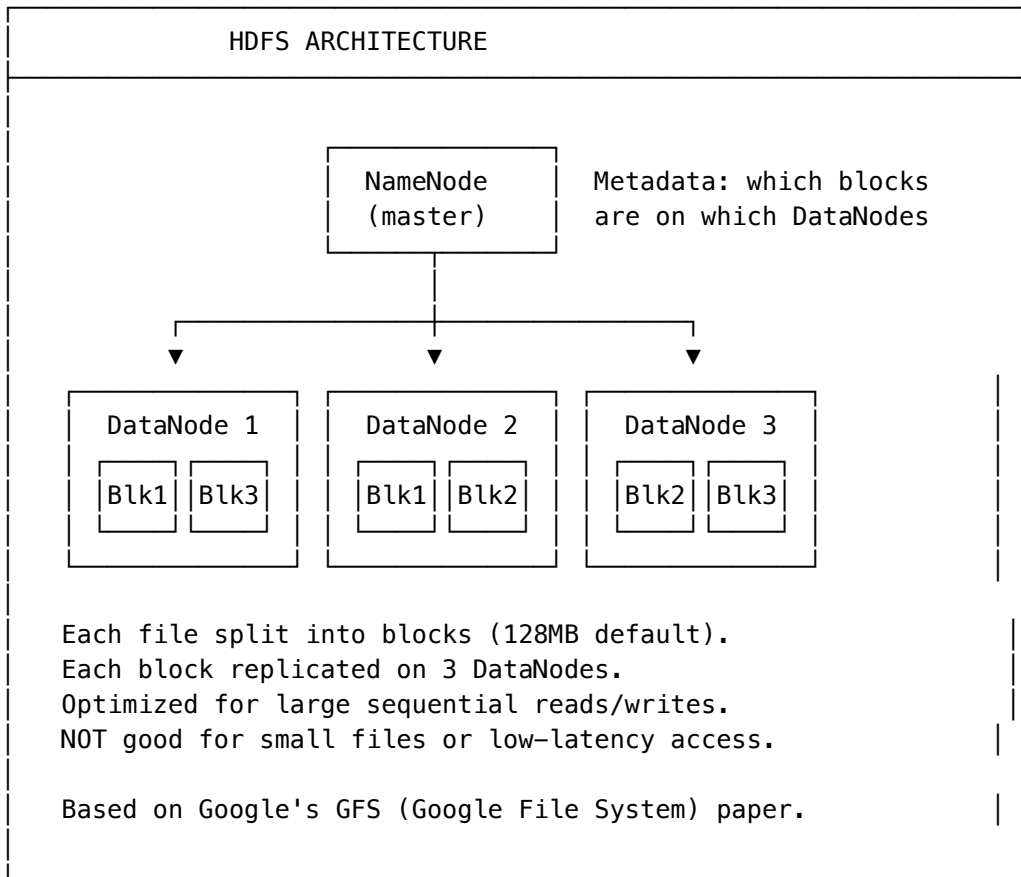
KEY IDEA: A uniform interface (text files on stdin/stdout)
allows programs to be composed freely.

MapReduce is the Unix pipe for distributed systems:

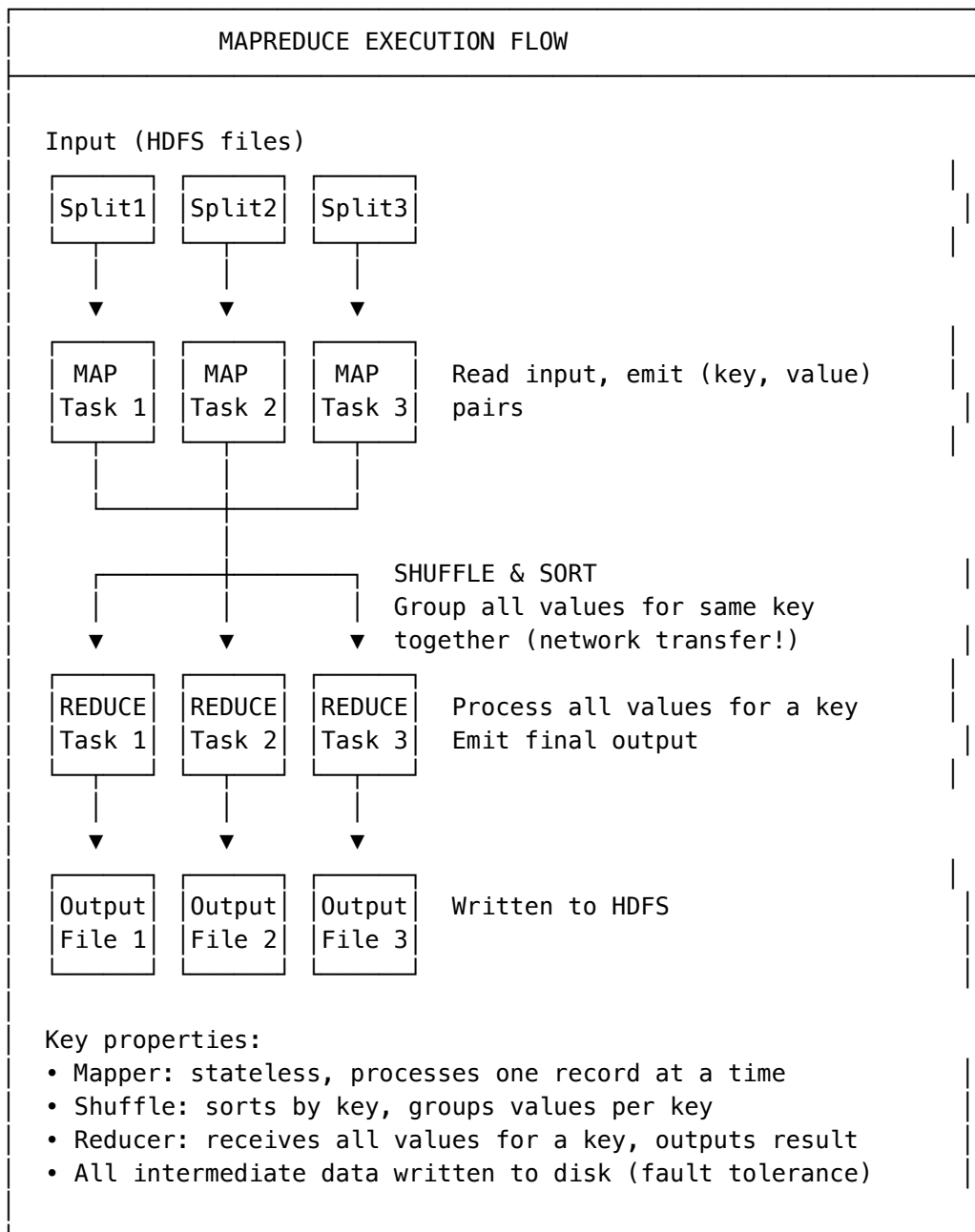
- Input: files on HDFS (not stdin)
- Output: files on HDFS (not stdout)
- Programs: map/reduce functions (not shell commands)

10.3 MapReduce and Distributed Filesystems

10.3.1 HDFS (Hadoop Distributed File System)



10.3.2 MapReduce Execution



10.3.3 Fault Tolerance in MapReduce

MapReduce achieves fault tolerance through:

1. DETERMINISTIC functions: If a task fails, just re-run it. Same input → same output. No side effects.
2. Input/output on HDFS: Data is replicated (3 copies). If a node dies, data is still available elsewhere.
3. Intermediate data on local disk: If a map task fails, the framework reassigns it to another node. If a reduce task fails, upstream map output is still available (on map node's local disk).
4. Speculative execution: If a task is slow, the framework launches a duplicate on another node. First to finish wins.

TRADE-OFF: This materialization to disk between stages is SLOW but makes the system very ROBUST.

10.4 MapReduce Joins

10.4.1 Sort-Merge Join (Reduce-Side Join)

SORT-MERGE JOIN

Join user activity logs with user profiles:

Activity log:

```
user_id=123, page=/foo
user_id=456, page=/bar
user_id=123, page=/baz
```

User profiles:

```
id=123, name=Bob
id=456, name=Amy
id=789, name=Eve
```

MAP PHASE:

Both datasets emit (user_id, record) pairs:

```
(123, {activity: /foo})
(123, {activity: /baz})
(123, {profile: name=Bob})
(456, {activity: /bar})
(456, {profile: name=Amy})
```

SHUFFLE: Groups by user_id

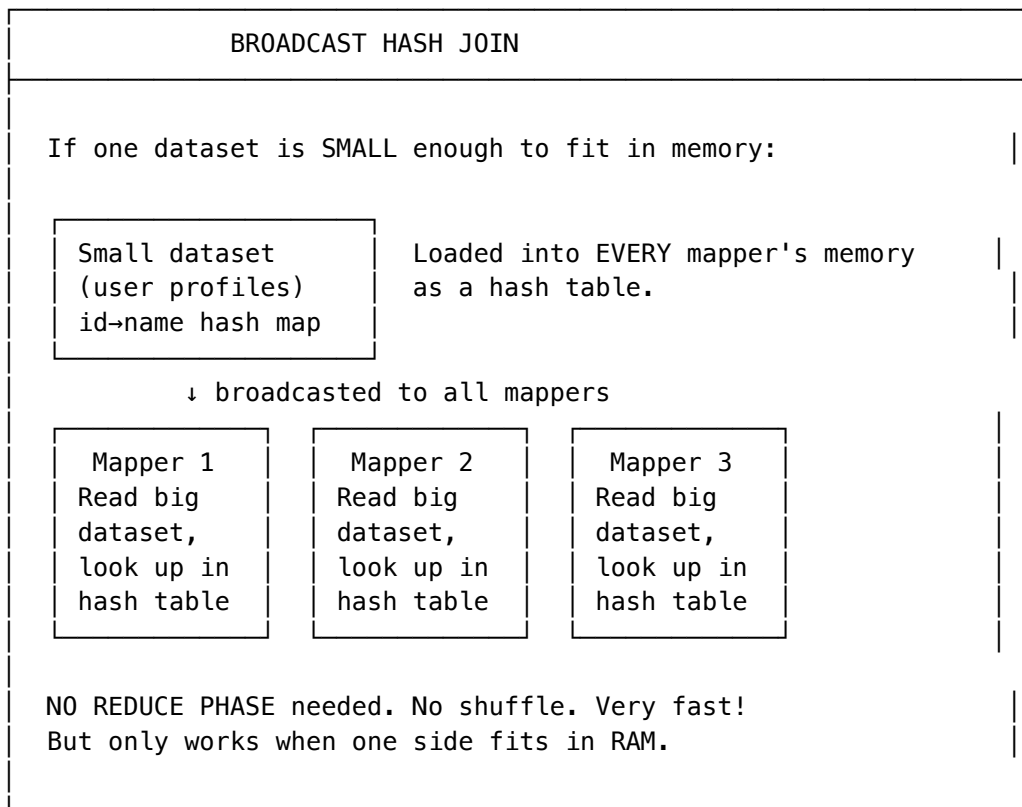
REDUCE PHASE (for user_id=123):

Receives: [{profile: name=Bob}, {activity: /foo},
 {activity: /baz}]

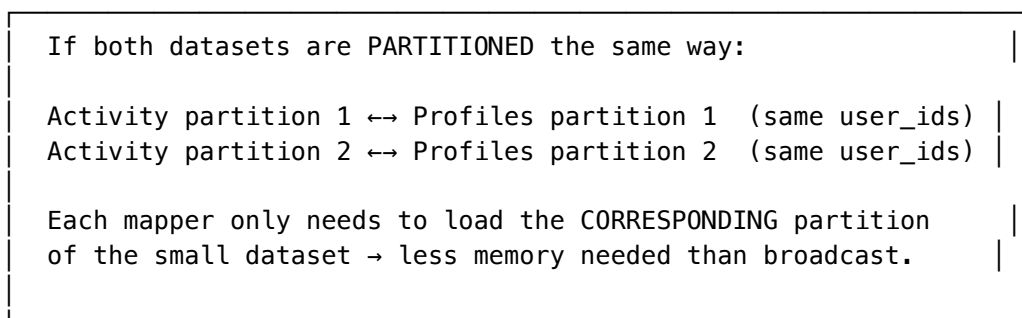
Outputs: (Bob, /foo) and (Bob, /baz)

Called "sort-merge" because the shuffle sorts by key,
and the reducer merges records from both datasets.

10.4.2 Broadcast Hash Join (Map-Side Join)



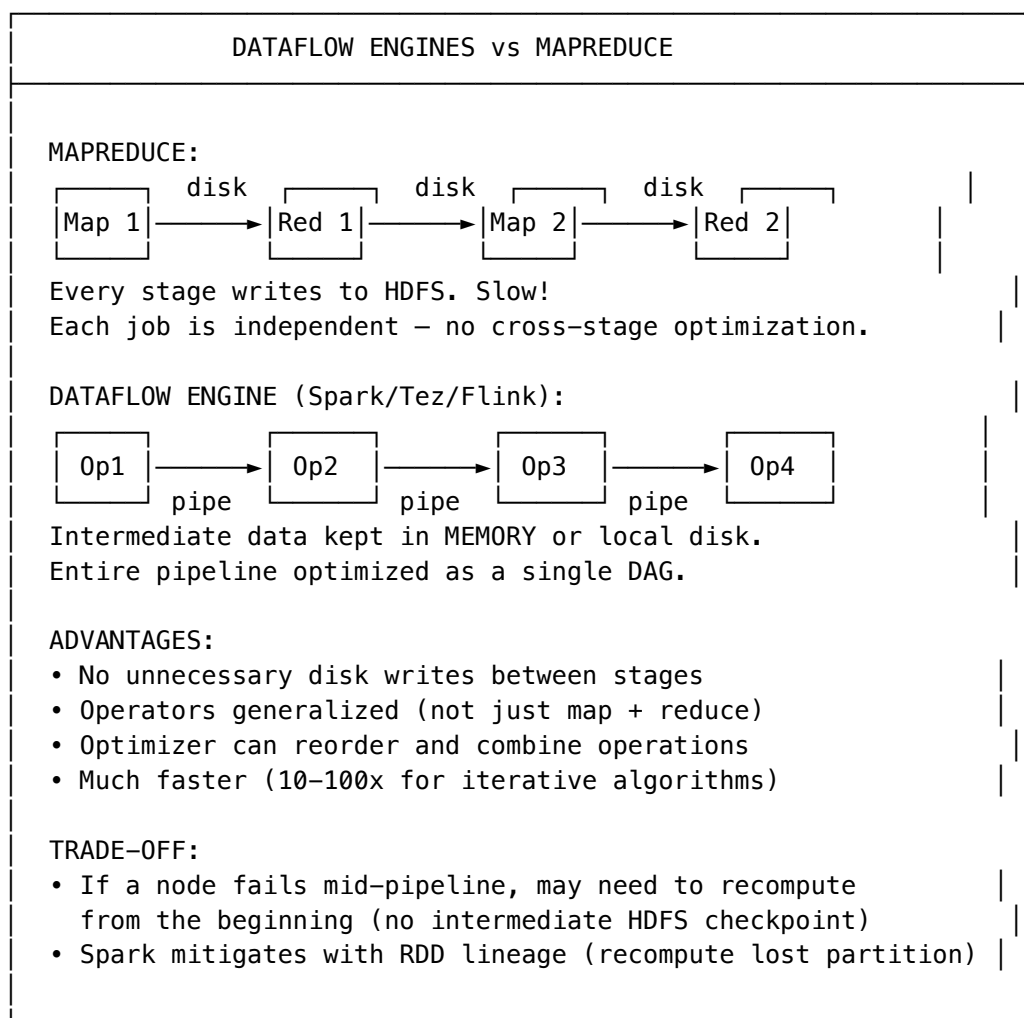
10.4.3 Partitioned Hash Join



10.5 Beyond MapReduce — Dataflow Engines

MapReduce has limitations: every stage writes to disk, chaining jobs is awkward, and it doesn't optimize across stages.

10.5.1 Spark, Tez, and Flink



10.5.2 Comparison

Feature	MapReduce	Spark	Flink
Model	Map → Shuffle → Reduce	DAG of operators	DAG of operators
Intermediate data	Always HDFS	In-memory (spill to disk)	In-memory / pipelined
Fault tolerance	Re-run failed task	Recompute from RDD lineage	Checkpoint + replay
Iteration	Chain multiple jobs	In-memory iterative	Native iteration
Latency	High (disk I/O)	Low (in-memory)	Very low
Streaming	No	Micro-batching	True streaming

10.6 Interview Questions

10.6.1 Q1: How does MapReduce achieve fault tolerance?

MapReduce functions (map, reduce) are **deterministic** — same input always produces same output. All input/output goes through HDFS (replicated). If a map task fails, the framework simply reruns it on another node using the same input split from HDFS. If a reduce task fails, it's rerun — map outputs are still available on the map nodes' local disks. Slow tasks are handled with **speculative execution**: a duplicate task is run on another node, and whichever finishes first wins. The cost is materializing all intermediate data to disk, which is slow but robust.

10.6.2 Q2: Explain the three types of joins in MapReduce.

1. **Sort-merge join (reduce-side):** Both datasets are mapped to emit (join_key, record) pairs. The shuffle sorts and groups by key. The reducer receives all records for each key from both datasets and performs the join. Works for any size datasets but requires a full shuffle.
2. **Broadcast hash join (map-side):** The small dataset is loaded into every mapper's memory as a hash table. Each mapper reads the large dataset and looks up the join key in the hash table. No shuffle needed — very fast, but the small dataset must fit in RAM.
3. **Partitioned hash join:** Both datasets are pre-partitioned by the join key. Each mapper loads only the corresponding partition of the small dataset. Requires less memory than broadcast.

10.6.3 Q3: Why is Spark faster than MapReduce?

Spark keeps intermediate data **in memory** instead of writing to HDFS between stages. It models the entire computation as a **DAG of operators** (not just map and reduce), allowing the optimizer to pipeline operations and avoid unnecessary materializations. For iterative algorithms (machine learning, graph processing) that reuse the same dataset, Spark can keep it cached in memory across iterations — MapReduce would re-read from disk each time. Spark is typically 10-100x faster for iterative workloads.

10.6.4 Q4: What is the Unix philosophy and how does MapReduce relate to it?

The Unix philosophy (Doug McIlroy): make each program do one thing well, design programs to work together, use a uniform interface (text streams). Unix pipes compose small programs into powerful pipelines. MapReduce is the distributed equivalent: map/reduce functions are composable processing steps, HDFS files are the “pipes” between stages, and the framework handles distribution, fault tolerance, and scheduling. The key difference: MapReduce operates on massive datasets across clusters, while Unix pipes run on a single machine.

10.6.5 Q5: What are the limitations of MapReduce that led to dataflow engines like Spark?

- (1) **Disk materialization:** Every intermediate result is written to HDFS — huge I/O overhead.
- (2) **Rigid two-phase model:** Only map and reduce — complex workflows require chaining multiple MapReduce jobs with intermediate HDFS files.
- (3) **No pipeline optimization:** Each job is independent; the framework can't optimize across job boundaries.
- (4) **No iteration support:** Iterative algorithms (ML training) must re-read data from disk each iteration.
- (5) **High latency:** Startup overhead per job makes small, interactive queries impractical.

Based on Chapter 10 of “Designing Data-Intensive Applications” by Martin Kleppmann

11 Chapter 11: Stream Processing

11.1 Transmitting Event Streams

A **stream** is data that is incrementally made available over time — an unbounded dataset.

EVENT STREAM CONCEPTS

EVENT: An immutable, timestamped record of something that happened. Small, self-contained.

Examples:

- User clicked button (user_id, page, timestamp)
- Temperature reading (sensor_id, temp, timestamp)
- Order placed (order_id, items, total, timestamp)

PRODUCER (publisher/sender) → TOPIC/STREAM → CONSUMER (subscriber)

TOPIC: Named group of related events (e.g., "user-clicks")

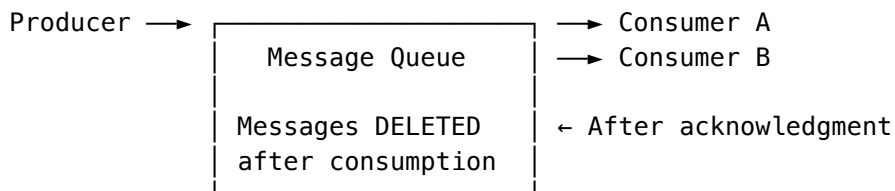
Batch processing: input is bounded (finite dataset)

Stream processing: input is UNBOUNDED (events keep arriving)

11.2 Message Brokers

11.2.1 Traditional Message Brokers vs Log-Based Brokers

TRADITIONAL MESSAGE BROKER (RabbitMQ, ActiveMQ, SQS)

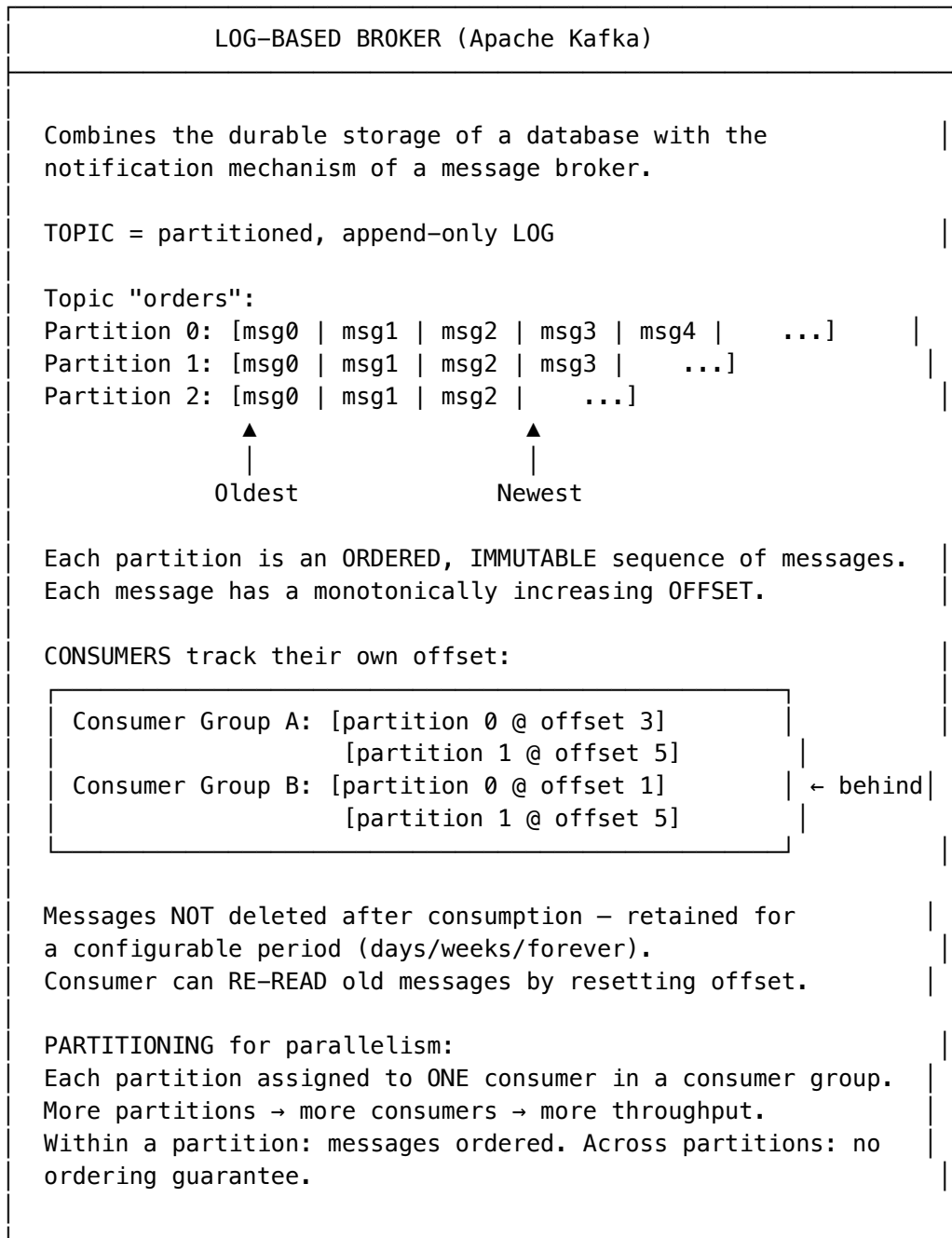


- Messages delivered to ONE consumer per queue (or fan-out with exchanges/topics)
- Message DELETED after acknowledgment
- Order NOT guaranteed across consumers
- Consumers can be added/removed dynamically
- If consumer is slow → messages queue up (bounded buffer)

DELIVERY GUARANTEES:

At-most-once: Message may be lost, never redelivered
At-least-once: Message never lost, may be redelivered
Exactly-once: Message processed exactly once
(hardest to achieve – see below)

11.3 Log-Based Message Brokers

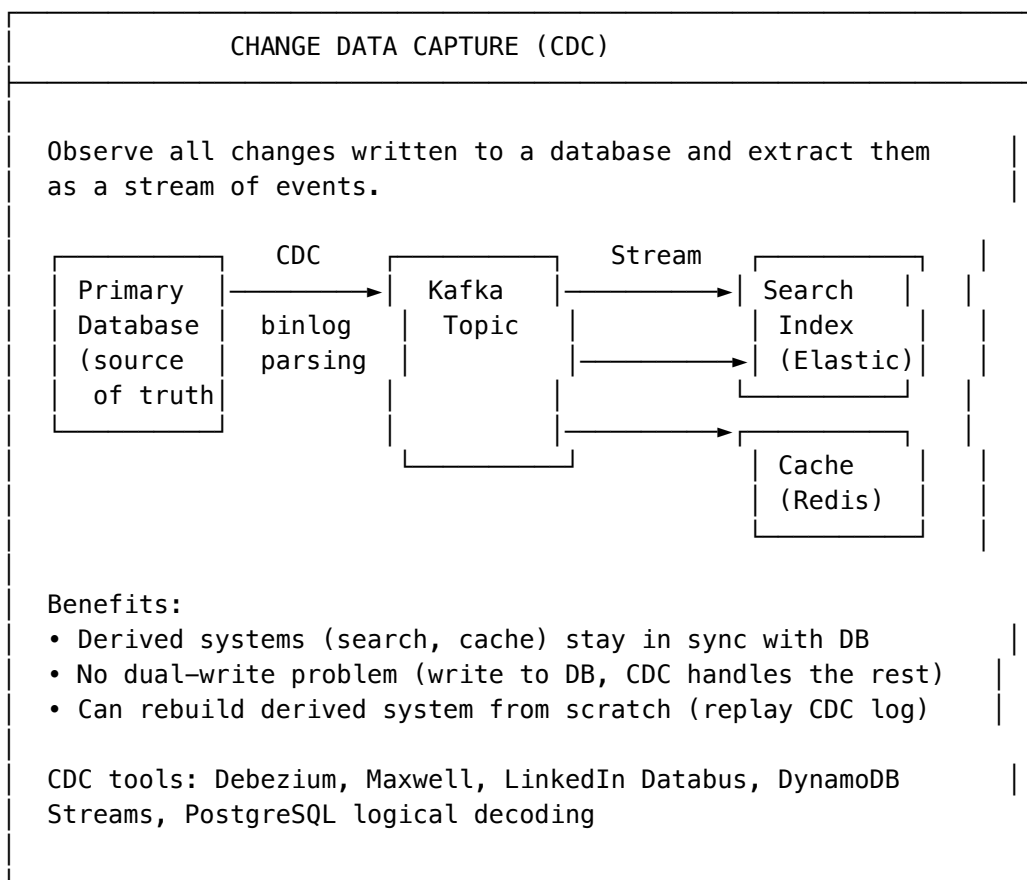


11.3.1 Kafka vs Traditional Brokers

Feature	Traditional (RabbitMQ)	Log-Based (Kafka)
Message retention	Deleted after ack	Retained for configured period
Replay	No (once consumed, gone)	Yes (reset offset)
Ordering	Per-consumer, not global	Per-partition guaranteed
Consumer parallelism	Any number of consumers	Max = number of partitions
Throughput	Lower (per-message ack)	Higher (sequential disk I/O)
Use case	Task queues, async processing	Event sourcing, CDC, analytics

11.4 Databases and Streams

11.4.1 Change Data Capture (CDC)



11.4.2 Event Sourcing

EVENT SOURCING

Instead of storing CURRENT STATE, store the SEQUENCE OF EVENTS that led to the current state.

Traditional (state-based):

Account balance: \$50
(just the latest state)

Event Sourcing:

Event 1: Deposit \$100
Event 2: Withdraw \$30
Event 3: Withdraw \$20
→ Derived: bal = \$50

Key principles:

- Events are IMMUTABLE (append-only log, never modified)
- Current state is DERIVED by replaying events
- Full AUDIT TRAIL (who did what, when)
- Can reconstruct state at ANY point in time
- Can create NEW derived views from old events

Difference from CDC:

- CDC: events describe low-level DB changes (row inserted)
- Event sourcing: events describe DOMAIN-LEVEL actions ("customer placed order", not "row inserted in orders table")

Used by: Event Store, Axon Framework, accounting systems

11.5 Processing Streams

11.5.1 Stream-Table Duality

STREAM-TABLE DUALITY

TABLE → STREAM:

Capture every change to a table → stream of changes (CDC).

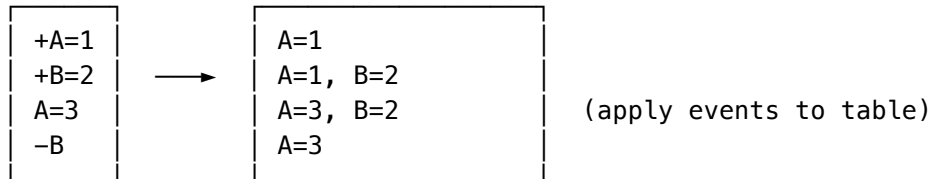
STREAM → TABLE:

Apply every event in a stream → materialized table.

(Replay the changelog to build current state.)

Stream

Table



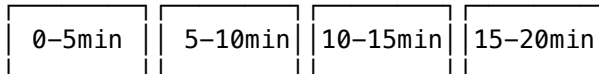
This duality is fundamental:

- A table is a materialized view of a stream
- A stream is the derivative (changelog) of a table
- Kafka's KSQL and Kafka Streams build on this idea

11.5.2 Window Types

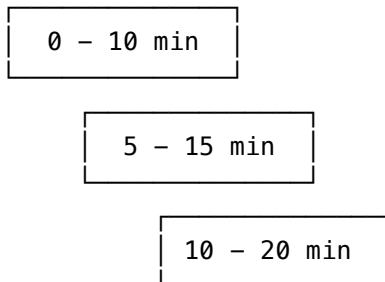
WINDOWING STRATEGIES

TUMBLING WINDOW (fixed, non-overlapping):



Each event belongs to exactly one window.

HOPPING WINDOW (fixed, overlapping):



Size=10min, hop=5min. Events can be in multiple windows.

SLIDING WINDOW:

Contains all events within a time INTERVAL of each other.

No fixed boundaries – slides continuously.

"All events within 5 minutes of each other."

SESSION WINDOW:

Groups events by user activity. Window ends after a period of INACTIVITY (e.g., 30 min gap → new session).

Variable length, depends on user behavior.

11.6 Stream Joins

THREE TYPES OF STREAM JOINS

1. STREAM-STREAM JOIN (Window Join):

Both inputs are streams. Match events within a time window.

Stream A (clicks): —[click user=1, url=/foo]→

Stream B (searches): —[search user=1, q="bar"]→

Join: "Find searches followed by a click within 1 hour"

Must buffer events from both streams (within window).

2. STREAM-TABLE JOIN (Enrichment):

One input is a stream, the other is a database table.

Stream (orders): —[order: user_id=123, product=X]→

Table (users): {123: {name: "Bob", tier: "gold"}}

Join: Enrich each order with user info.

Processor keeps local copy of table (updated via CDC).

3. TABLE-TABLE JOIN (Materialized View Maintenance):

Both inputs are CDC streams (changelogs of tables).

Stream A (tweets CDC): —[user=1 posted tweet]→

Stream B (follows CDC): —[user=2 follows user=1]→

Join: Maintain a timeline view (who sees which tweets).

Like a continuously-maintained materialized view.

11.7 Fault Tolerance in Streaming

FAULT TOLERANCE APPROACHES

1. **MICROBATCHING** (Spark Streaming):
Break stream into small batches (e.g., 1 second).
Process each batch as a mini batch job.
If batch fails → re-run the entire batch.
Latency = batch interval (at least 1 second).
2. **CHECKPOINTING** (Flink):
Periodically save processor state to durable storage.
On failure: restore from last checkpoint, replay events since checkpoint from the message broker.
Lower latency than microbatching.
3. **IDEMPOTENT WRITES**:
Make outputs idempotent: writing the same result twice has the same effect as writing once.
Use (partition, offset) as a deterministic key.
If failure causes replay → same result written again → no duplicate effect.
4. **TRANSACTIONAL OUTPUT** (Exactly-Once):
Atomically commit processing result AND offset advance.
Kafka transactions: read → process → write output + commit offset as a single atomic operation.

EXACTLY-ONCE is really "effectively-once":
Events may be PROCESSED more than once (on retry),
but the EFFECT on the output is as if processed once.

11.8 Interview Questions

11.8.1 Q1: What is the difference between a traditional message broker and a log-based message broker?

Traditional brokers (RabbitMQ, SQS) delete messages after consumer acknowledgment, don't support replay, and deliver messages to individual consumers. **Log-based brokers** (Kafka) retain messages in an append-only log for a configurable period, support replay by resetting the consumer's offset, guarantee ordering within partitions, and allow multiple consumer groups to independently read the same topic. Log-based brokers are better for event sourcing, CDC, and analytics pipelines; traditional brokers are better for task queues where messages should be processed exactly once and discarded.

11.8.2 Q2: What is Change Data Capture (CDC) and why is it useful?

CDC captures all changes (inserts, updates, deletes) made to a database and publishes them as a stream of events. It solves the **dual-write problem**: instead of writing to both a database and a search index (which can fail inconsistently), you write only to the database and let CDC propagate changes to derived systems (search, cache, data warehouse). Benefits: derived systems stay in sync, you can rebuild them from scratch by replaying the CDC log, and there's no risk of inconsistency from partial failures in dual writes.

11.8.3 Q3: Explain the stream-table duality.

A **table** can be turned into a stream by capturing every change (CDC → changelog stream). A **stream** can be turned into a table by replaying all events and building up current state (materialized view). They are two sides of the same coin: a table is the accumulated state at a point in time; a stream is the derivative (sequence of changes) of that state. This duality underpins systems like Kafka Streams and KSQL, where you can switch between stream and table representations.

11.8.4 Q4: How does exactly-once processing work in stream processing?

True “exactly-once” means events may be processed more than once (after failure and replay) but the **effect** is as if processed once. Approaches: (1) **Idempotent writes**: use deterministic keys so duplicate writes overwrite with the same value. (2) **Transactional output**: atomically commit the output and the consumer offset — if the transaction fails, both are rolled back, and the event is reprocessed. (3) **Flink checkpointing**: periodically snapshot operator state; on failure, restore and replay from the checkpoint. (4) **Microbatching** (Spark): process small batches atomically.

11.8.5 Q5: What is event sourcing and how does it differ from traditional CRUD?

In traditional CRUD, you store the **current state** and overwrite it on updates. In event sourcing, you store an **immutable log of domain events** (“order placed”, “item shipped”) and derive current state by replaying events. Benefits: complete audit trail, ability to reconstruct state at any point in time, ability to create new derived views from historical events, and natural fit for event-driven architectures. The main challenge is that the event log grows indefinitely — you need compaction or snapshotting for practical query performance.

11.8.6 Q6: Compare the four windowing strategies in stream processing.

Window	Boundaries	Overlap	Use Case
Tumbling	Fixed, aligned	None	Hourly aggregates
Hopping	Fixed, overlapping	Yes	Moving averages
Sliding	Event-relative	Yes	“Events within 5 min of each other”
Session	Activity-based, variable length	None	User session analytics

Based on Chapter 11 of “Designing Data-Intensive Applications” by Martin Kleppmann

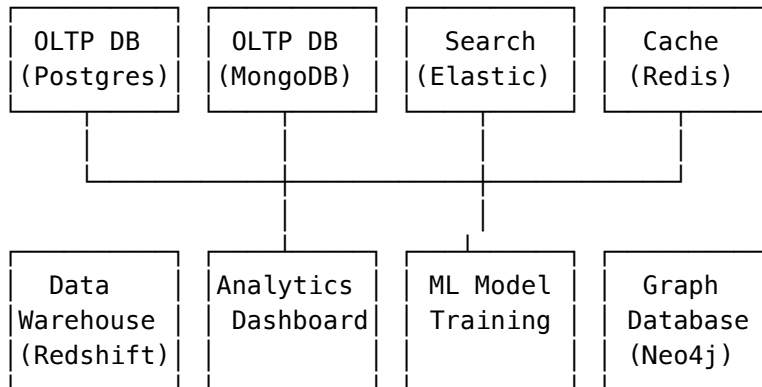
12 Chapter 12: The Future of Data Systems

12.1 Data Integration

The biggest challenge in modern data systems is not building any single tool, but **integrating data across different systems**.

THE DATA INTEGRATION PROBLEM

A typical organization has data in:



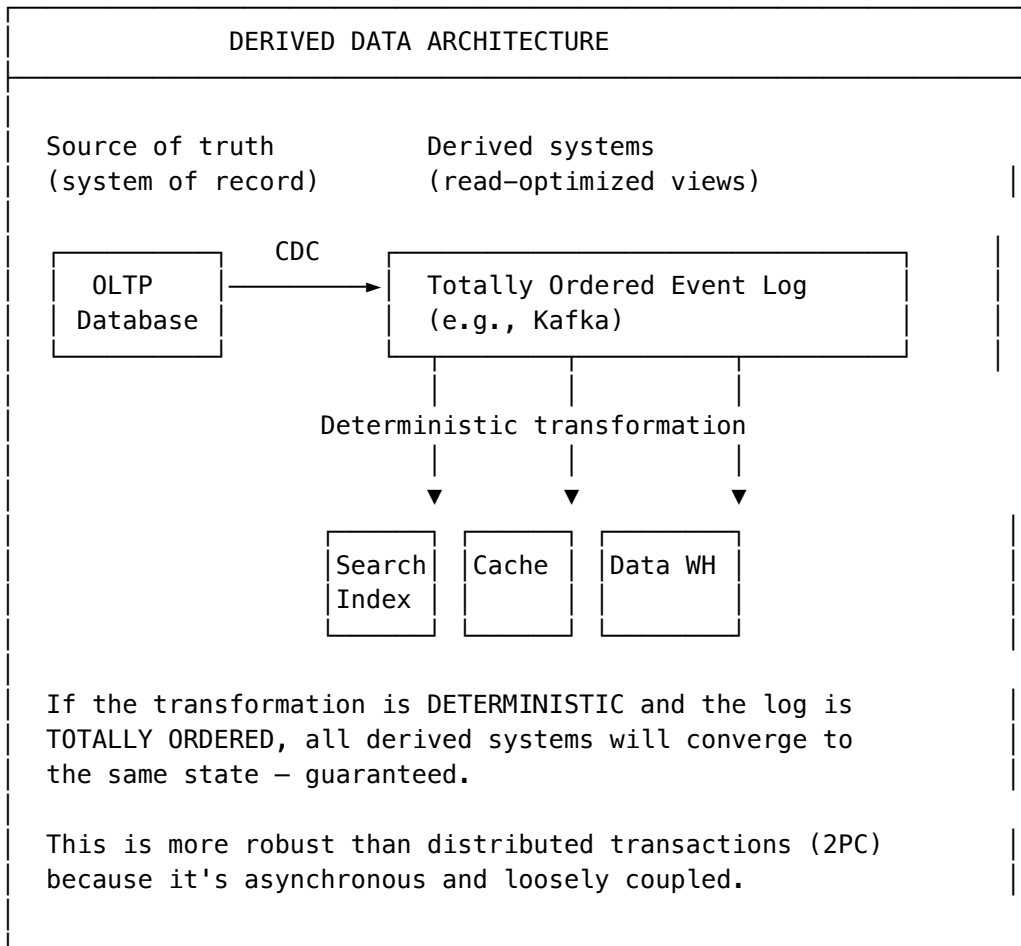
PROBLEM: How do you keep all these systems consistent?

DUAL WRITES (write to each system directly):

- x Race conditions → systems go out of sync
- x Partial failure → some updated, some not

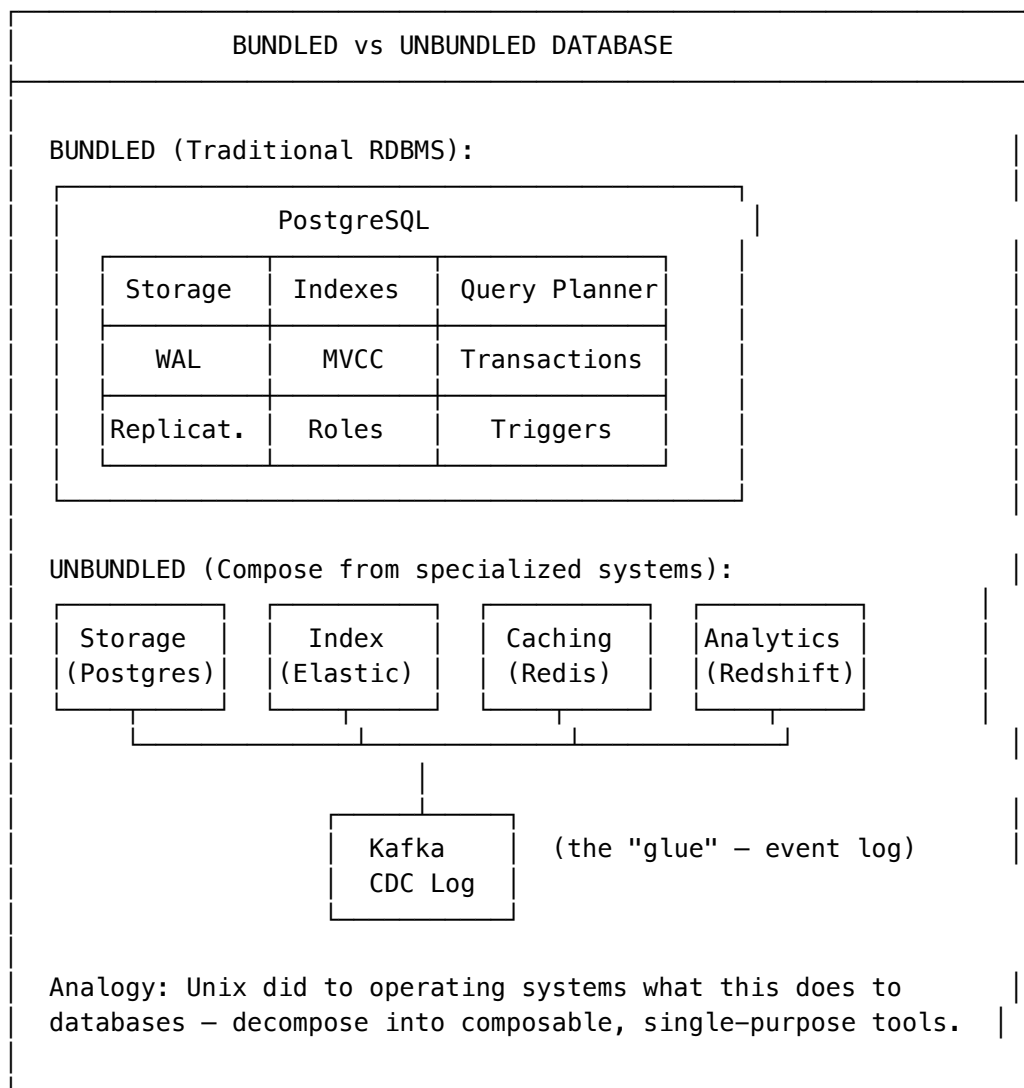
SOLUTION: Use a single SOURCE OF TRUTH (e.g., OLTP database) and derive all other representations via CDC or event streams.

12.1.1 Total Ordering and Deterministic Transforms



12.2 Unbundling Databases

A database provides many features: storage, indexing, query processing, transactions, replication, access control. The “unbundling” idea: compose these from **separate, specialized tools**.



12.2.1 The Advantages of Unbundling

Aspect	Bundled (RDBMS)	Unbundled
Flexibility	One-size-fits-all	Best tool for each job
Evolution	Upgrade entire DB	Replace/upgrade individual components
Scaling	Scale the entire system	Scale each component independently
Specialization	Good at many things	Excellent at one thing
Complexity	Simple to operate	More moving parts
Consistency	Built-in transactions	Eventual consistency (async)

12.3 Designing for Correctness

12.3.1 The Problem with Exactly-Once

END-TO-END CORRECTNESS

Even if the stream processor guarantees exactly-once, the END-TO-END system may not be correct:

User → Web App → Message Broker → Processor → Database

What if the user's BROWSER retries a request?

The message broker might see it as a new event!

Exactly-once at the broker level ≠ exactly-once end-to-end.

SOLUTIONS:

1. **IDEMPOTENCY KEYS:** Client generates a unique request ID. Server deduplicates by checking if ID was already processed.
2. **OPERATION IDENTIFIERS:** Include a unique operation ID that flows through the entire pipeline end-to-end. Each stage deduplicates.
3. **DETERMINISTIC DERIVATION:** If derived data is a deterministic function of input, replaying the same input always produces the same output.

12.3.2 Enforcing Constraints

ENFORCING UNIQUENESS IN ASYNC SYSTEMS

Unique username constraint in an event-driven system:

APPROACH 1: Log-based (single partition per username)

All requests for the same username → same Kafka partition

→ processed sequentially by a single consumer

→ first request wins, later requests rejected

APPROACH 2: Consensus (e.g., via ZooKeeper/etcd)

Use a linearizable compare-and-set operation

More expensive but immediately consistent

APPROACH 3: Compensating transaction

Accept all requests, detect conflicts later, and

send an apology email if a duplicate was accepted.

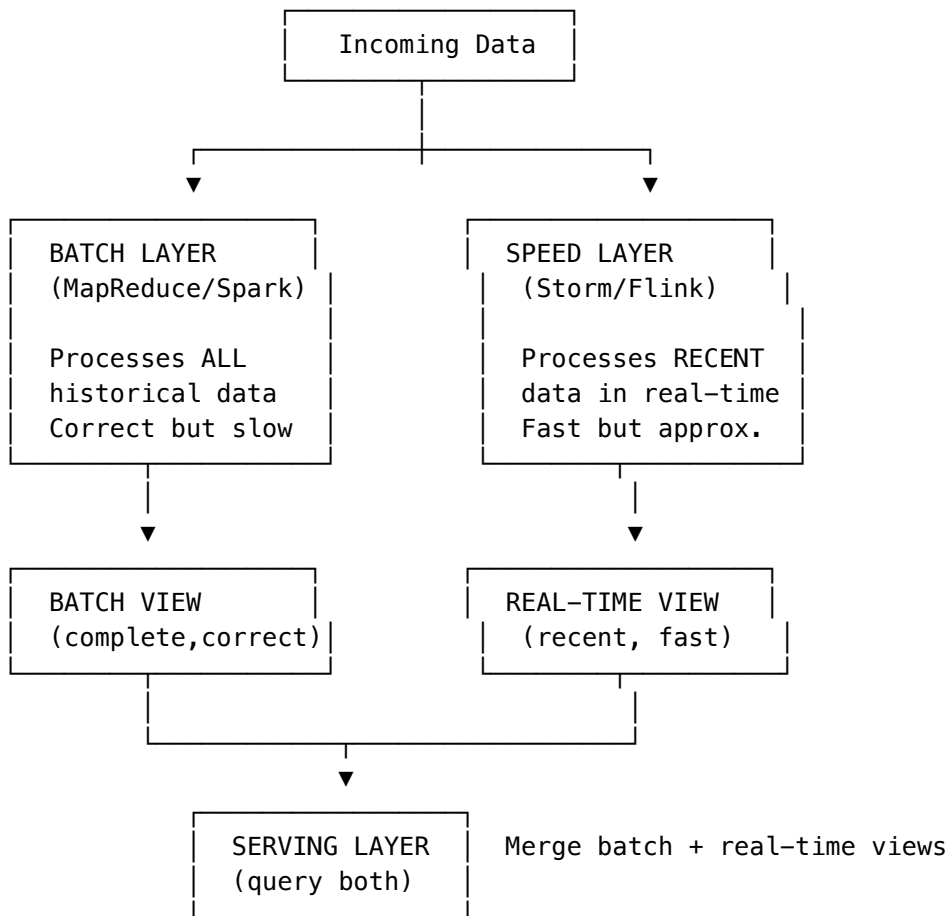
Works for many business cases (booking, orders).

Trade-off: timeliness vs. integrity vs. availability.

Not everything needs synchronous enforcement.

12.4 Lambda and Kappa Architecture

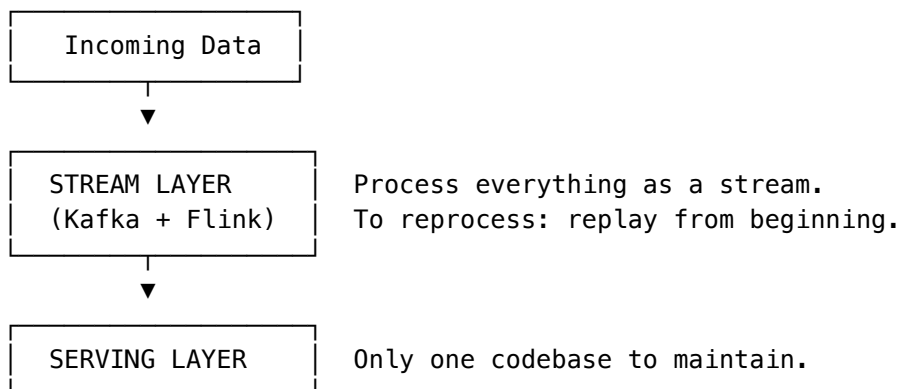
LAMBDA ARCHITECTURE



Problems: Must maintain TWO codebases (batch + stream).
Merging outputs is complex. Batch layer is slow to update.

KAPPA ARCHITECTURE (simpler alternative)

Use ONLY a stream processing layer. No separate batch layer.



Requires: long-term log retention (Kafka with infinite retention or tiered storage).

12.5 End-to-End Argument for Databases

END-TO-END ARGUMENT
The "end-to-end argument" (Saltzer, Reed, Clark, 1984): Reliability features at low levels of the stack (TCP checksums, database transactions) are NOT sufficient – you still need end-to-end checks at the APPLICATION level. Example: TCP guarantees in-order, reliable delivery. BUT: an application-level bug could still corrupt data. You need an APPLICATION-LEVEL checksum to detect this. For databases: A database provides ACID transactions. BUT: the application may still have bugs that cause inconsistencies (write skew, race conditions in app logic). You need END-TO-END verification (auditing, reconciliation). AUDIT TRAIL: If you store an immutable event log, you can VERIFY that derived state matches the events. If there's a bug, you can detect it and recompute. Event sourcing + deterministic derivation + auditing = a self-verifying, repairable data system.

12.6 Doing the Right Thing — Ethics

ETHICAL CONSIDERATIONS FOR DATA SYSTEMS

As engineers building data-intensive applications, we must consider the **IMPACT** of our systems on people:

PREDICTIVE ANALYTICS AND BIAS:

- ML models trained on biased data → biased predictions
- Credit scoring, hiring, criminal justice, insurance
- Feedback loops: biased predictions → biased actions → biased training data → worse predictions

PRIVACY AND SURVEILLANCE:

- Data collected for one purpose used for another
- Users rarely understand what data is collected
- "If you have nothing to hide" is a fallacy
- GDPR, CCPA attempt to address this legally

DATA AS AN ASSET vs DATA AS A LIABILITY:

- More data = more useful for analytics
- More data = bigger target for breaches
- Data minimization: collect only what you need
- Right to deletion: can you actually delete it from all backups, caches, logs, and derived systems?

ACCOUNTABILITY:

- Automated decisions affect real lives
- Right to explanation (GDPR Article 22)
- Engineers have a professional responsibility

"Just because it's technically possible doesn't mean it's ethically acceptable."

12.7 Interview Questions

12.7.1 Q1: What is the dual-write problem and how do you solve it?

The dual-write problem occurs when an application writes to two systems (e.g., database and search index) independently. If one write succeeds and the other fails, or if two concurrent writes arrive in different orders at the two systems, they go out of sync. The solution is to use a **single source of truth** and derive all other representations. Write to the database, and use **Change Data Capture (CDC)** to propagate changes to search indexes, caches, and other derived systems via an event log (Kafka). This ensures a single total order of changes.

12.7.2 Q2: Compare Lambda and Kappa architectures.

Lambda architecture runs both a batch layer (for correctness using all historical data) and a speed layer (for low-latency using recent data), then merges their outputs. It provides accurate results but requires maintaining two separate codebases doing the same computation. **Kappa architecture** uses only a stream processing layer — to reprocess historical data, replay the event log from the beginning. It's simpler (one codebase) but requires the event log to retain data long-term. Kappa is increasingly preferred due to improvements in stream processing frameworks (Flink, Kafka Streams).

12.7.3 Q3: What does “unbundling the database” mean?

Traditional databases bundle many features: storage, indexing, queries, transactions, replication, access control. “Unbundling” means decomposing these into separate specialized systems (e.g., PostgreSQL for storage, Elasticsearch for search, Redis for caching, Kafka for replication/CDC) and composing them together. The event log (Kafka) plays the role of the database’s internal WAL but at a system level. This gives flexibility to use the best tool for each job but adds operational complexity and trades strong consistency for eventual consistency.

12.7.4 Q4: Why is end-to-end verification important even with ACID transactions?

ACID transactions guarantee correctness at the database level, but they don’t protect against: application-level bugs, race conditions in application logic, data corruption in network transit, or issues in derived systems downstream. The **end-to-end argument** says reliability checks must also exist at the application level. Techniques include: immutable event logs for audit trails, checksums on application data, periodic reconciliation between source and derived systems, and deterministic derivation (so you can recompute and verify derived data matches the event log).

12.7.5 Q5: What ethical considerations should engineers think about when building data systems?

- (1) **Bias and fairness:** ML models can perpetuate and amplify biases in training data, affecting credit, hiring, and justice.
- (2) **Privacy:** Data collected for one purpose may be used for surveillance or sold; users rarely understand the full scope.
- (3) **Data as liability:** Every piece of data collected is a breach risk; practice data minimization.
- (2) **Right to deletion:** With CDC, event logs, caches, and backups, true deletion is technically very difficult.
- (5) **Accountability:** Automated decisions affect real lives; engineers share responsibility for the impact of their systems.

12.7.6 Q6: How do you enforce a uniqueness constraint in an event-driven architecture?

Three approaches: (1) **Log-based:** Route all requests for the same key to the same Kafka partition → single consumer processes them sequentially → first wins. (2) **Consensus:** Use a linearizable store (ZooKeeper, etcd) with compare-and-set to atomically claim the unique value. (3) **Compensating transactions:** Accept all requests optimistically, detect duplicates asynchronously, and compensate (e.g., cancel one, notify the user). The choice depends on whether the business can tolerate a brief window of inconsistency.

Based on Chapter 12 of “Designing Data-Intensive Applications” by Martin Kleppmann